Liberté • Égalité • Fraternité
RÉPUBLIQUE FRANÇAISE

MINISTÈRE CHARGÉ DE L'EMPLOI

DOSSIER PROFESSIONNEL (DP)

Nom de naissance	Berben
Nom d'usage	Berben
Prénom	Mathias
Adresse	16 rue de la Redoute, 67500 Haguenau

Titre professionnel visé

Développeur Web et Web Mobile (DWWM) – Titre professionnel de niveau 5

MODALITÉ D'ACCÈS :

- ☒ Parcours de formation
- ☐ Validation des Acquis de l'Expérience (VAE)

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel. **Ce titre est délivré par le Ministère chargé de l'emploi.**

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen**.

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.

Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi]

Ce dossier comporte :

- pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- une déclaration sur l'honneur à compléter et à signer ;
- des documents illustrant la pratique professionnelle du candidat (facultatif)
- des annexes, si nécessaire.

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.

http://travail-emploi.gouv.fr/titres-professionnels

Sommaire

Exemples de pratique professionnelle

Activité-type 1 : Développer la partie front-end d'une application web ou web mobile sécurisée p. 5

- Portfolio de Thomas, graphiste (HTML / CSS) p. 5
- Site de l'association ACT NOW (WordPress) p. 13
- Application To-Do List (HTML / CSS / JavaScript + API REST) p. 19

Activité-type 2 : Développer la partie back-end d'une application web ou web mobile sécurisée p. 31

- Site du cabinet dentaire du Dr Dupont (PHP / MySQL, MVC sans framework) p. 31
- Plateforme e-commerce InnovShop (Symfony / Doctrine) p. 50
- Base de données de la salle de concerts SoundWave (SQL) p. 73

Titres, diplômes, CQP, attestations de formation *(facultatif)* p. 81

Déclaration sur l'honneur p. 82

Documents illustrant la pratique professionnelle *(facultatif)* p. 83

Annexes *(Si le RC le prévoit)* p. 84

EXEMPLES DE PRATIQUE PROFESSIONNELLE

Correspondance entre les exemples et les compétences du titre

Exemple (projet)	CP 1	CP 2	CP 3	CP 4	CP 5	CP 6	CP 7	CP 8
Activité-type 1								
Ex. 1 – Portfolio de Thomas, graphiste (HTML / CSS)	✔	✔	✔					✔
Ex. 2 – Site de l'association ACT NOW (WordPress)	✔		✔					✔
Ex. 3 – Application To-Do List (HTML / CSS / JavaScript + API REST)	✔	✔	✔	✔				✔
Activité-type 2								
Ex. 1 – Site du cabinet dentaire du Dr Dupont (PHP / MySQL, MVC sans framework)	✔	✔			✔	✔	✔	✔
Ex. 2 – Plateforme e-commerce InnovShop (Symfony / Doctrine)	✔			✔	✔	✔	✔	✔
Ex. 3 – Base de données de la salle de concerts SoundWave (SQL)	✔				✔	✔		

CP 1 : Installer et configurer son environnement de travail
CP 2 : Maquetter des interfaces utilisateur
CP 3 : Réaliser des interfaces utilisateur statiques
CP 4 : Développer la partie dynamique des interfaces
CP 5 : Mettre en place une base de données relationnelle
CP 6 : Développer des composants d'accès aux données SQL et NoSQL
CP 7 : Développer des composants métier côté serveur
CP 8 : Documenter le déploiement d'une application

Activité-type 1 — Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°1 — *Portfolio de Thomas, graphiste (HTML / CSS)*

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

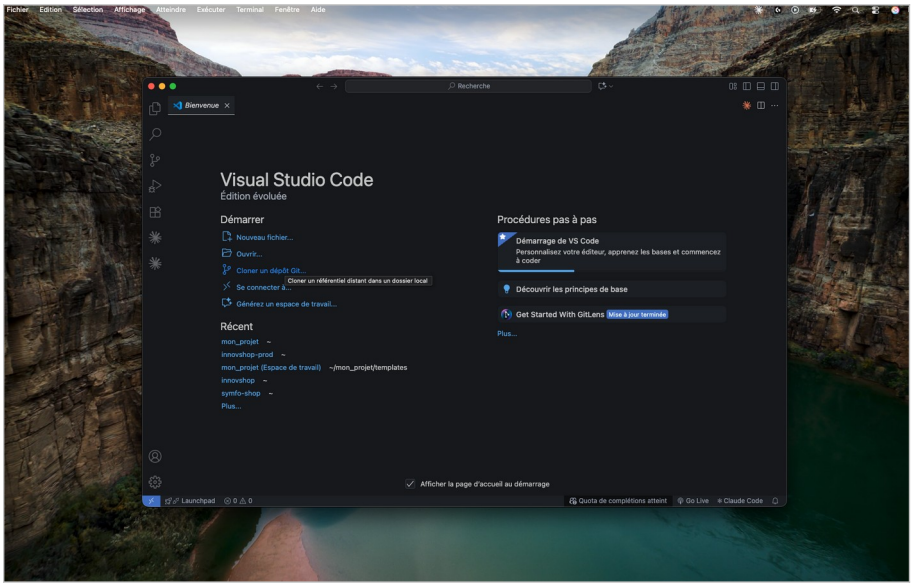
Compétences mises en œuvre : CP 1 Installer et configurer son environnement de travail · CP 2 Maquetter des interfaces utilisateur · CP 3 Réaliser des interfaces utilisateur statiques · CP 8 Documenter le déploiement d'une application

Le portfolio est mon premier projet de formation au Learning Campus. Il m'a permis d'apprendre les bases du développement front-end : préparer mon poste de travail, partir d'une maquette Figma, l'intégrer en HTML et CSS de façon responsive, puis mettre le site en ligne.

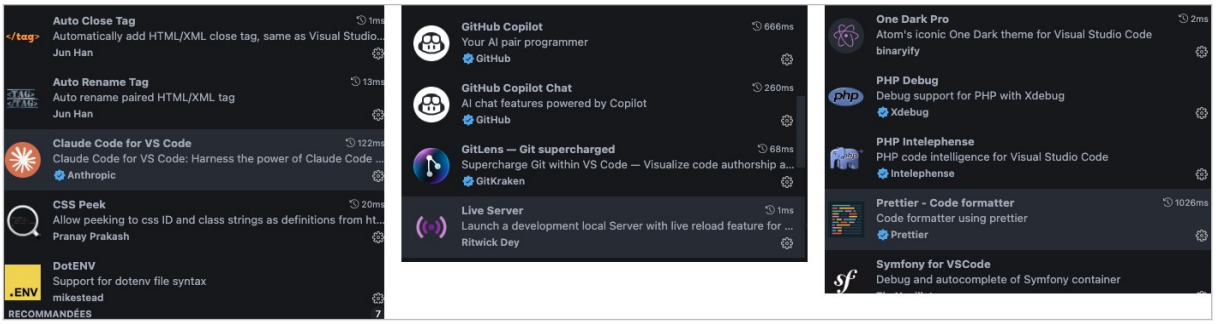
Préparation de l'environnement de travail (CP 1)

J'utilise **Visual Studio Code** comme éditeur principal, configuré avec des extensions adaptées aux langages de la formation :

- **Auto Close Tag** et **Auto Rename Tag** : fermeture et renommage automatiques des balises HTML/XML ;
- **CSS Peek** : accès direct à la règle CSS d'une classe ou d'un id depuis le HTML ;
- **Live Server** : serveur local avec rechargement automatique pendant le développement front-end ;
- **Prettier** : formatage automatique et homogène du code ;
- **PHP Intelephense** et **PHP Debug** (Xdebug) : autocomplétion, analyse et débogage du code PHP ;
- **Symfony for VSCode** et **DotENV** : support du framework Symfony et des fichiers d'environnement .env ;
- **GitLens** : visualisation de l'historique Git et de l'auteur de chaque ligne ;
- **GitHub Copilot / Copilot Chat** et **Claude Code** : assistants IA utilisés pour obtenir des explications et vérifier mon code ;
- **One Dark Pro** : thème sombre pour le confort de lecture.



Écran d'accueil de Visual Studio Code et projets récents



Extensions installées dans Visual Studio Code

J'ai versionné le projet avec **Git** et l'ai publié sur **GitHub** (dépôt Mberben/portfolio).

Maquette Figma (CP 2)

Dans le cadre du projet **portfolio** du Learning Campus, j'ai travaillé sur la maquette **Figma** du site de « Thomas », graphiste, avant de l'intégrer en HTML et CSS. La maquette définit l'organisation de la page, la charte graphique et l'adaptation aux différentes tailles d'écran.

Analyse du besoin et structure de la page

Le portfolio doit présenter le graphiste, mettre en avant ses travaux et permettre de le contacter. La page unique est découpée en zones :

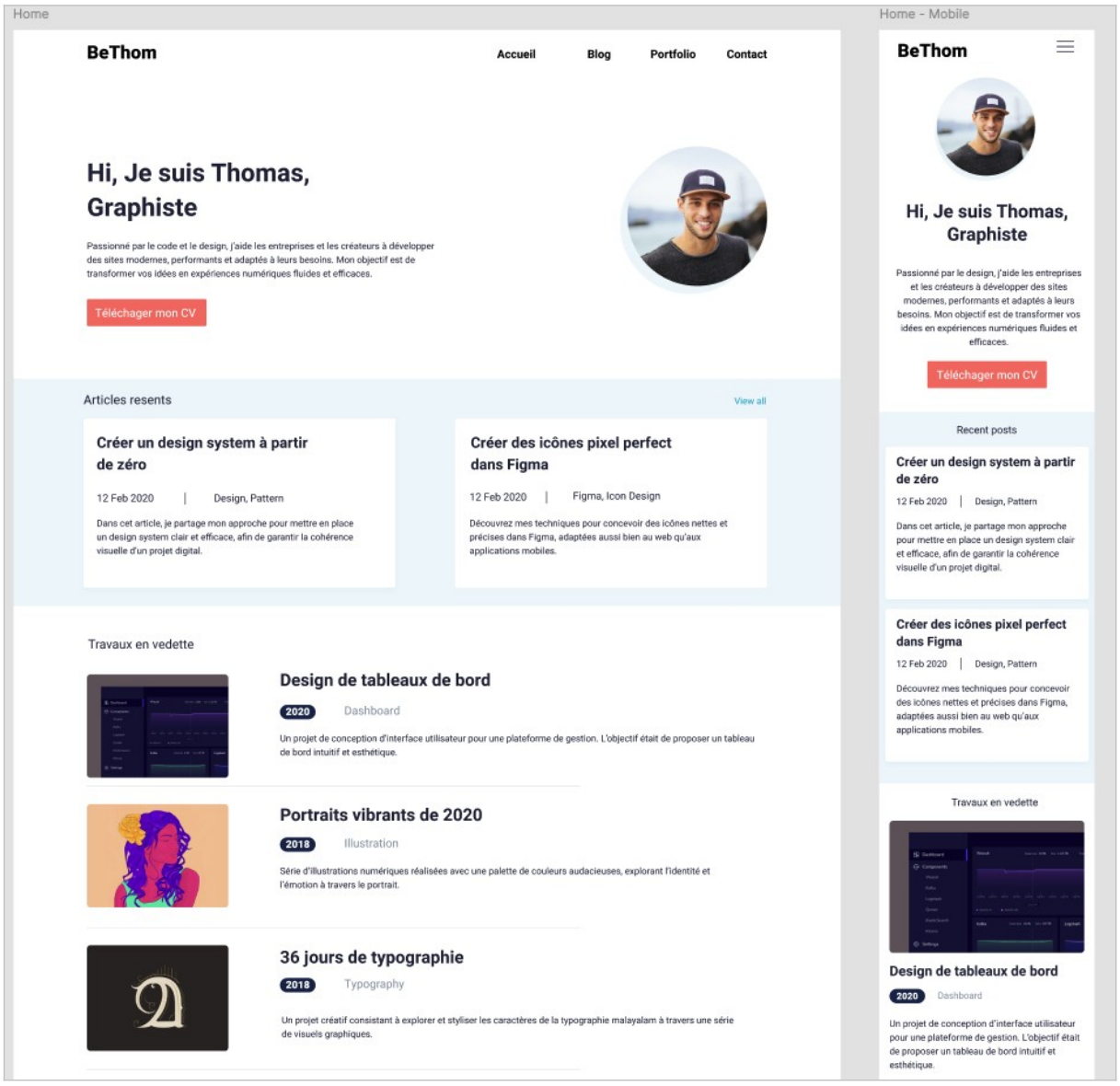
- **navigation** : logo et liens vers Accueil, Blog, Portfolio et Contact ;
- **en-tête** : présentation, bouton « Télécharger mon CV » et photo ;
- **articles récents** : deux cartes d'articles (titre, date, catégories, résumé) ;
- **travaux en vedette** : trois projets avec image, année, type et description ;
- **contact** : formulaire (nom, email, type de prestation, budget, message) et coordonnées ;
- **pied de page** : icônes des réseaux sociaux et copyright.

Charte graphique

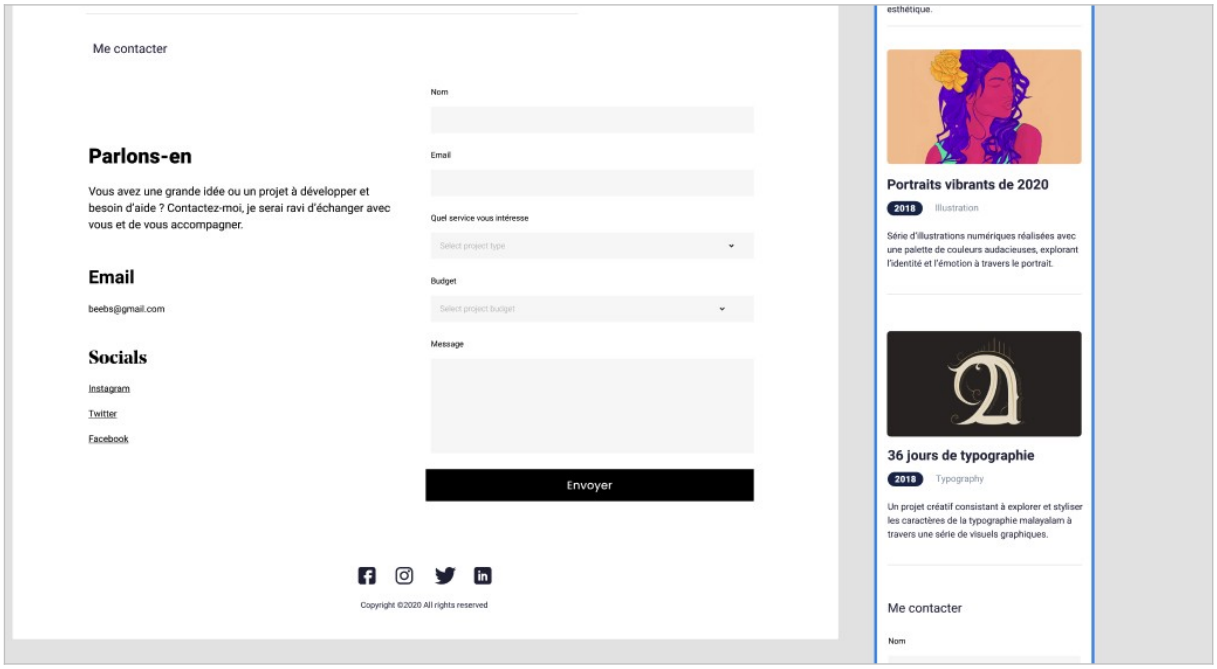
- **Typographie** : police Heebo, titres en gras (44 px pour le titre principal), texte courant en 16 px ;
- **Couleurs** : texte bleu nuit (#21243D), accent corail (#FF6464) pour le bouton principal, fond bleu très clair (#EDF7FA) pour la section des articles ;
- **Mise en page** : contenu centré sur une largeur maximale de 1200 px, cartes blanches à coins arrondis.

Versions desktop et mobile

La maquette prévoit une version ordinateur et une version mobile. Sur mobile, les liens du menu sont remplacés par une icône hamburger, la photo passe au-dessus du texte de présentation, et les articles et projets s'affichent sur une seule colonne.



Maquette Figma du portfolio : page d'accueil en version desktop (Home) et mobile (Home - Mobile)



Maquette Figma : section contact et pied de page, desktop et mobile

Intégration HTML / CSS (CP 3)

Ce premier projet m'a permis d'apprendre les bases de l'intégration web. J'ai intégré en HTML et CSS la maquette d'un portfolio de graphiste fournie par la formation : barre de navigation, en-tête avec photo et bouton « Télécharger mon CV », articles de blog, travaux en vedette, formulaire de contact et pied de page avec les réseaux sociaux.

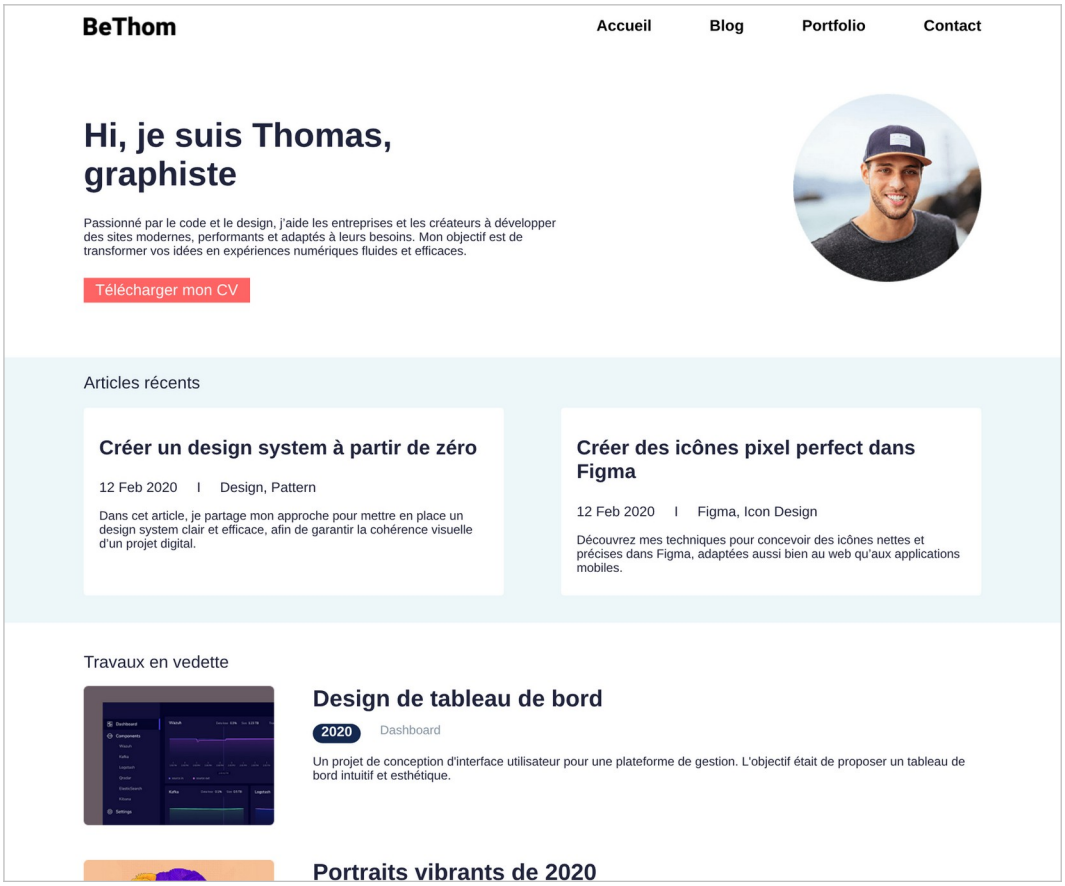
- **Structure sémantique** : balises nav, header, section, article, form et footer, un seul titre h1, une hiérarchie de titres h2 / h3 cohérente, des textes alternatifs sur les images ;
- **Mise en page avec Flexbox** : chaque zone (navigation, en-tête, liste d'articles, cartes de projets, formulaire) est organisée en flex, avec des espacements et des alignements fidèles à la maquette, sur une largeur maximale de 1200 px ;
- **Typographie** : police Heebo chargée depuis Google Fonts, avec une police de secours ;
- **Formulaire de contact** : champs nom, email, type de prestation, budget et message, avec les types adaptés (email, liste déroulante, zone de texte).

portfolio/index.html

```
<nav>
  <img src="Image/BeThom.png" alt="Logo du sites" />

  <div id="nav-links">
    <a class="nav-link" href="index.html">Accueil</a>
    <a class="nav-link" href="#blog">Blog</a>
    <a class="nav-link" href="#projet">Portfolio</a>
    <a class="nav-link" href="#contact">Contact</a>
    <a href="nav-links" id="menu-button"><img src="Image/menu.png" alt="Menu"></a>
  </div>
</nav>
<header>
  <div id="Header-content">
    <h1>
      Hi, je suis Thomas,<br />
      graphiste
    </h1>
    <p id="Subtitle">
      Passionné par le code et le design, j'aide les entreprises et les
      créateurs à développer des sites modernes, performants et adaptés à
      leurs besoins. Mon objectif est de transformer vos idées en
      expériences numériques fluides et efficaces.
    </p>
    <a href=" #" id="button-cv">Télécharger mon CV</a>
  </div>
  <div>
    <img id="photo" src="Image/Ellipse1.png" alt="Photo de Thomas" />
  </div>
</header>
```

Extrait de index.html : navigation et en-tête en balises sémantiques



Portfolio – version ordinateur (en-tête, articles récents et travaux en vedette)

Adaptation au mobile (responsive)

J'ai ajouté la balise meta viewport et une **media query** (max-width: 700px) : le menu est remplacé par une icône hamburger, l'en-tête et les cartes passent en colonne, et l'ordre de certains blocs est inversé pour que la photo apparaisse en premier.

portfolio/style.css

```css
@media (max-width: 700px) {
  #menu-button {
    display: block;
  }

  .nav-link {
    display: none;
  }

  .projet {
    flex-direction: column;
  }

  div#Header-content {
    flex-direction: column-reverse;
  }

  header {
    flex-direction: column-reverse;
    text-align: center;
  }

  div#Header-content {
    width: 100%;
  }

  h2 {
    text-align: center;
  }

  .date-category {
    gap: 5px;
  }

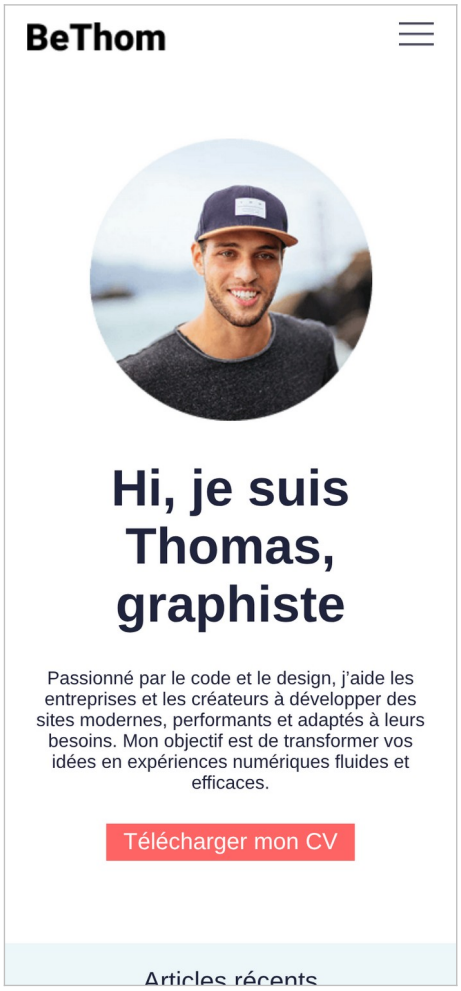
  .article-blog {
    width: 100%;
    max-width: none;
  }

  #Articles {
    flex-direction: column;
    gap: 20px;
  }

  #coordonnees {
    display: none;
  }

    .projet-image {
      width: 100%;
    }
  }
```

Extrait de style.css : media query pour les écrans de moins de 700 px



Portfolio – version mobile

J'ai comparé le rendu à la maquette et testé l'affichage dans l'outil de simulation d'appareils du navigateur. J'ai publié le site sur **GitHub Pages** (https://mberben.github.io/portfolio/) et sur **alwaysdata** (https://berben.alwaysdata.net/).

Mise en ligne (CP 8)

J'ai publié le site sur **GitHub Pages** (https://mberben.github.io/portfolio/) et déposé les fichiers statiques sur mon hébergement **alwaysdata**, où il est servi à la racine du domaine (https://berben.alwaysdata.net/).

2. Précisez les moyens utilisés :

- Visual Studio Code et ses extensions (Live Server, Prettier, Auto Close Tag, CSS Peek…)
- Figma : maquette desktop et mobile, relevé des mesures, couleurs et polices
- HTML5, CSS3 (Flexbox, media queries), police Heebo (Google Fonts)
- Outils de développement du navigateur (inspecteur, mode responsive)
- Git, GitHub et GitHub Pages ; hébergement alwaysdata

3. Avec qui avez-vous travaillé ?

J'ai réalisé ce projet seul, à partir de la maquette et des consignes du Learning Campus, avec les retours de mon formateur sur la structure HTML et le responsive.

4. Contexte

Nom de l'entreprise, organisme ou association	**Learning Campus**
Chantier, atelier, service	Projet Portfolio : maquette et intégration HTML/CSS
Période d'exercice	**Du** 04/05/2026 **au** 25/05/2026

5. Informations complémentaires *(facultatif)*

Site en ligne : https://berben.alwaysdata.net/ – dépôt https://github.com/Mberben/portfolio

Activité-type 1

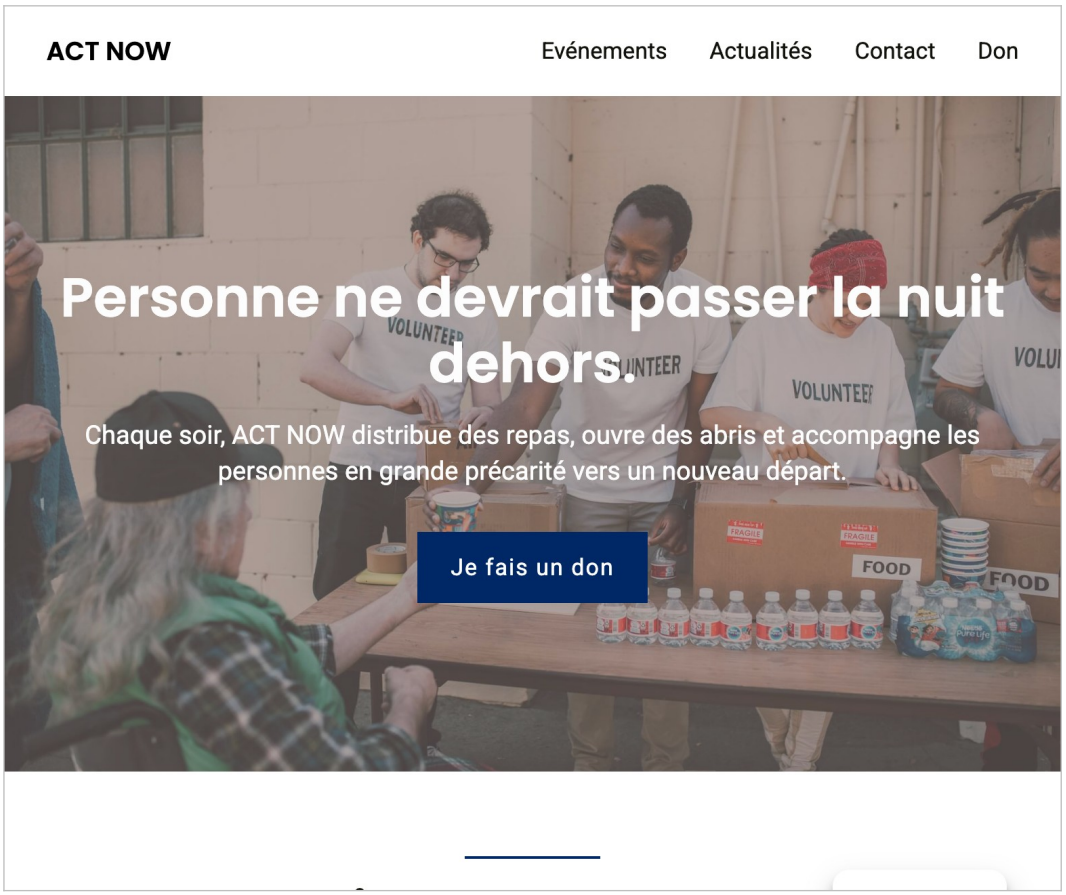
Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°2 | *Site de l'association ACT NOW (WordPress)*

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Compétences mises en œuvre : CP 1 Installer et configurer son environnement de travail · CP 3 Réaliser des interfaces utilisateur statiques · CP 8 Documenter le déploiement d'une application

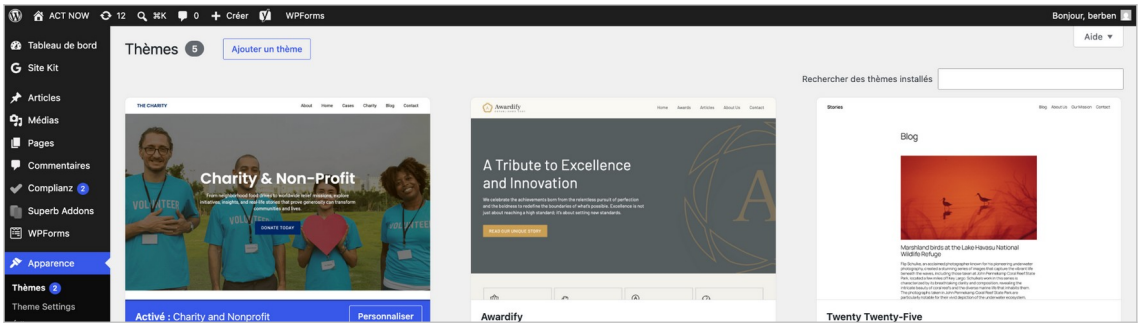
J'ai ensuite créé le site vitrine d'**ACT NOW**, une association fictive qui vient en aide aux personnes sans abri. L'objectif était de comprendre le fonctionnement d'un CMS : installation, thème, pages, articles, menus et extensions.



Page d'accueil du site ACT NOW : en-tête, menu et bandeau d'appel au don

Installation et configuration de WordPress (CP 1 et CP 8)

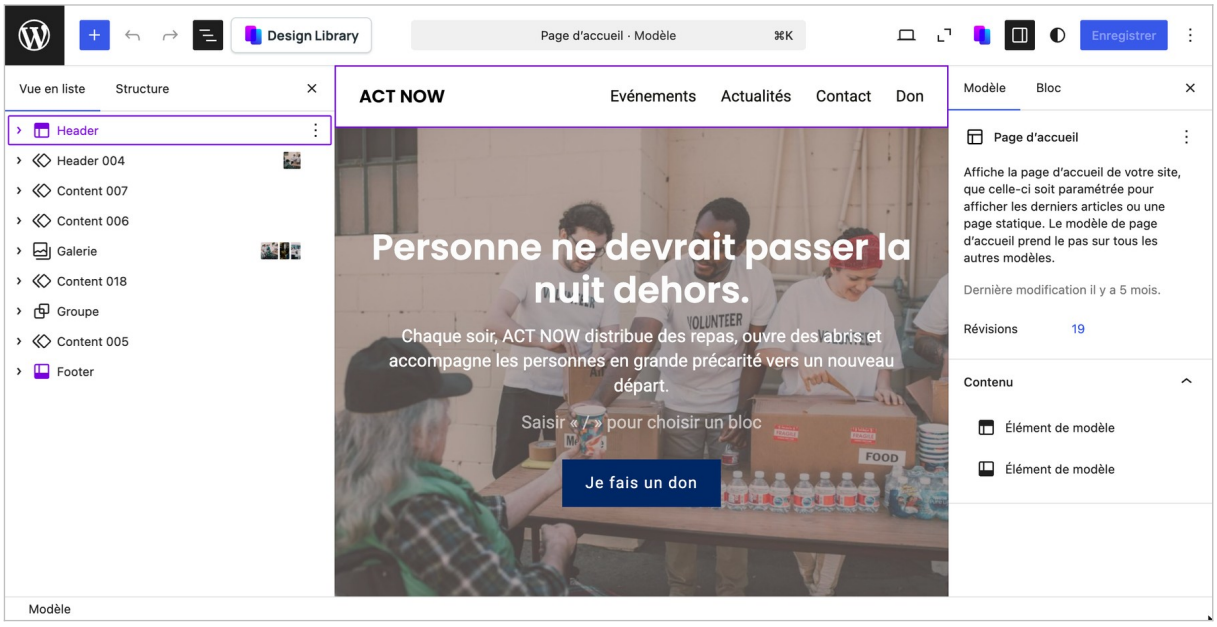
Installation et thème. J'ai installé **WordPress** sur l'hébergement alwaysdata (https://berben.alwaysdata.net/wordpress/) avec sa propre base de données. J'ai choisi le thème **Charity and Nonprofit**, un thème « blocs » adapté aux associations, que j'ai personnalisé (couleurs, typographies, logo, en-tête et pied de page).



Back-office : thème Charity and Nonprofit activé

Construction des pages (CP 3)

Construction de la page d'accueil. La page d'accueil est construite dans l'**éditeur de site** (Full Site Editing), directement dans le modèle « Page d'accueil ». Je l'ai assemblée à partir de compositions de blocs (patterns) de l'extension **Superb Addons**, que j'ai adaptées au contenu de l'association : en-tête, bandeau avec le message « Personne ne devrait passer la nuit dehors » et le bouton « Je fais un don », présentation de l'association, trois services (distribuer des repas, offrir un abri, accompagner vers la réinsertion), témoignages, galerie photo, newsletter, actualités et présentation du fondateur. La vue en liste de l'éditeur montre cette structure ; l'historique compte 19 révisions du modèle.

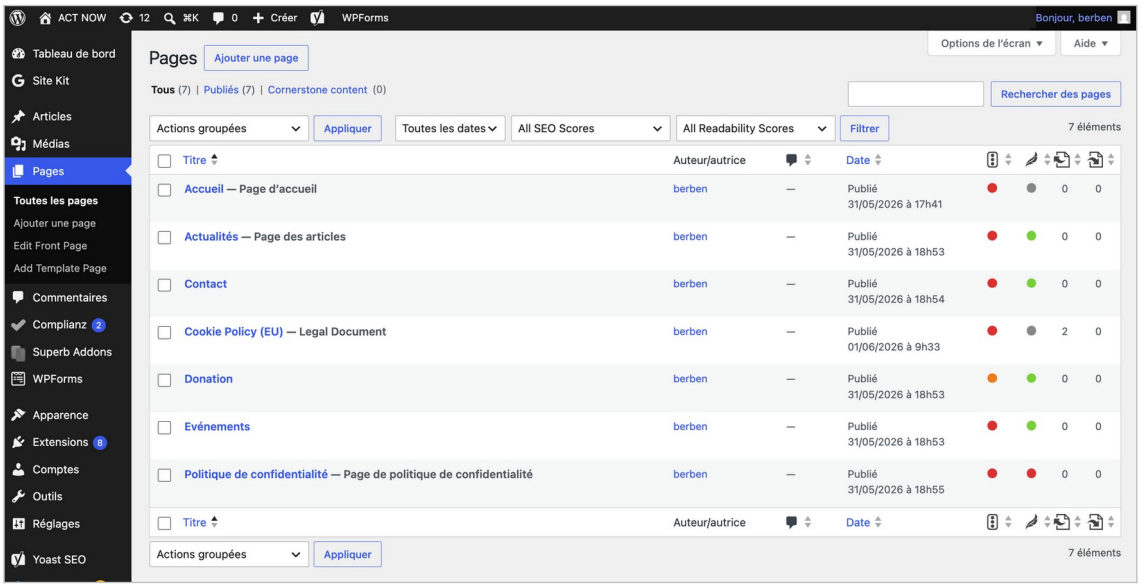


Éditeur de site : modèle « Page d'accueil » et vue en liste des blocs



Section « Qui sommes-nous » et cartes des services

Pages, articles et menus. J'ai créé 7 pages (Accueil, Actualités, Contact, Donation, Événements, Politique de confidentialité, Cookie Policy) et défini Accueil comme page d'accueil et Actualités comme page des articles. Trois **articles** alimentent la rubrique Actualités (réinsertion, collecte d'hiver, maraudes). Les menus d'en-tête (Événements, Actualités, Contact, Don) et de pied de page (Accueil, Politiques de confidentialité, Contact) sont paramétrés dans l'éditeur de site.



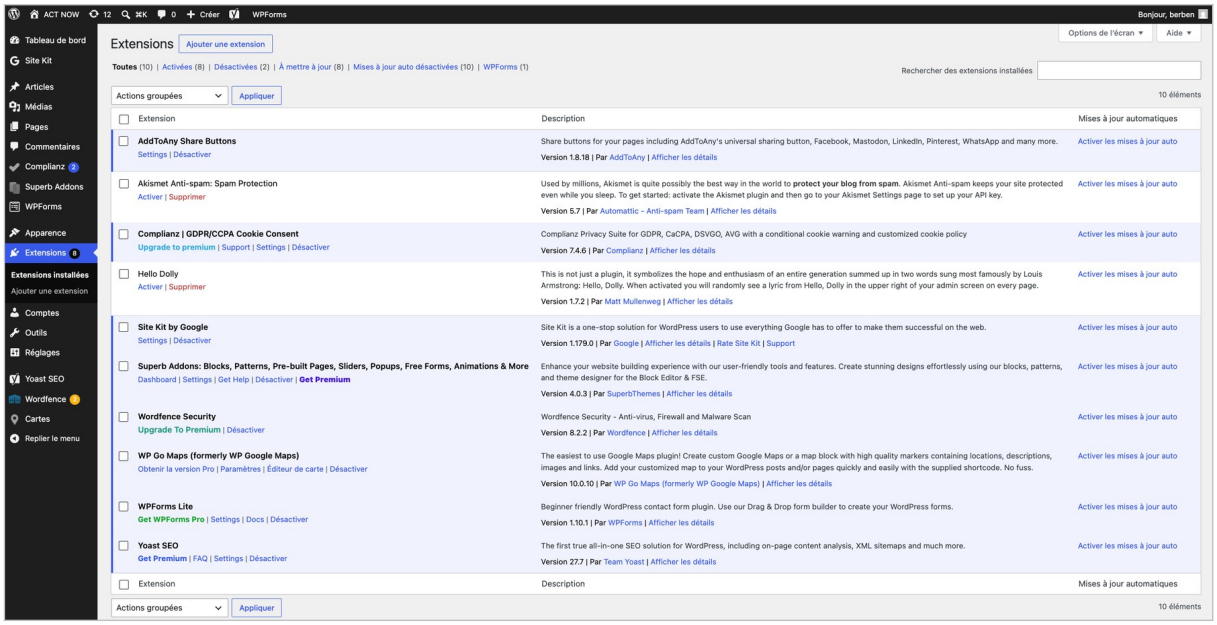
Back-office : liste des pages du site

Extensions et conformité RGPD (CP 3)

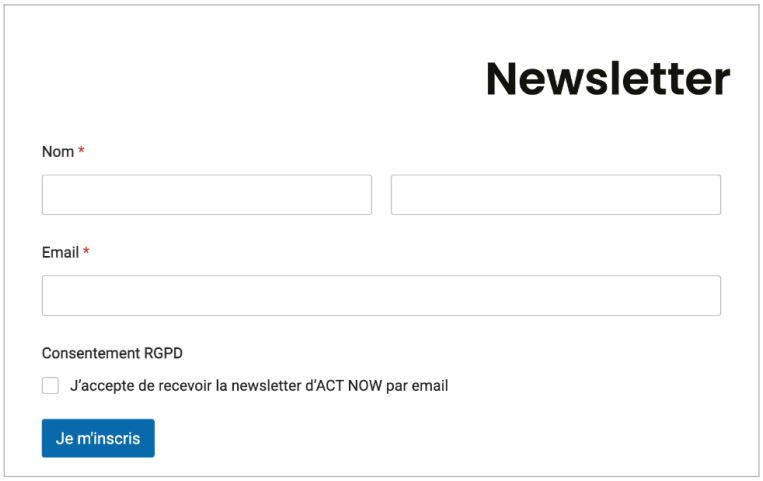
Extensions. J'ai installé et configuré plusieurs extensions selon les besoins du site :

- **WPForms Lite** : formulaires d'inscription à la newsletter (nom, email, case de consentement RGPD obligatoire), de contact et de don ;
- **Complianz** : bannière de consentement aux cookies (accepter, refuser, voir les préférences) et génération de la page Cookie Policy ;

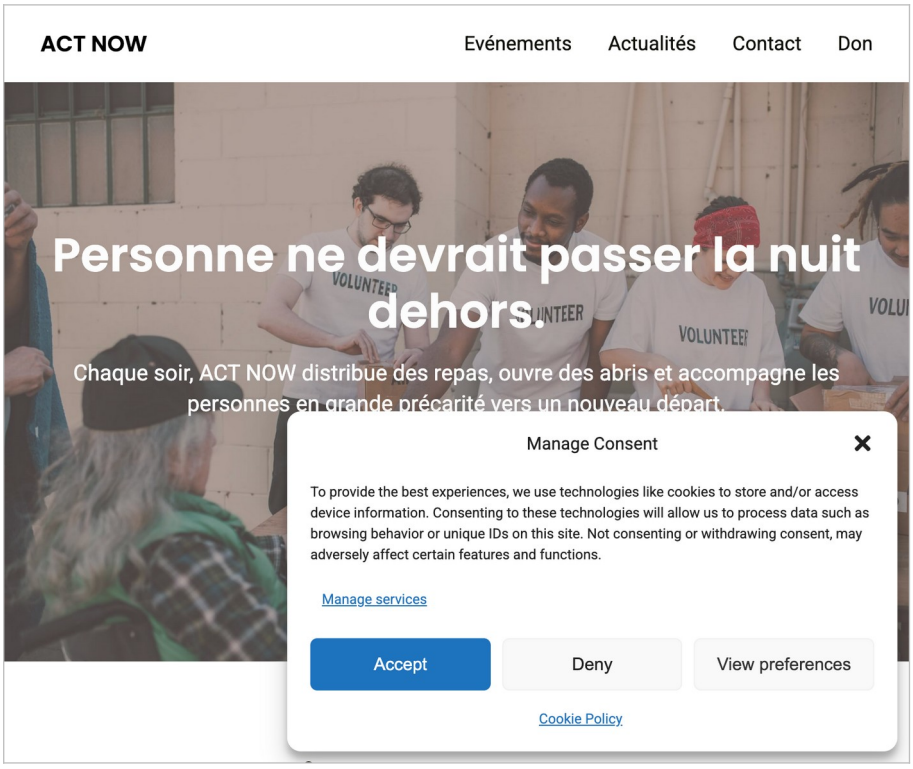
- **Yoast SEO** : référencement des pages (titre, méta-description, analyse de lisibilité) ;
- **Site Kit by Google** : liaison avec les outils de statistiques de Google ;
- **Wordfence Security** : pare-feu et analyse de sécurité ;
- **Superb Addons** : blocs et compositions pour la mise en page ; **WP Go Maps** : carte ; **AddToAny** : boutons de partage sur les réseaux sociaux.



Back-office : extensions installées et activées



Formulaire d'inscription à la newsletter (WPForms) avec consentement RGPD



Bannière de consentement aux cookies (Complianz)

2. Précisez les moyens utilisés :

- WordPress : éditeur de blocs et éditeur de site (Full Site Editing)
- Thème Charity and Nonprofit ; extensions Superb Addons, WPForms Lite, Complianz, Yoast SEO, Site Kit by Google, Wordfence, WP Go Maps et AddToAny
- Hébergement alwaysdata (base de données MySQL dédiée)
- Banque d'images libres de droits pour les photos

3. Avec qui avez-vous travaillé ?

J'ai réalisé ce site seul, dans le cadre de la formation. Mon formateur m'a conseillé sur le choix du thème et la configuration des extensions.

4. Contexte

Nom de l'entreprise, organisme ou association	**Learning Campus**
Chantier, atelier, service	Projet ACT NOW : site vitrine WordPress d'une association
Période d'exercice	**Du** 26/05/2026 **au** 02/06/2026

5. Informations complémentaires *(facultatif)*

Site en ligne : https://berben.alwaysdata.net/wordpress/

Activité-type 1 — Développer la partie front-end d'une application web ou web mobile sécurisée

Exemple n°3 — *Application To-Do List (HTML / CSS / JavaScript + API REST)*

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Compétences mises en œuvre : CP 1 Installer et configurer son environnement de travail · CP 2 Maquetter des interfaces utilisateur · CP 3 Réaliser des interfaces utilisateur statiques · CP 4 Développer la partie dynamique des interfaces · CP 8 Documenter le déploiement d'une application

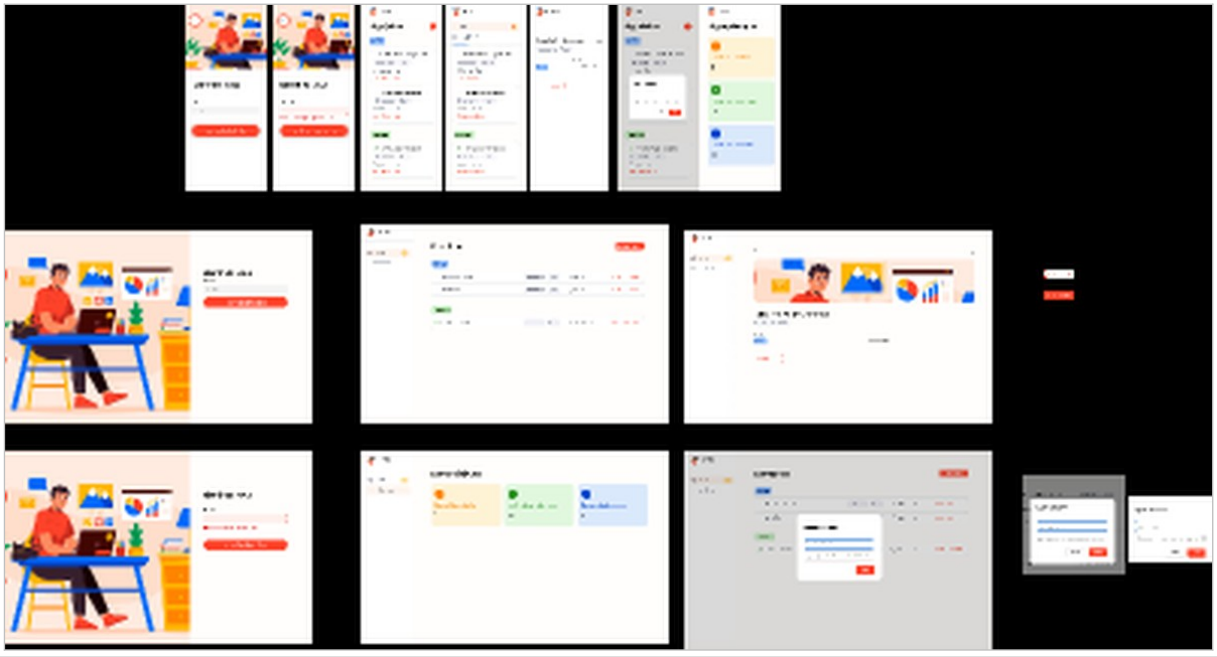
La To-Do List est une application de gestion de tâches : l'utilisateur s'identifie, consulte ses tâches classées par statut, en ajoute avec des étiquettes, affiche le détail d'une tâche et suit des statistiques. Les données proviennent d'une **API REST** Node.js/Express fournie par la formation.

Environnement de travail (CP 1)

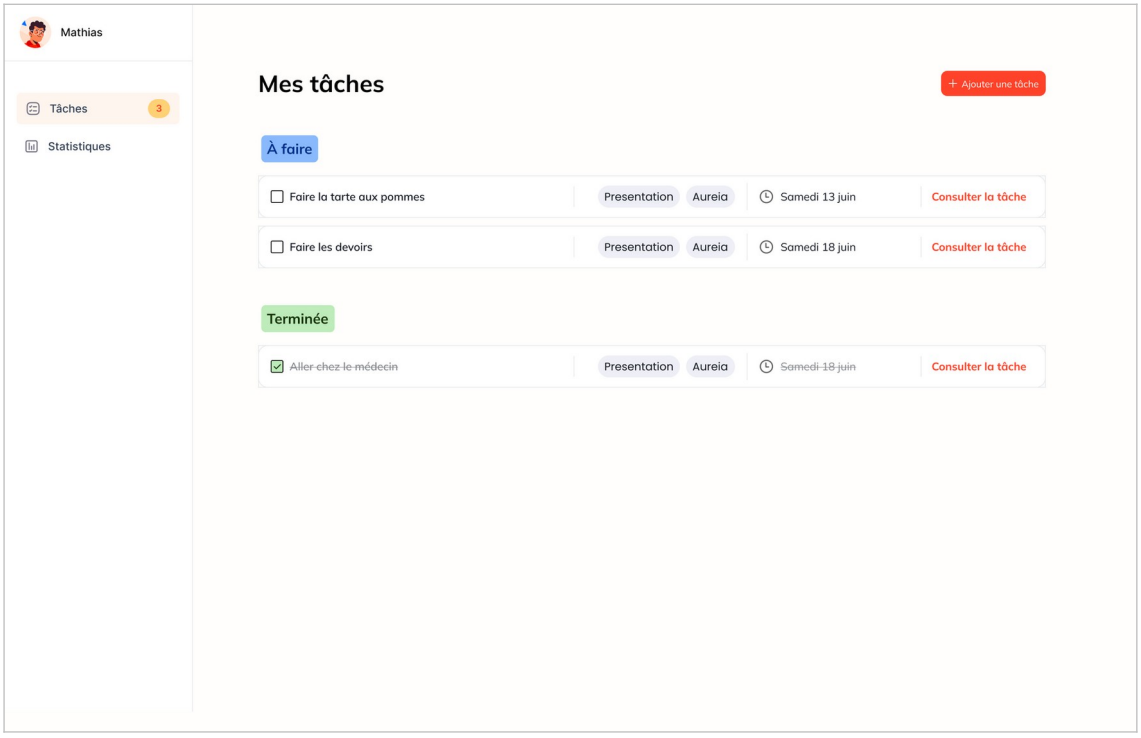
Pour le projet To-Do List, j'ai installé **Node.js** et **npm**, cloné le dépôt de l'API fournie par la formation, installé ses dépendances avec « npm install » (Express, CORS, Swagger) et lancé le serveur avec « npm start » sur http://localhost:3000. La documentation Swagger de l'API était consultable sur /api-docs.

Maquette Figma (CP 2)

Point de départ : la maquette Figma. J'ai d'abord conçu la maquette de l'application sur Figma, en version ordinateur (1440 px) et mobile (375 px) : identification (avec son état d'erreur), liste des tâches, fiche d'une tâche, fenêtre modale d'ajout et statistiques. Les pages internes partagent une barre latérale (avatar, prénom, menu Tâches avec un compteur et Statistiques) ; sur mobile, elle est remplacée par un menu « burger » et les tâches s'affichent en cartes empilées. J'ai défini une palette cohérente : orange pour l'action principale, badges de couleur pour les statuts (bleu pour « À faire », vert pour « Terminée »).



Vue d'ensemble de la maquette Figma de la To-Do List (écrans mobiles et desktop)



Maquette Figma – écran « Mes tâches » en version ordinateur

J'ai conçu chaque écran en version ordinateur (1440 px) puis mobile (375 px), y compris les états d'erreur (prénom non saisi). J'ai créé des composants pour les éléments répétés, comme les états « Actif » et « Inactif » des liens du menu, et prévu des repères visuels : tâches terminées barrées, badges de statut colorés, champs avec libellé et exemple de saisie.

Intégration HTML / CSS (CP 3)

J'ai intégré les maquettes en quatre pages : identification (index.html), tâches (Taches.html), détail (Details.html) et statistiques (Statistiques.html). Les pages internes partagent la même barre latérale ; la fenêtre d'ajout est une modale affichée par l'ajout d'une classe CSS. Une media query adapte toutes les pages au mobile : barre latérale remplacée par un menu burger, tâches en cartes empilées, cartes de statistiques les unes sous les autres.

Partie dynamique en JavaScript (CP 4)

J'ai écrit un fichier JavaScript par page, en JavaScript natif (fetch, async/await, manipulation du DOM).

J'ai intégré ces maquettes en quatre pages (index.html, Taches.html, Details.html, Statistiques.html), puis je les ai rendues dynamiques en écrivant **un fichier JavaScript par page**. Les données proviennent d'une **API REST** Node.js/Express fournie par la formation (routes GET, POST, PUT et DELETE sur /todos), que j'ai déployée sur Vercel.

Identification (login.js). Le formulaire est contrôlé en JavaScript : j'empêche l'envoi par défaut (event.preventDefault), je vérifie que le prénom n'est pas vide puis qu'il est reconnu, et j'affiche un message d'erreur sinon. Le prénom est enregistré dans le **localStorage**, puis l'utilisateur est redirigé vers la liste des tâches.

totolist/js/login.js

```
// Le formulaire
const form = document.getElementById("form-connection");
// Le champ de saisie
const champPrenom = document.getElementById("Prénom");
// Le paragraphe d'erreur
const messageErreur = document.getElementById("message-erreur");

form.addEventListener("submit", function(event) {

  // On empêche le rechargement automatique de la page
  event.preventDefault();

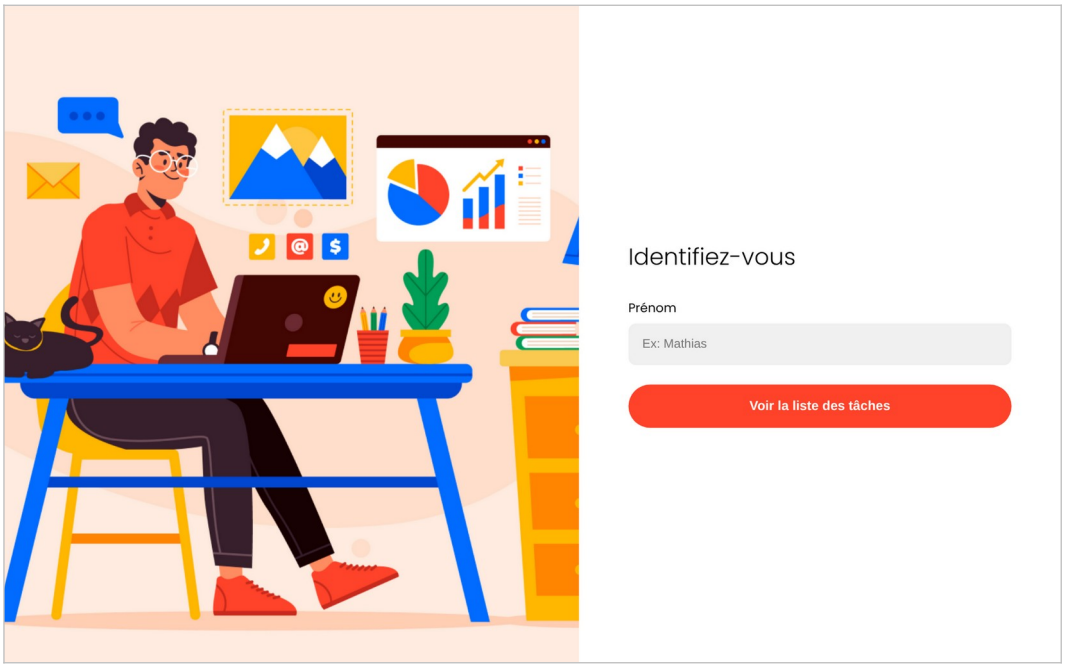
  messageErreur.style.display = "none";  // on repart d'un état propre

  // On récupère le prénom saisi, sans espaces inutiles
  const prenom = champPrenom.value.trim();

  // Validation : si le champ est vide, on affiche l'erreur et on s'arrête
  if (prenom === "") {
    messageErreur.style.display = "block";
    return;
  }
  // Validation : si le nom n'est pas Mathias, on affiche l'erreur et on s'arrête
if ( prenom !== "Mathias") {
  messageErreur.textContent = "Ce prénom n'est pas reconnu.";
  messageErreur.style.display = "block";
  return;
}
  // On enregistre le prénom dans le localStorage
  localStorage.setItem("prenom", prenom);

  // On redirige vers la page des tâches
  window.location.href = "Taches.html";
```

js/login.js : contrôle du formulaire et enregistrement du prénom



Page d'identification de l'application To-Do List

Liste des tâches (taches.js). La fonction asynchrone chargerTaches() récupère les tâches avec **fetch** et **await** (requête GET), convertit la réponse en JSON, puis génère une ligne HTML par tâche : case à cocher, titre, étiquettes, date

formatée en français avec toLocaleDateString et lien vers le détail. Chaque tâche est rangée dans « À faire » ou « Terminées » selon son statut.

totolist/js/taches.js

```
async function chargerTaches() {
  // fetch = on demande les données à l'API (le backend sur le port 3000).
  // await = on attend que la réponse arrive avant de continuer.
  const reponse = await fetch("http://localhost:3000/todos");
  // La réponse brute n'est pas directement exploitable : on la convertit
  // en objet JavaScript avec .json().
  const donnees = await reponse.json();
  // Les tâches ne sont pas à la racine : l'API les renvoie emballées
  // dans donnees[0].todolist. On va donc chercher le tableau à cet endroit précis.
  const taches = donnees[0].todolist;

  const listeAFaire = document.getElementById("liste-a-faire");
  const listeTerminee = document.getElementById("liste-terminee");

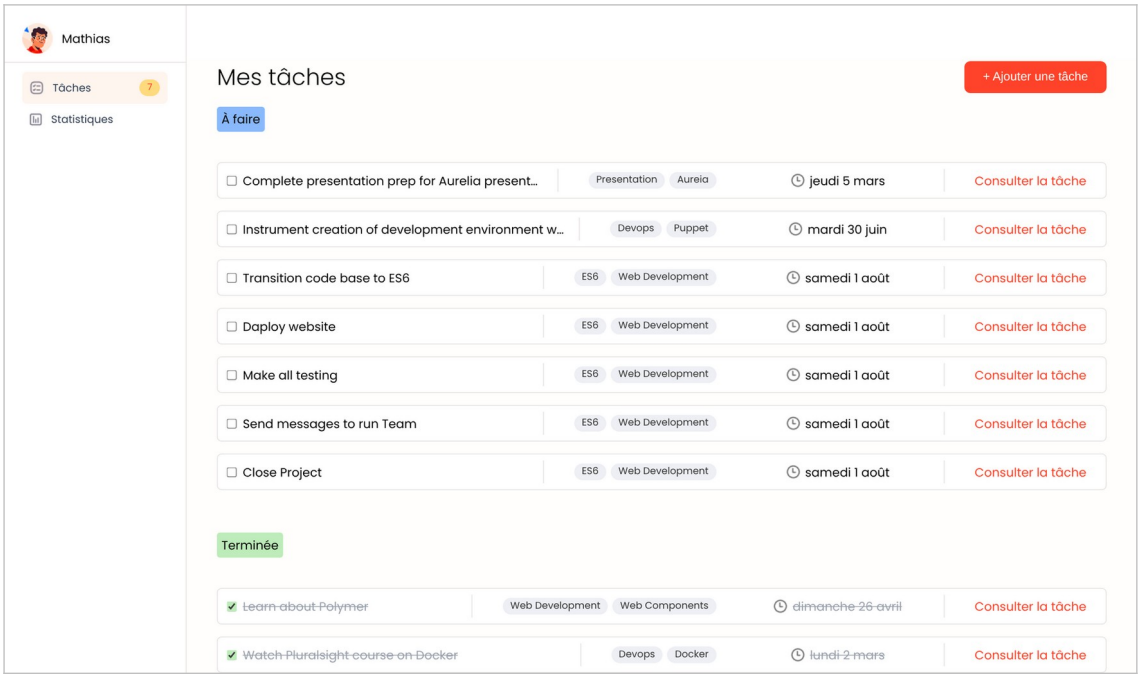
  // Pour chaque tâche, on génère une ligne HTML
  taches.forEach(function (tache) {
    const dateFormatee = new Date(tache.created_at).toLocaleDateString(
      "fr-FR",
      {
        weekday: "long",
        day: "numeric",
        month: "long",
      },
    );
      // 👇 NOUVEAU — tache.Tags est un TABLEAU. On le transforme en HTML :
    // un <span> "chip" par tag, puis on colle le tout avec join("").
    // (tache.Tags || []) protège les vieilles tâches sans champ Tags → pas de crash.
    const tagsHTML = (tache.Tags || [])
      .map(function (tag) {
        return `<span class="tache-tag">${tag}</span>`;
      })
      .join("");



    const ligne = `
      <li class="tache">
        <div class="tache-titre">
          <input type="checkbox" id="tache-${tache.id}" ${tache.is_complete ? "checked" : ""} />
          <label for="tache-${tache.id}">${tache.text}</label>
        </div>
        <div class="tache-tags">${tagsHTML}</div>
        <p class="tache-date">
          <img src="Images/proicons_clock.png" alt="" />${dateFormatee}
        </p>
        <a href="Details.html?id=${tache.id}" class="tache-lien">Consulter la tâche</a>
      </li>
    `;
    // On range la tâche dans la bonne liste selon son statut.
    // += pour AJOUTER la ligne aux lignes déjà présentes (sans écraser).
    if (tache.is_complete) {
      listeTerminee.innerHTML += ligne;
    } else {
      listeAFaire.innerHTML += ligne;
    }
  }); // ← le forEach se ferme ICI, proprement

  // ── Modale d'ajout ──
  // On récupère les éléments de la modale UNE SEULE FOIS, hors de la boucle,
  // car ce sont des éléments uniques
```

js/taches.js : récupération des tâches avec fetch et génération du HTML



Liste des tâches générée dynamiquement à partir de l'API

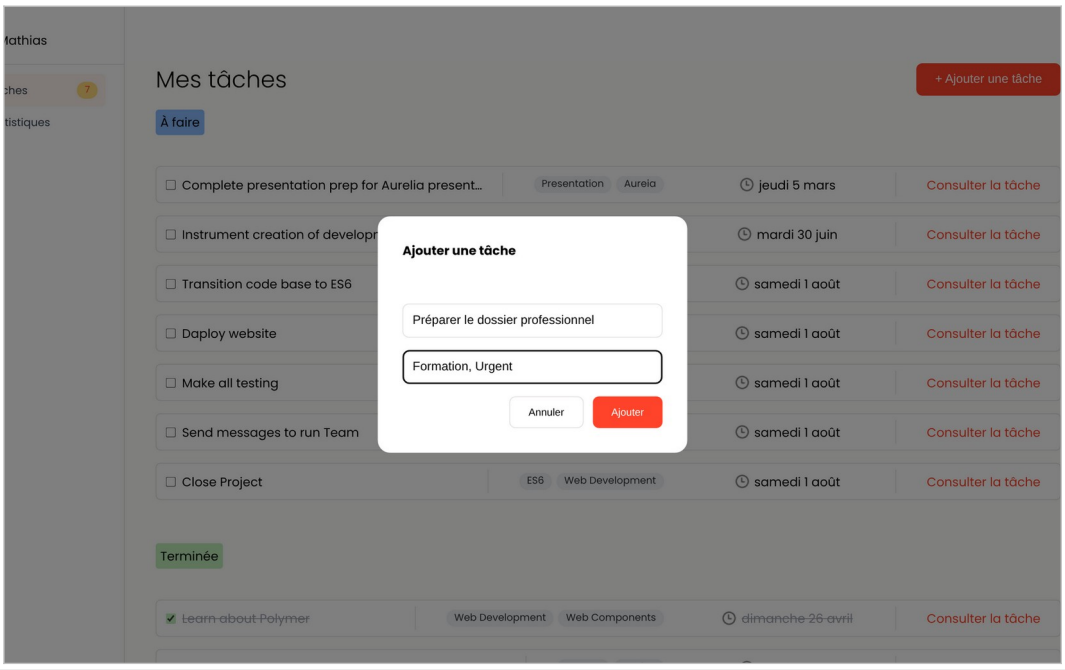
Ajout d'une tâche. Le bouton « Ajouter une tâche » ouvre une **fenêtre modale** (ajout d'une classe CSS). À la validation, je vérifie que le champ n'est pas vide, je transforme la saisie « perso, urgent » en tableau d'étiquettes propre (split, trim, filter), puis j'envoie la tâche à l'API avec une requête **POST** au format JSON.

totolist/js/taches.js

```
  // Clic sur "Ajouter" (valider) : on crée la tâche via l'API.
  boutonValider.addEventListener("click", async function () {
    const texte = input.value;
    // Si le champ est vide, on arrête là (return) : pas de tâche vide créée.
    if (!texte) {
      return;
    }
      // 👇 NOUVEAU — on lit le champ tags et on le transforme en tableau propre.
  // À ce stade inputTags.value est UNE CHAÎNE : "perso, urgent"
  const tags = inputTags.value
    .split(",")                 // 1) découpe sur les virgules → ["perso", " urgent"]
    .map(function (tag) {
      return tag.trim();        // 2) enlève les espaces autour de CHAQUE tag → ["perso", "urgent"]
    })
    .filter(function (tag) {
      return tag !== "";        // 3) vire les vides → gère champ vide et virgule en trop
    });
  // Champ tags laissé vide → tags vaut []  (tableau vide, aucun plantage)
    // POST = on ENVOIE une nouvelle tâche au serveur.
    // headers : on précise qu'on envoie du JSON.
    // body : les données de la tâche, converties en texte JSON avec JSON.stringify.
    await fetch("http://localhost:3000/todos", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({
        text: texte,
        Tags: tags,  // 👈 NOUVEAU : on envoie le tableau de tags
        is_complete: false,
      }),
    });
    // On recharge la page pour que la nouvelle tâche apparaisse dans la liste.
    window.location.reload();
  });
}

chargerTaches();
```

js/taches.js : création d'une tâche avec ses étiquettes (requête POST)



Fenêtre modale d'ajout d'une tâche avec ses étiquettes

Détail d'une tâche (detail.js). La page lit l'identifiant dans l'URL avec **URLSearchParams**, charge la tâche (GET /todos/:id) et affiche son titre, ses étiquettes, son statut et sa date. Le bouton « Terminer / Réouvrir » envoie une requête **PUT** qui inverse le statut, et le bouton de suppression une requête **DELETE** avant de revenir à la liste.

totolist/js/detail.js

```
// Charge et affiche une tâche précise, puis branche les 2 boutons d'action
async function chargerDetail() {
  // Lit les paramètres de l'URL (?id=3) pour savoir quelle tâche afficher
  const params = new URLSearchParams(window.location.search);
  const id = params.get("id");

   // Requête sur UNE seule tâche : l'id est injecté dans l'URL
  const reponse = await fetch(`http://localhost:3000/todos/${id}`);
  const donnees = await reponse.json();

  console.log(donnees);
  document.getElementById("titre-tache").textContent = donnees.text;
  const dateFormatee = new Date(donnees.created_at).toLocaleDateString("fr-FR", {
  weekday: "long",
  day: "numeric",
  month: "long",
  year: "numeric"
});
document.getElementById("date-tache").textContent = dateFormatee;
  document.getElementById("statut-tache").textContent = donnees.is_complete
    ? "Terminée"
    : "À faire";

    // 👇 NOUVEAU — affichage des tags
  // donnees.Tags est un TABLEAU (accès direct, pas de donnees[0]... ici).
  // Même principe que sur la page Tâches : un <span> chip par tag, collés avec join("").
  // (donnees.Tags || []) protège le cas où la tâche n'a pas de tags → pas de crash.
  const tagsHTML = (donnees.Tags || [])
    .map(function (tag) {
      return `<span class="tache-tag">${tag}</span>`;
    })
    .join("");
  document.getElementById("tags-tache").innerHTML = tagsHTML;

  // ── Bouton Supprimer ──
    const boutonSupprimer = document.getElementById("supprimer-tache");
```

js/detail.js : lecture de l'id dans l'URL et affichage de la tâche

totolist/js/detail.js

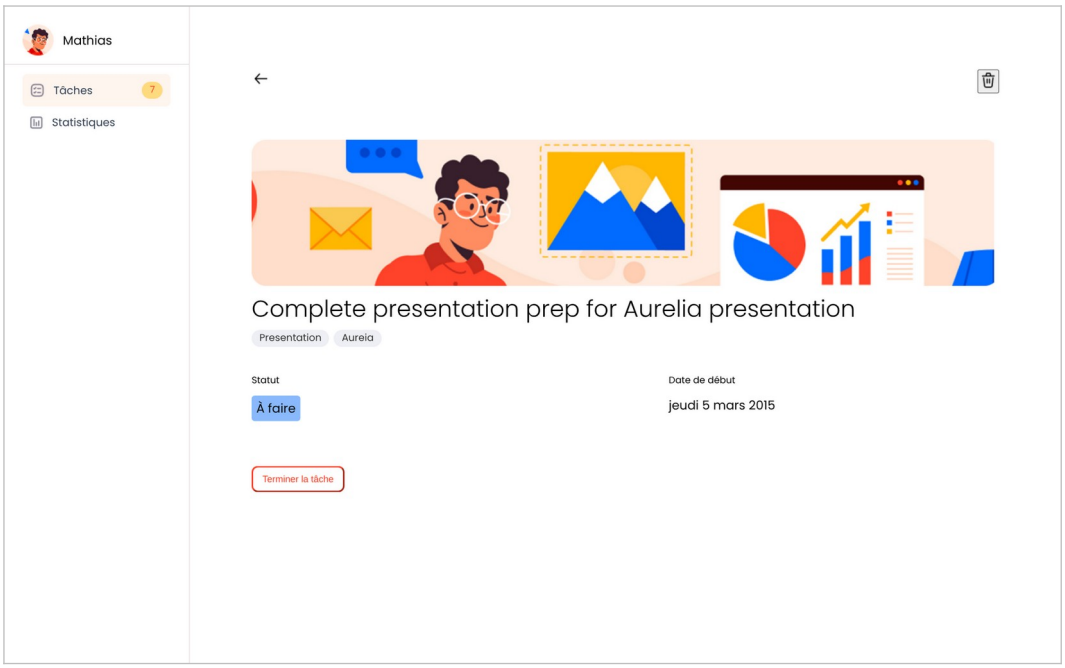
```
  boutonSupprimer.addEventListener("click", async function () {
    // DELETE sur l'API, puis retour à la liste (la tâche n'existe plus)
    await fetch(`http://localhost:3000/todos/${id}`, {
      method: "DELETE",
    });
    window.location.href = "Taches.html";
  });
  // ── Bouton Terminer / Réouvrir ──
  const boutonBascule = document.getElementById("basculer-statut");
boutonBascule.textContent = donnees.is_complete ? "Réouvrir la tâche" : "Terminer la tâche";

boutonBascule.addEventListener("click", async function() {
  await fetch(`http://localhost:3000/todos/${id}`, {
    method: "PUT",
    headers: {
      "Content-Type": "application/json"
    },
    // JSON.stringify convertit l'objet JS en texte JSON
      // Le ! inverse le statut : c'est ce qui fait la bascule
    body: JSON.stringify({
      is_complete: !donnees.is_complete
    })
  });

  // On recharge la page pour voir le nouveau statut
  window.location.reload();
});
}

chargerDetail();
```

js/detail.js : suppression (DELETE) et changement de statut (PUT)



Page de détail d'une tâche

Statistiques (stats.js) et barre latérale (sidebar.js). La page Statistiques calcule le nombre total de tâches, de tâches terminées (avec filter) et de tâches à faire. Un script commun met à jour le badge du menu avec le nombre de tâches restantes, affiche le prénom enregistré et gère l'ouverture du menu burger en version mobile.

totolist/js/stats.js

```
async function chargerStats() {
  const reponse = await fetch("http://localhost:3000/todos");
  const donnees = await reponse.json();
  const taches = donnees[0].todolist;

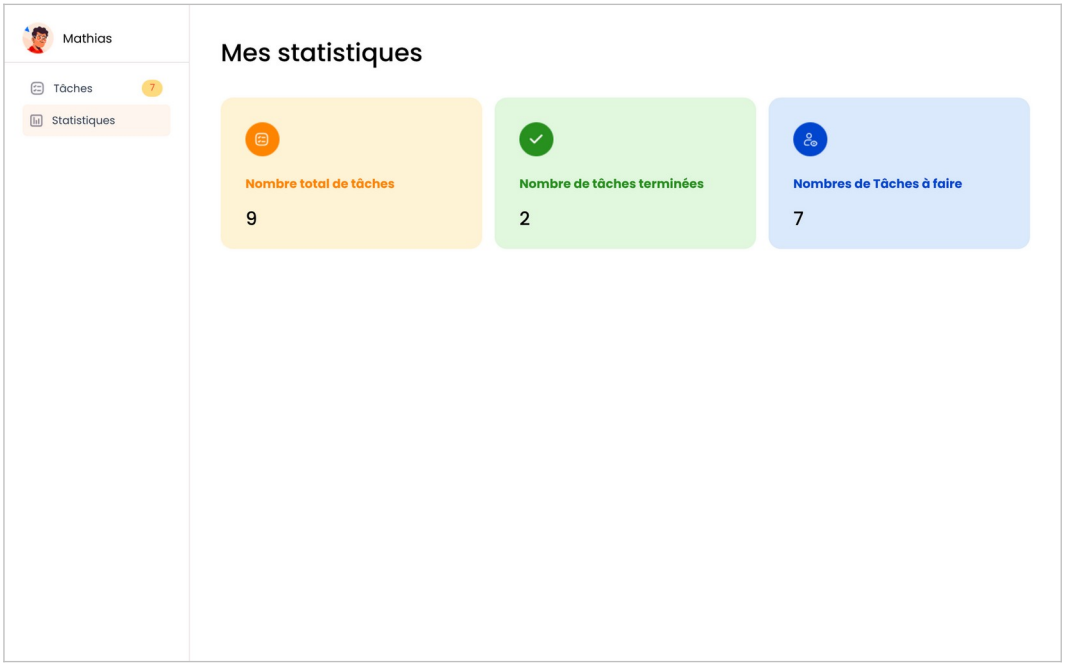
  // Nombre total de tâches
  const total = taches.length;
  // Garde uniquement les tâches terminées, puis compte le résultat
  const terminees = taches.filter(function (tache) {
    return tache.is_complete;
  }).length;

  // Soustraction total - terminees pour avoir les a faire
  const aFaire = total - terminees;

// Injection des valeurs dans les 3 cartes de la page
  document.getElementById("nombre-taches").textContent = total;
  document.getElementById("taches-terminees").textContent = terminees;
  document.getElementById("taches-a-faire").textContent = aFaire;

}
 // On lance la fonction au chargement de la page
chargerStats();
```

js/stats.js : calcul des statistiques



Page Statistiques

totolist/js/sidebar.js

```
// Pour mettre a jour la petite pastille de taches restantes dans la sidebar
async function majBadge() {
  const reponse = await fetch("http://localhost:3000/todos");
  const donnees = await reponse.json();
  const taches = donnees[0].todolist;
  const prenom = localStorage.getItem("prenom");
 document.getElementById("username").textContent = prenom;

  // Le ! inverse le test : on garde les tâches non terminées
  const aFaire = taches.filter(function(tache) {
    return !tache.is_complete;
  }).length;

  // Sélection de badge-taches et injection du nombre dans le badge de la sidebar
  document.getElementById("badge-taches").textContent = aFaire;
}

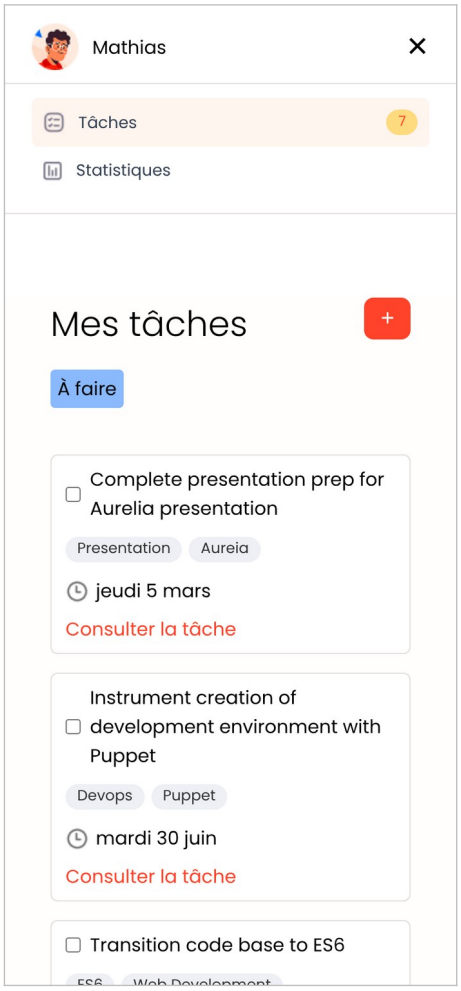
// Lancement au chargement de chaque page incluant sidebar.js
majBadge();

// Toggle du menu hamburger en mobile
const hamburger = document.getElementById("hamburger");
const sidebar = document.querySelector(".sidebar");

hamburger.addEventListener("click", function() {
  const estOuverte = sidebar.classList.toggle("ouverte");
  hamburger.textContent = estOuverte ? "×" : "☰";
```

js/sidebar.js : badge des tâches restantes et menu burger

Mise en ligne (CP 8)



Version mobile avec le menu burger ouvert

2. Précisez les moyens utilisés :

- Figma (cadres desktop et mobile, composants et variantes)
- HTML5, CSS3 responsive, police Poppins
- JavaScript natif : fetch, async / await, DOM, événements, localStorage, URLSearchParams, map / filter / forEach
- Node.js, npm, API REST Express documentée avec Swagger
- Console et onglet Réseau du navigateur pour le débogage
- Git / GitHub (commits réguliers), Vercel et GitHub Pages

3. Avec qui avez-vous travaillé ?

J'ai développé ce projet seul. L'API back-end était fournie par la formation ; mon formateur a validé les maquettes avant l'intégration et répondu à mes questions sur la programmation asynchrone.

4. Contexte

Nom de l'entreprise, organisme ou association	**Learning Campus**
Chantier, atelier, service	Projet To-Do List : maquette, intégration et front-end dynamique
Période d'exercice	**Du** 05/07/2026 **au** 08/08/2026

5. Informations complémentaires *(facultatif)*

Maquettes Figma : https://www.figma.com/design/O9Z004LEmotAiDlTU61ayK/Projet-To-Do-Mathias

Dépôt : https://github.com/Mberben/Todolist

Activité-type 2 — Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°1 — *Site du cabinet dentaire du Dr Dupont (PHP / MySQL, MVC sans framework)*

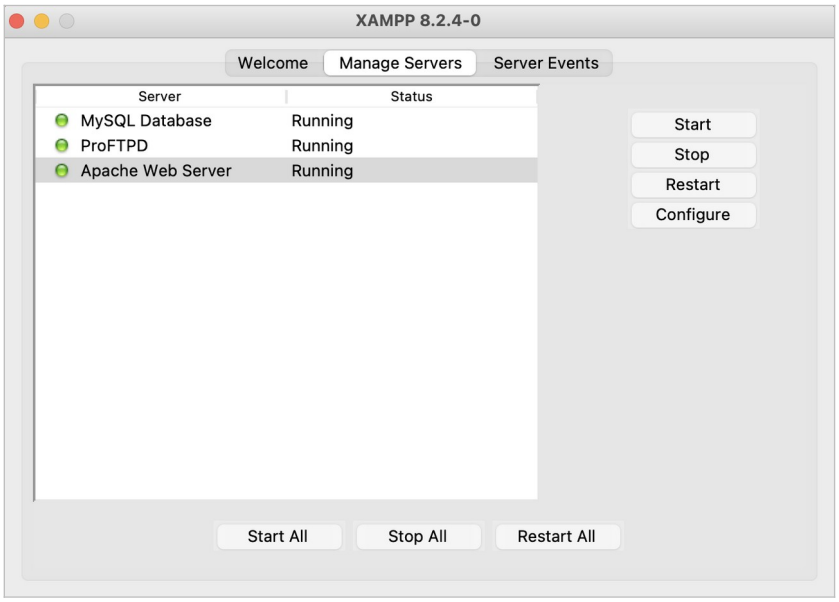
1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Compétences mises en œuvre : CP 1 Installer et configurer son environnement de travail · CP 2 Maquetter des interfaces utilisateur · CP 5 Mettre en place une base de données relationnelle · CP 6 Développer des composants d'accès aux données SQL et NoSQL · CP 7 Développer des composants métier côté serveur · CP 8 Documenter le déploiement d'une application

Ce projet consiste à réaliser le site d'un cabinet dentaire : les visiteurs consultent les services, les horaires et les actualités, et prennent rendez-vous en ligne ; l'équipe du cabinet gère tout depuis un back-office. Je l'ai développé en **PHP orienté objet**, sans framework, avec une base **MySQL**.

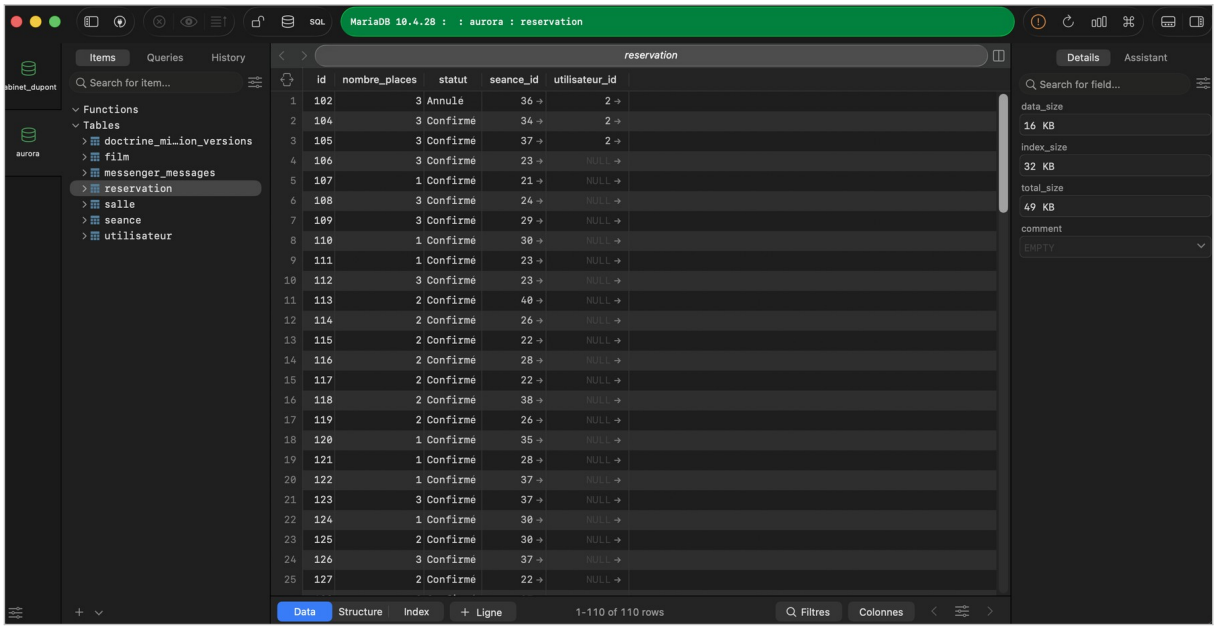
Environnement de travail (CP 1)

J'ai installé **XAMPP 8.2.4**, qui regroupe le serveur web **Apache**, le serveur de base de données **MySQL** (en réalité **MariaDB 10.4.28**), **PHP 8.2** et **phpMyAdmin**. Pour les projets PHP et WordPress, j'utilise Apache et MySQL ; pour InnovShop, je ne démarre que le service MySQL, le site étant servi par le serveur local de Symfony.



Panneau de contrôle XAMPP : MySQL, ProFTPD et Apache démarrés

Pour administrer les bases, j'utilise phpMyAdmin et un client de base de données graphique (TablePlus) connecté au serveur MariaDB local. Il me permet de parcourir les tables, de tester mes requêtes et de contrôler la structure (types, index, clés étrangères).



Client de base de données connecté au serveur MariaDB local

Analyse des besoins : cas d'utilisation (CP 2)

Analyse des besoins par cas d'utilisation (cabinet du Dr Dupont)

Pour le site de prise de rendez-vous d'un cabinet dentaire (projet PHP), j'ai commencé la conception par un **diagramme de cas d'utilisation UML**, afin de définir les écrans à concevoir et les droits de chaque profil :

- le **visiteur** consulte l'accueil, les services, la page À propos et les actualités, prend rendez-vous et s'inscrit ;
- le **patient** (un visiteur inscrit) consulte en plus ses rendez-vous ;
- le **membre de l'équipe** gère les rendez-vous, les patients, les services, les actualités et les horaires ; le **Dr Dupont** hérite de ces droits ;
- toutes les actions de gestion, ainsi que la consultation de ses rendez-vous, nécessitent de **s'authentifier** (relation « include »).

J'ai fait évoluer ce diagramme : une première version ne distinguait que le visiteur et l'équipe, la seconde ajoute l'acteur Patient, l'inscription et la consultation de ses rendez-vous, avec la relation d'héritage entre Visiteur et Patient.

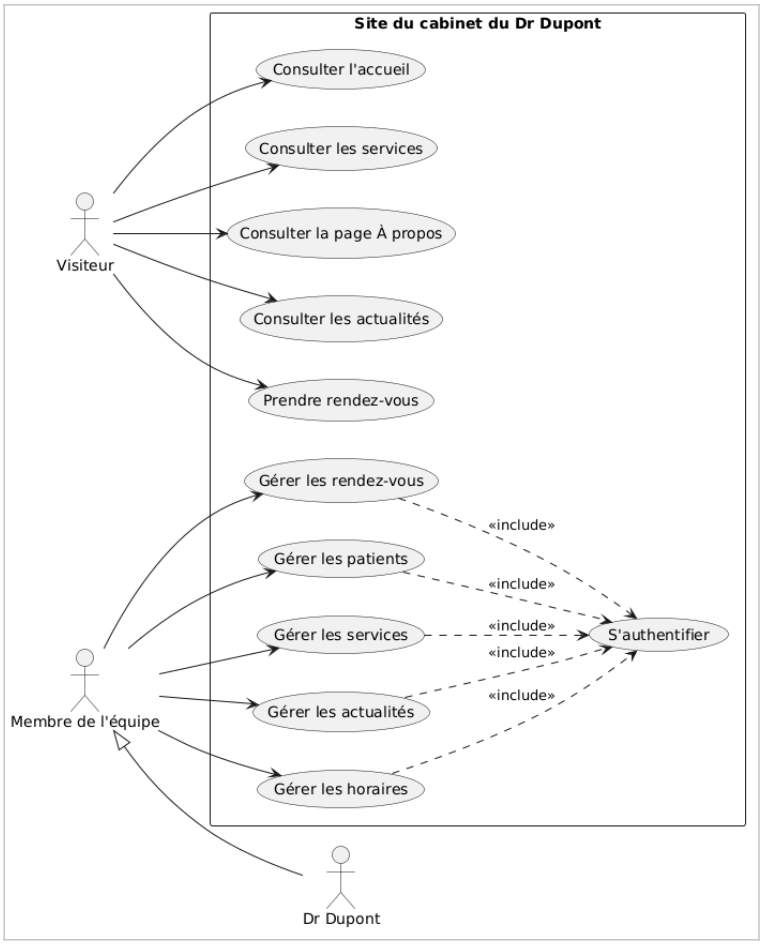
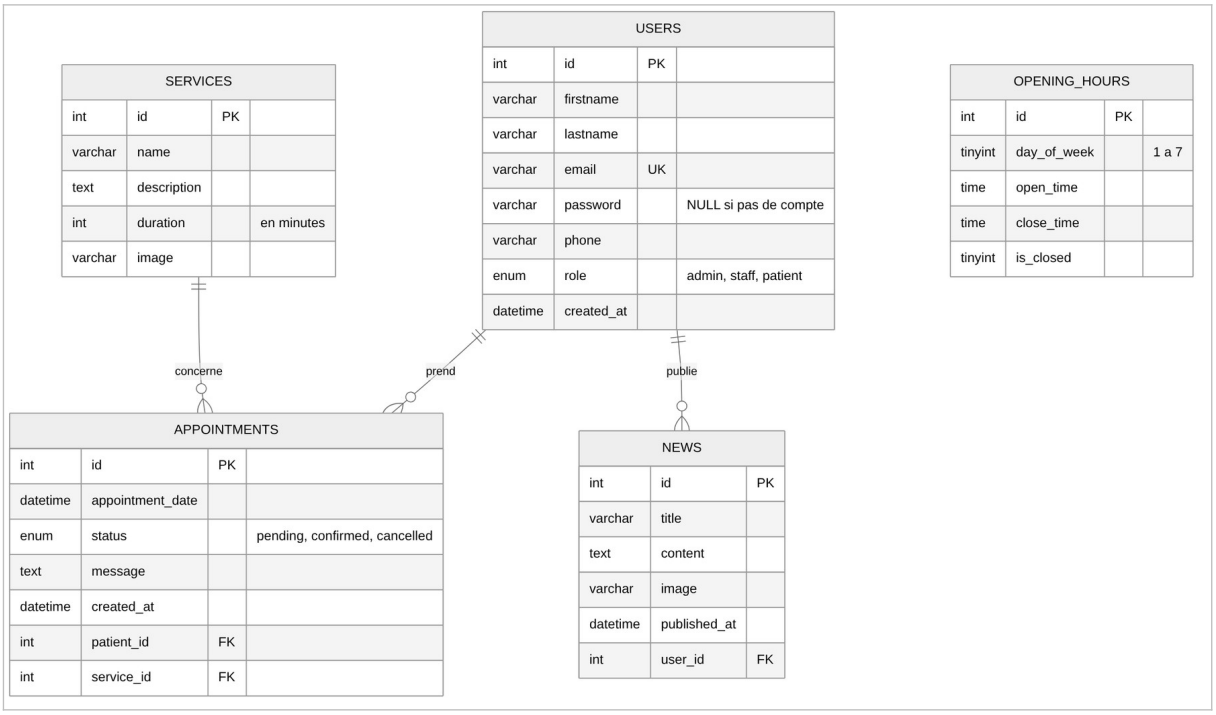


Diagramme de cas d'utilisation – version 1

Base de données relationnelle (CP 5)

Pour le site de prise de rendez-vous d'un cabinet dentaire, j'ai conçu la base MySQL **cabinet_dupont** à partir du diagramme de cas d'utilisation (voir « Maquetter des interfaces »). Elle compte cinq tables :

- **users** : patients et personnel, avec un rôle en ENUM('admin','staff','patient'), un email UNIQUE et un mot de passe facultatif ;
- **services** : soins proposés, avec leur durée en minutes, utilisée pour calculer les créneaux ;
- **appointments** : rendez-vous, avec un statut en ENUM('pending','confirmed','cancelled') et des clés étrangères vers le patient et le soin ;
- **news** : actualités du cabinet, avec une clé étrangère vers l'auteur ;
- **opening_hours** : horaires d'ouverture, avec CHECK (day_of_week BETWEEN 1 AND 7) et la possibilité d'indiquer un jour fermé.



Modèle physique de la base cabinet_dupont

Quelques choix de conception :

- **Clés étrangères avec des comportements adaptés** : supprimer un patient supprime ses rendez-vous (ON DELETE CASCADE), alors qu'un soin déjà réservé ne peut pas être supprimé (contrainte par défaut) ;
- **Mot de passe facultatif** : le personnel peut créer la fiche d'un patient qui appelle par téléphone ; le patient complète son compte plus tard en s'inscrivant avec le même email ;
- **Valeurs par défaut** : un rendez-vous est créé « en attente » et sa date de création est remplie par la base (DEFAULT current_timestamp()) ;
- **Moteur InnoDB et encodage utf8mb4**, pour les clés étrangères et les accents.

cabinet-dupont-main/database.sql

```
CREATE TABLE `appointments` (
  `id` int(11) NOT NULL,
  `appointment_date` datetime NOT NULL,
  `status` enum('pending','confirmed','cancelled') NOT NULL DEFAULT 'pending',
  `message` text DEFAULT NULL,
  `created_at` datetime NOT NULL DEFAULT current_timestamp(),
  `patient_id` int(11) NOT NULL,
  `service_id` int(11) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci;
```

database.sql : création de la table appointments

cabinet-dupont-main/database.sql

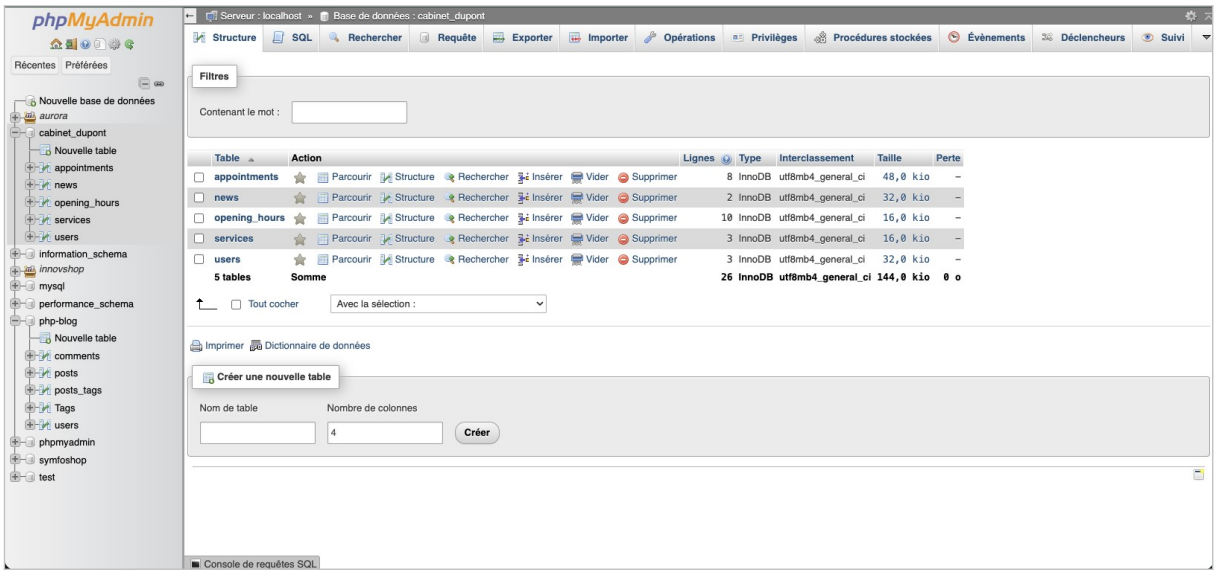
```
--
-- Contraintes pour les tables déchargées
--


--
-- Contraintes pour la table `appointments`
--
ALTER TABLE `appointments`
  ADD CONSTRAINT `fk_appointment_patient` FOREIGN KEY (`patient_id`) REFERENCES `users` (`id`) ON DELETE CASCADE,
  ADD CONSTRAINT `fk_appointment_service` FOREIGN KEY (`service_id`) REFERENCES `services` (`id`);

--
-- Contraintes pour la table `news`
--
ALTER TABLE `news`
  ADD CONSTRAINT `fk_news_user` FOREIGN KEY (`user_id`) REFERENCES `users` (`id`);
COMMIT;
```

database.sql : clés étrangères de la base

La structure et un jeu de données de test sont exportés dans le fichier **database.sql** du dépôt, pour recréer la base en une commande.



phpMyAdmin : les 5 tables InnoDB de la base cabinet_dupont (utf8mb4)

Les données de test montrent les choix de conception : les mots de passe ne sont jamais stockés en clair (hachés avec password_hash()), un patient créé lors d'une prise de rendez-vous sans compte a un mot de passe NULL, et le rôle distingue l'administrateur des patients.

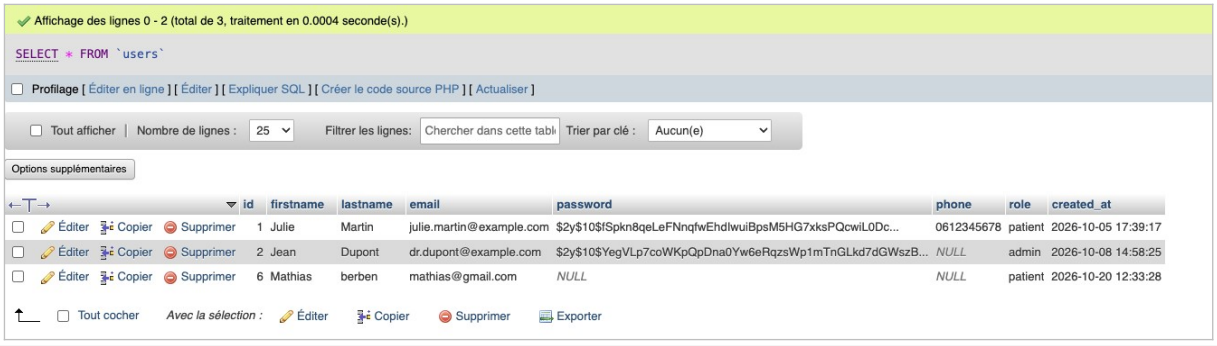
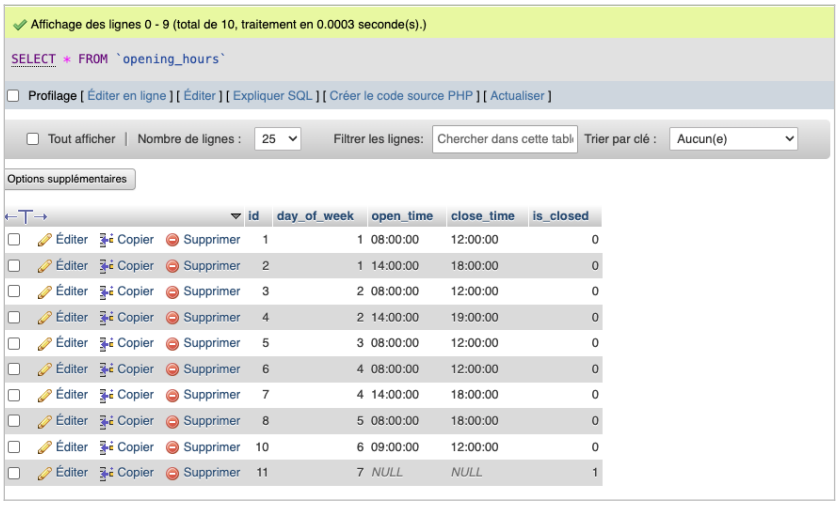


Table users : mots de passe hachés, patient sans compte (mot de passe NULL) et rôles

La table opening_hours permet plusieurs plages le même jour (matin et après-midi) et un jour fermé (dimanche : is_closed = 1, horaires NULL).



	id	day_of_week	open_time	close_time	is_closed
Éditer Copier Supprimer	1	1	08:00:00	12:00:00	0
Éditer Copier Supprimer	2	1	14:00:00	18:00:00	0
Éditer Copier Supprimer	3	2	08:00:00	12:00:00	0
Éditer Copier Supprimer	4	2	14:00:00	19:00:00	0
Éditer Copier Supprimer	5	3	08:00:00	12:00:00	0
Éditer Copier Supprimer	6	4	08:00:00	12:00:00	0
Éditer Copier Supprimer	7	4	14:00:00	18:00:00	0
Éditer Copier Supprimer	8	5	08:00:00	18:00:00	0
Éditer Copier Supprimer	10	6	09:00:00	12:00:00	0
Éditer Copier Supprimer	11	7	NULL	NULL	1

Table opening_hours : plages d'ouverture par jour de la semaine

Composants d'accès aux données (CP 6)

J'ai organisé le code selon le modèle **MVC**, avec une classe **Manager** par table (UsersManager, AppointmentsManager, ServicesManager, NewsManager, OpeningHoursManager) qui regroupe toutes ses requêtes SQL. Les contrôleurs n'écrivent jamais de SQL, et les résultats sont transformés en objets (User, Appointment…) avant d'être transmis aux vues.

- **Connexion avec PDO** : les identifiants sont des constantes définies dans config.php, exclu de Git (un modèle config.example.php est fourni). PDO est configuré en utf8mb4 et lève une exception en cas d'erreur SQL ;
- **Requêtes préparées** avec paramètres nommés pour toutes les données venant de l'utilisateur, ce qui empêche l'injection SQL ;
- **CRUD complet** sur les patients, soins, rendez-vous, actualités et horaires, depuis le back-office.

cabinet-dupont-main/models/AppointmentsManager.php

```php
<?php

require_once __DIR__ . '/Appointment.php';

// Toutes les requêtes SQL sur la table appointments
class AppointmentsManager {

    private function connectDB(): PDO
    {
        require_once __DIR__ . '/../config.php';

        $pdo = new PDO(
            'mysql:host=' . DB_HOST . ';dbname=' . DB_NAME . ';charset=utf8mb4',
            DB_USER,
            DB_PASS
        );
        $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

        return $pdo;
    }

    // Vrai si aucun rendez-vous actif ne chevauche le créneau [début, début + durée]
```

models/AppointmentsManager.php : connexion PDO

La requête la plus intéressante vérifie qu'un **créneau est libre** avant d'enregistrer un rendez-vous. Elle cherche un rendez-vous non annulé qui chevauche le créneau demandé, en tenant compte de la durée du soin déjà réservé (DATE_ADD). Le paramètre exclude_id permet de réutiliser la même vérification lors du déplacement d'un rendez-vous, sans qu'il entre en conflit avec lui-même. Une autre méthode vérifie que le créneau tombe pendant les horaires d'ouverture.

cabinet-dupont-main/models/AppointmentsManager.php

```php
    // $excludeId : en modification, on ignore le rendez-vous qu'on est en train de déplacer
    public function isSlotFree(string $start, int $duration, int $excludeId = 0): bool
    {
        $end = date('Y-m-d H:i:s', strtotime($start) + $duration * 60);

        $pdo = $this->connectDB();
        $request = $pdo->prepare("
            SELECT COUNT(*)
            FROM appointments
            INNER JOIN services ON services.id = appointments.service_id
            WHERE appointments.status != 'cancelled'
              AND appointments.id != :exclude_id
              AND appointments.appointment_date < :end
              AND DATE_ADD(appointments.appointment_date, INTERVAL services.duration MINUTE) > :start
        ");
        $request->execute(['start' => $start, 'end' => $end, 'exclude_id' => $excludeId]);

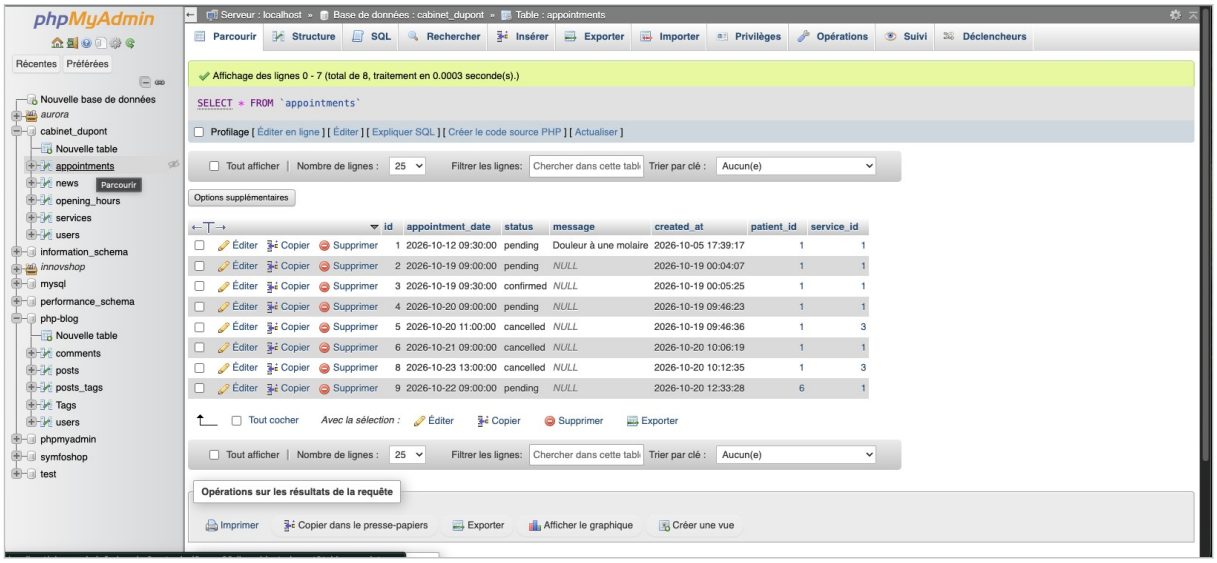
        return (int) $request->fetchColumn() === 0;
    }

    // Enregistre un nouveau rendez-vous (statut "pending" par défaut grâce à la base)
    public function createAppointment(string $date, ?string $message, int $patientId, int $serviceId): void
    {
        $pdo = $this->connectDB();
        $request = $pdo->prepare('INSERT INTO appointments (appointment_date, message, patient_id, service_id) VALUES (:date,
:message, :patient_id, :service_id)');
        $request->execute([
            'date'       => $date,
            'message'    => $message,
            'patient_id' => $patientId,
            'service_id' => $serviceId
        ]);
    }
```

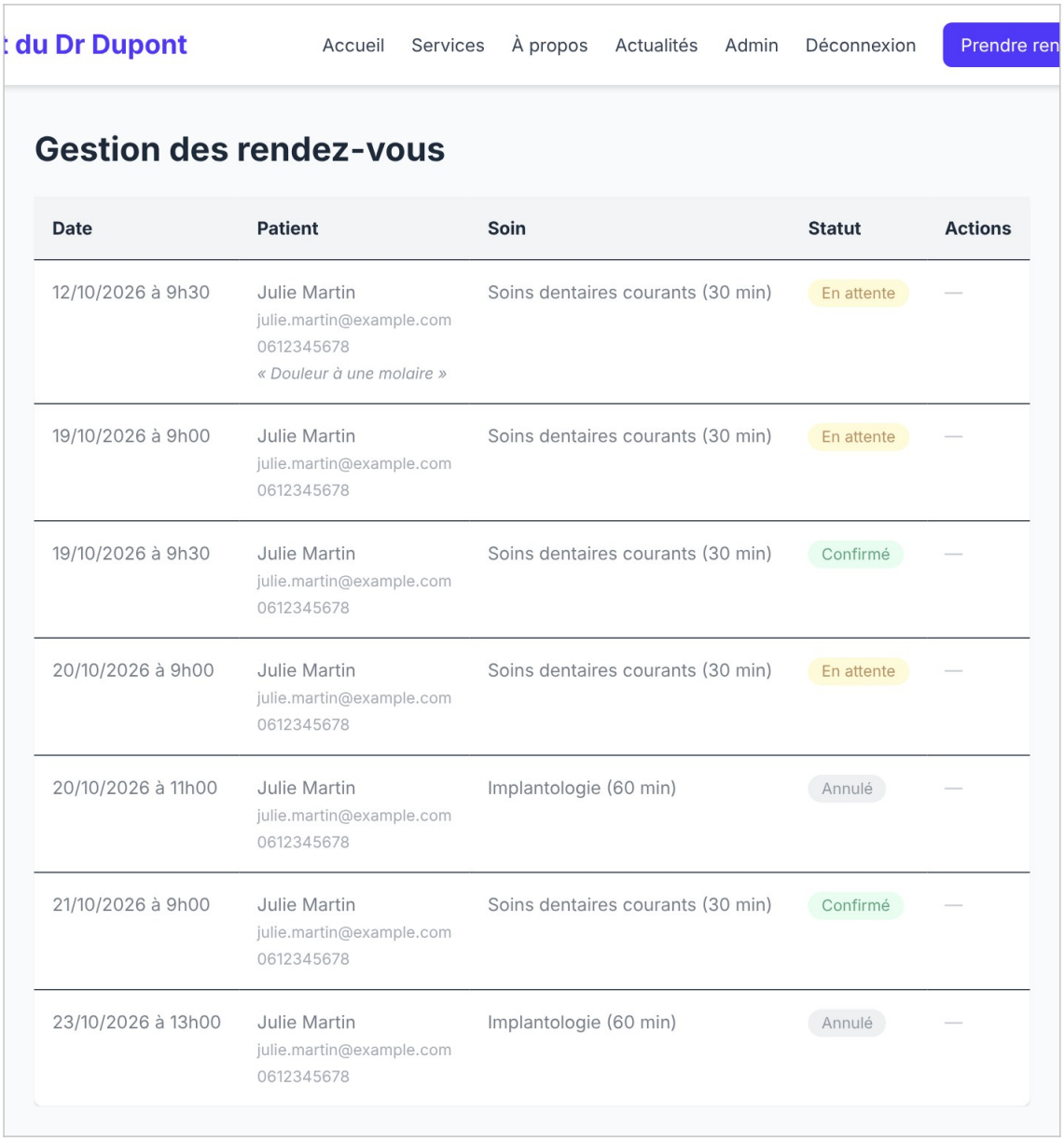
models/AppointmentsManager.php : vérification d'un créneau libre et création d'un rendez-vous

Pour l'affichage, une requête commune joint le rendez-vous, le patient et le soin, et construit le nom complet du

patient directement en SQL (CONCAT(users.firstname, ' ', users.lastname) AS patient_name). D'autres méthodes répondent à des besoins précis : emailUsedByOther() pour éviter les doublons d'email en modification, lastInsertId() pour rattacher un rendez-vous au patient qui vient d'être créé, ou updateStatus() qui n'accepte que les statuts autorisés.



phpMyAdmin : table appointments avec les statuts pending, confirmed et cancelled et les clés étrangères patient_id et service_id



Back-office : liste des rendez-vous obtenue par jointure (patient, soin, statut)

Composants métier côté serveur (CP 7)

J'ai développé en **PHP orienté objet** la logique métier du site, selon une architecture **MVC** : un routeur unique (index.php) appelle le bon contrôleur selon le paramètre page, le contrôleur applique les règles métier et affiche la vue correspondante.

Prise de rendez-vous en ligne. Le traitement du formulaire applique plusieurs règles avant d'enregistrer quoi que ce soit :

- le soin choisi doit exister (sa durée sert aux vérifications suivantes) ;
- la date doit être dans le futur ;
- le rendez-vous doit tenir entièrement dans une **plage d'ouverture** du cabinet ;
- le créneau doit être **libre** (aucun chevauchement avec un autre rendez-vous) ;

- le patient est identifié : s'il est connecté, son compte est utilisé ; sinon l'application recherche son email ou crée une fiche patient. Un email du personnel ne peut pas servir à réserver.

Le rendez-vous est enregistré « en attente » ; l'équipe le confirme ou l'annule depuis le back-office.

cabinet-dupont-main/controllers/AppointmentsController.php

```php
public function appointmentValid()
{
    $serviceId = (int) ($_POST['service_id'] ?? 0);
    $date      = $_POST['date'] ?? '';
    $time      = $_POST['time'] ?? '';
    $message   = trim($_POST['message'] ?? '') ?: null;

    // 1. Le service doit exister (sa durée sert aux vérifications)
    $servicesManager = new ServicesManager();
    $service = $servicesManager->getService($serviceId);

    if (!$service || !$date || !$time) {
        header('Location: index.php?page=appointment&error=fields');
        exit;
    }

    $start = $date . ' ' . $time . ':00';   // format SQL : "2026-10-19 09:00:00"
    $duration = $service->getDuration();

    // 2. Dans le futur
    if (strtotime($start) <= time()) {
        header('Location: index.php?page=appointment&error=past');
        exit;
    }

    // 3. Pendant une plage d'ouverture
    $hoursManager = new OpeningHoursManager();
    if (!$hoursManager->isOpen($start, $duration)) {
        header('Location: index.php?page=appointment&error=closed');
        exit;
    }

    // 4. Créneau libre (aucun chevauchement)
    $appointmentsManager = new AppointmentsManager();
    if (!$appointmentsManager->isSlotFree($start, $duration)) {
        header('Location: index.php?page=appointment&error=taken');
        exit;
    }

    // 5. Identifier le patient
    $usersManager = new UsersManager();
```

controllers/AppointmentsController.php : règles de prise de rendez-vous (1/2)

cabinet-dupont-main/controllers/AppointmentsController.php

```php
        if (Auth::isLoggedIn() && !Auth::isStaff()) {
            // Patient connecté : on utilise son compte
            $patientId = (int) $_SESSION['user_id'];
        } else {
            // Visiteur : on cherche son email, sinon on crée le patient
            $firstname = trim($_POST['firstname'] ?? '');
            $lastname  = trim($_POST['lastname'] ?? '');
            $email     = trim($_POST['email'] ?? '');
            $phone     = trim($_POST['phone'] ?? '') ?: null;

            if (!$firstname || !$lastname || !filter_var($email, FILTER_VALIDATE_EMAIL)) {
                header('Location: index.php?page=appointment&error=fields');
                exit;
            }

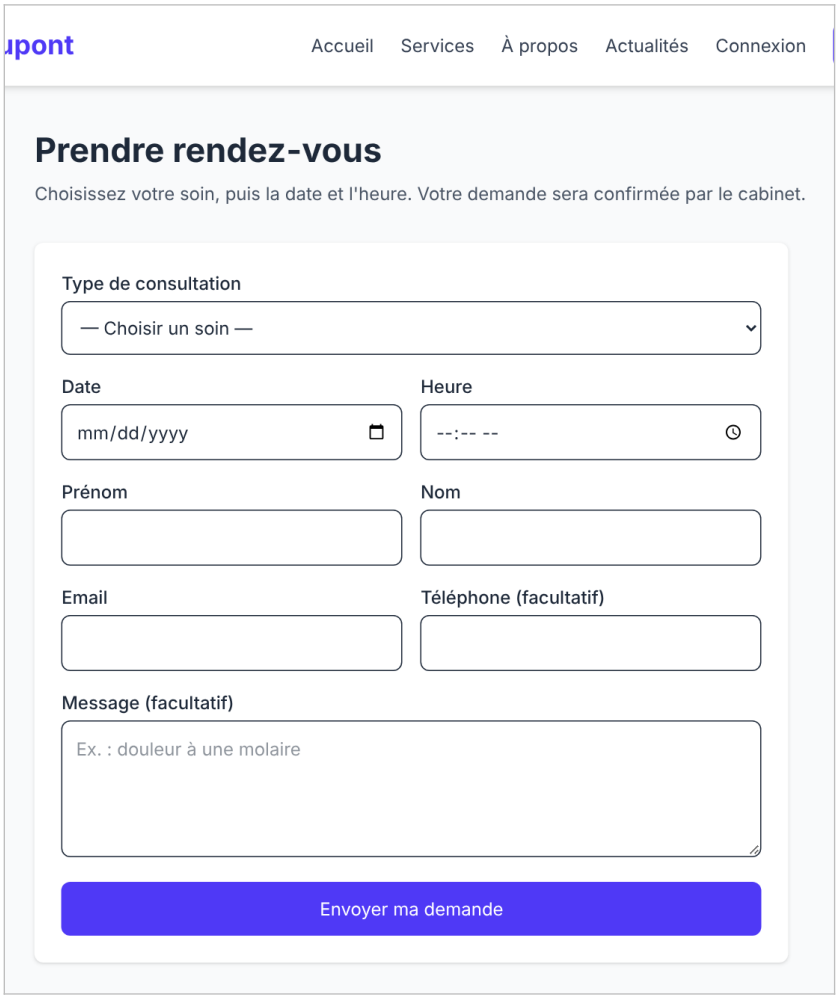
            $existing = $usersManager->getUser($email);

            if ($existing) {
                // Un email du personnel ne peut pas servir à réserver comme patient
                if ($existing->getRole() !== 'patient') {
                    header('Location: index.php?page=appointment&error=email');
                    exit;
                }
                $patientId = $existing->getId();
            } else {
                $patientId = $usersManager->createPatient($firstname, $lastname, $email, $phone);
            }
        }

        // 6. Tout est bon : on enregistre (statut "pending")
        $appointmentsManager->createAppointment($start, $message, $patientId, $serviceId);

        header('Location: index.php?page=appointment&success=1');
        exit;
    }
```

controllers/AppointmentsController.php : identification du patient et enregistrement (2/2)



Formulaire de prise de rendez-vous

Authentification et rôles. À la connexion, le mot de passe est vérifié avec password_verify(), l'identifiant de session est régénéré (session_regenerate_id) pour éviter le vol de session, et un message d'erreur unique est affiché quel que soit le problème, pour ne pas révéler si un email existe. Une classe **Auth** centralise les contrôles d'accès : requireLogin() pour les pages patient et requireStaff() pour le back-office. L'identifiant du patient est toujours lu dans la session, jamais dans l'URL.

cabinet-dupont-main/controllers/UsersController.php

```php
public function loginValid() {
    $email = trim($_POST['email'] ?? '');
    $password = $_POST['password'] ?? '';   // pas de trim : un espace peut faire partie du mot de passe

    // Champs vides
    if (!$email || !$password) {
        header('Location: index.php?page=login&error=fields');
        exit;
    }

    $usersManager = new UsersManager();
    $user = $usersManager->getUser($email);

    // Email inconnu, patient sans compte, ou mauvais mot de passe : même message d'erreur
    if (!$user || !$user->hasAccount() || !password_verify($password, $user->getPassword())) {
        header('Location: index.php?page=login&error=credentials');
        exit;
    }

    // Nouvel identifiant de session à la connexion (protection contre le vol de session)
    session_regenerate_id(true);

    // On garde en session ce dont les pages auront besoin
    $_SESSION['user_id'] = $user->getId();
    $_SESSION['user_name'] = $user->getFirstname();
    $_SESSION['user_role'] = $user->getRole();

    // Le personnel va au back office, les patients à l'accueil
    if ($user->isStaff()) {
        header('Location: index.php?page=admin');
    } else {
        header('Location: index.php');
    }
    exit;
}

// Déconnexion : on vide la session puis on retourne à l'accueil
public function logout() {
    session_destroy();
    header('Location: index.php');
    exit;
}
```

controllers/UsersController.php : traitement de la connexion

cabinet-dupont-main/helpers/Auth.php

```php
<?php

// Middleware d'authentification : vérifications à appeler avant les pages restreintes
class Auth {

    // Vrai si un utilisateur est connecté
    public static function isLoggedIn(): bool
    {
        return isset($_SESSION['user_id']);
    }

    // Vrai si l'utilisateur connecté fait partie du personnel (admin ou staff)
    public static function isStaff(): bool
    {
        return self::isLoggedIn() && in_array($_SESSION['user_role'] ?? '', ['admin', 'staff']);
    }

    // Bloque l'accès si personne n'est connecté (pages patient : "Mes rendez-vous"...)
    public static function requireLogin(): void
    {
        if (!self::isLoggedIn()) {
            header('Location: index.php?page=login');
            exit;
        }
    }

    // Bloque l'accès si l'utilisateur n'est pas du personnel (pages du back office)
    public static function requireStaff(): void
    {
        if (!self::isStaff()) {
            header('Location: index.php?page=login');
            exit;
        }
    }
}
```

helpers/Auth.php : contrôle d'accès selon la connexion et le rôle

Back-office et sécurité. J'ai développé la gestion des rendez-vous (confirmer, annuler, modifier), des patients, des services, des actualités et des horaires. Les données affichées passent par htmlspecialchars() contre les failles XSS, et les images envoyées sont contrôlées (vraie image, 2 Mo maximum, extensions autorisées, nom unique).

cabinet-dupont-main/controllers/AdminController.php

```php
// Gère l'upload d'une image : renvoie le nouveau nom, null si aucun fichier,
// ou redirige vers $errorUrl si l'image est invalide
private function uploadImage(string $errorUrl): ?string
{
    // Aucun fichier envoyé : on renvoie null (l'appelant décide quoi faire)
    if (!isset($_FILES['image']) || $_FILES['image']['error'] !== UPLOAD_ERR_OK) {
        return null;
    }

    $imageFileType = strtolower(pathinfo($_FILES['image']['name'], PATHINFO_EXTENSION));

    // Vérifications : vraie image, 2 Mo maximum, format autorisé
    if (!getimagesize($_FILES['image']['tmp_name'])
        || $_FILES['image']['size'] > 2 * 1024 * 1024
        || !in_array($imageFileType, ['jpeg', 'jpg', 'png', 'gif', 'webp'])) {
        header('Location: ' . $errorUrl . '&error=image');
        exit;
    }

    // Nom unique pour éviter d'écraser une image existante
    $newFileName = uniqid('img_', true) . '.' . $imageFileType;

    if (!move_uploaded_file($_FILES['image']['tmp_name'], __DIR__ . '/../uploads/' . $newFileName)) {
        header('Location: ' . $errorUrl . '&error=upload');
        exit;
    }

    return $newFileName;
}
```

controllers/AdminController.php : contrôle et enregistrement des images envoyées

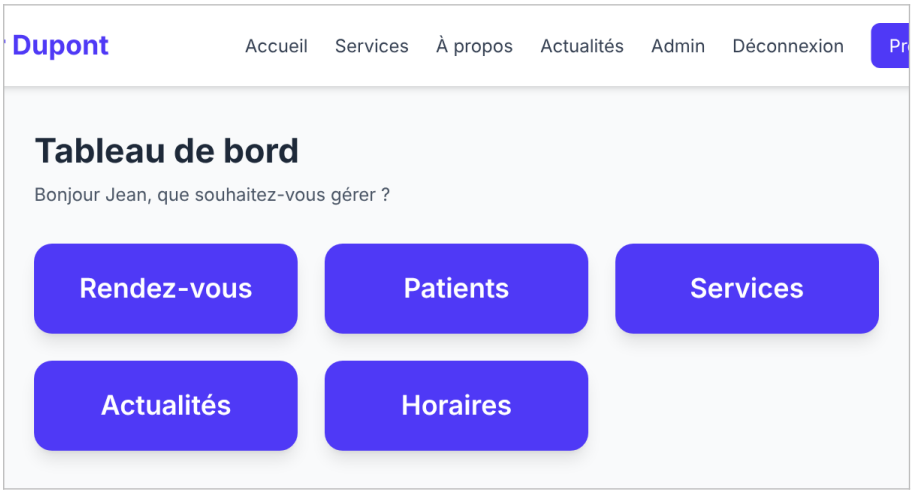
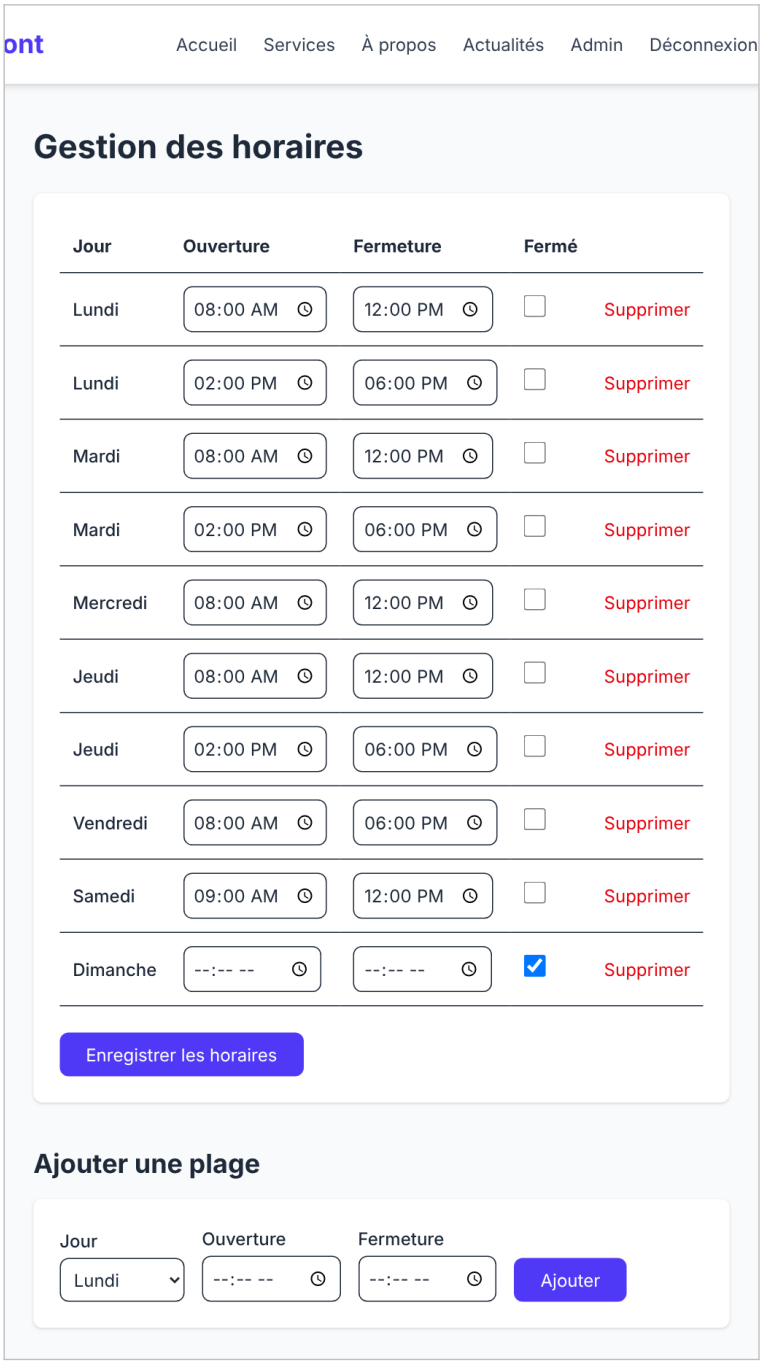


Tableau de bord du back-office



Gestion des horaires d'ouverture

Déploiement documenté (CP 8)

J'ai déployé le site PHP du cabinet sur mon hébergement **alwaysdata**, à l'adresse https://berben.alwaysdata.net/cabinet/, à côté de mon portfolio déjà en ligne à la racine du domaine.

Préparation du code. J'ai séparé la configuration du code : le fichier config.php (identifiants de base de données) n'est pas versionné. Le dépôt contient à la place un modèle **config.example.php** qui indique les constantes à définir sur chaque environnement. Dans index.php, l'affichage des erreurs dépend de la constante **DEBUG** : actif en local pour déboguer, désactivé en production pour la sécurité.

cabinet-dupont-main/config.example.php

```php
<?php
// Copier ce fichier en config.php et renseigner ses propres informations
define('DB_HOST', 'localhost');
define('DB_NAME', 'cabinet_dupont');
define('DB_USER', 'root');
define('DB_PASS', '');
define('DEBUG', true);   // false en production
```

config.example.php : modèle de configuration versionné

cabinet-dupont-main/index.php

```php
<?php
session_start();

require_once __DIR__ . '/config.php';

// En local (DEBUG = true) : on affiche les erreurs pour déboguer
// En ligne (DEBUG = false) : on les cache aux visiteurs
error_reporting(E_ALL);
ini_set('display_errors', DEBUG ? '1' : '0');

require __DIR__ . '/controllers/PostsController.php';
require __DIR__ . '/controllers/AdminController.php';
require __DIR__ . '/controllers/UsersController.php';
require_once __DIR__ . '/helpers/Auth.php';
require __DIR__ . '/controllers/AppointmentsController.php';

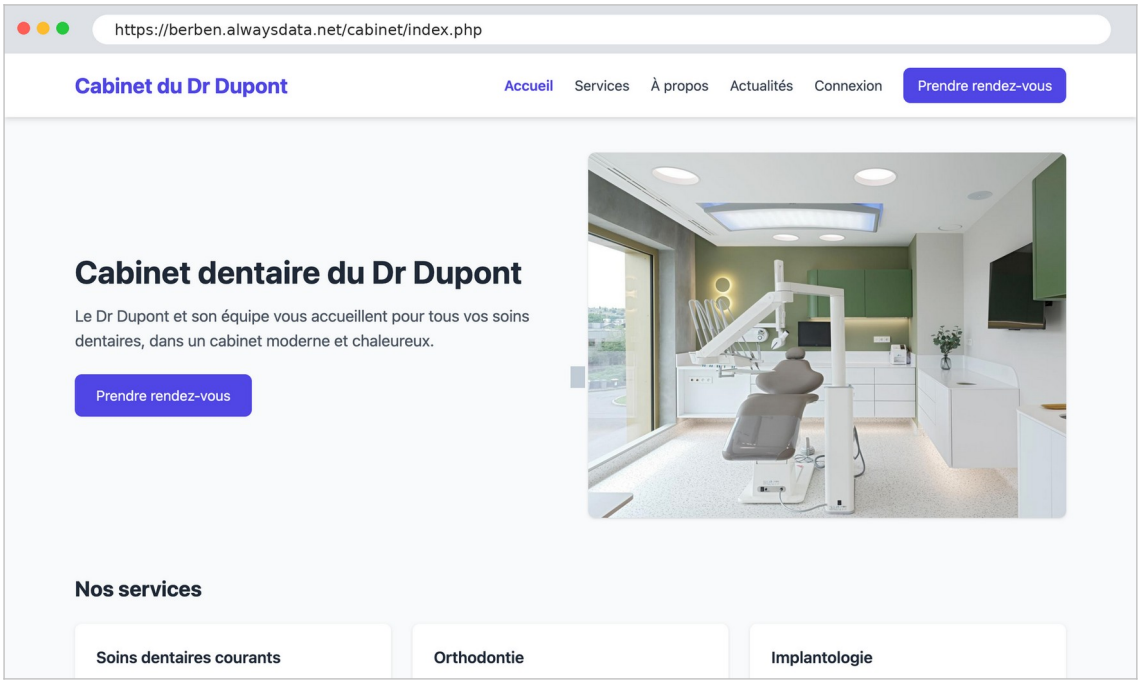
$postsController = new PostsController();
$adminController = new AdminController();
$usersController = new UsersController();
$appointmentsController = new AppointmentsController();
$page =$_GET['page'] ?? 'home';

if($page === 'home'){
 $postsController->home();
}
else if($page === 'admin') {
    $adminController->admin();
}
else if($page=== 'register') {
    $usersController->register();
```

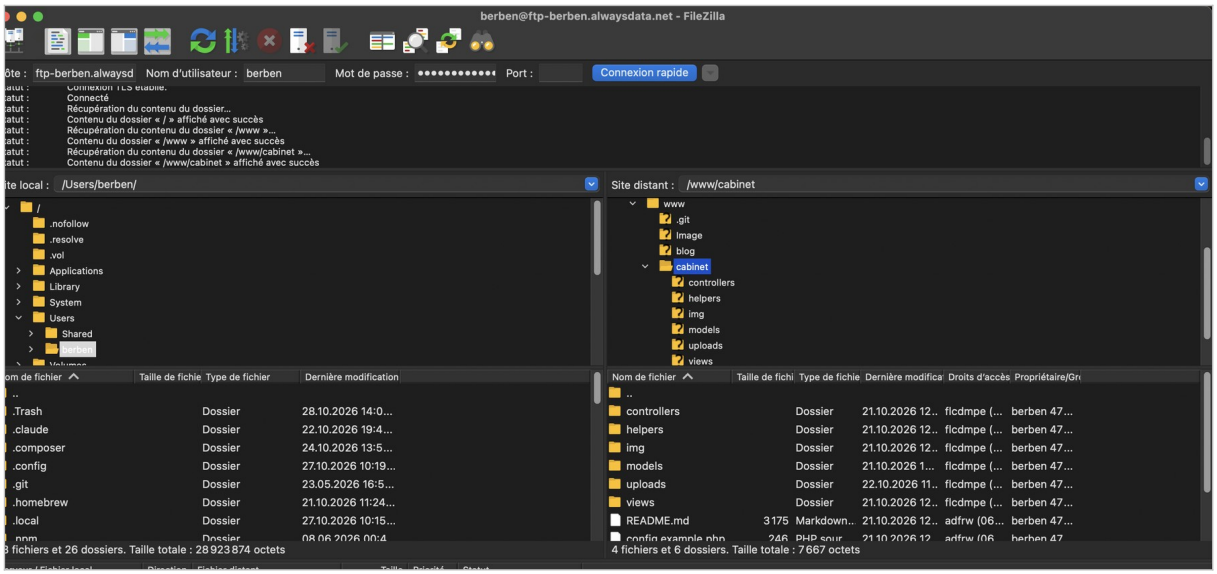
index.php : affichage des erreurs selon DEBUG et routeur

Étapes du déploiement :

1. **Base de données** : création d'une base MySQL et de son utilisateur dans l'interface d'administration d'alwaysdata.
2. **Import des données** : import dans phpMyAdmin du fichier database.sql (structure et données de test, exporté depuis ma base locale).
3. **Transfert des fichiers** : envoi du code dans le dossier www/cabinet/ avec **FileZilla**, par une connexion FTP chiffrée en TLS.
4. **Configuration** : création d'un fichier config.php propre au serveur, avec l'hôte, le nom de la base, l'utilisateur et le mot de passe de production, et DEBUG à false.
5. **Vérifications en ligne** : navigation publique, inscription, connexion, prise de rendez-vous, accès au back-office selon le rôle, envoi d'images.



Le site du cabinet en ligne sur alwaysdata



FileZilla connecté à ftp-berben.alwaysdata.net (connexion chiffrée TLS) : le projet déposé dans /www/cabinet

2. Précisez les moyens utilisés :

- PHP 8.2 orienté objet, architecture MVC, PDO, sessions, password_hash / password_verify
- MySQL / MariaDB 10.4 (XAMPP), phpMyAdmin, TablePlus
- Tailwind CSS pour les vues
- UML : diagramme de cas d'utilisation
- Hébergement alwaysdata, FileZilla (FTP chiffré TLS)

- Visual Studio Code, PHP Intelephense, PHP Debug ; Git / GitHub

3. Avec qui avez-vous travaillé ?

J'ai développé ce projet seul, à partir du cahier des charges de la formation. Mon formateur a validé le modèle de données et m'a aidé sur les points de sécurité (sessions, upload de fichiers, contrôle des accès).

4. Contexte

Nom de l'entreprise, organisme ou association	**Learning Campus**
Chantier, atelier, service	Projet Cabinet du Dr Dupont : site de prise de rendez-vous
Période d'exercice	**Du** 05/10/2026 **au** 28/10/2026

5. Informations complémentaires *(facultatif)*

Site en ligne : https://berben.alwaysdata.net/cabinet/

Activité-type 2

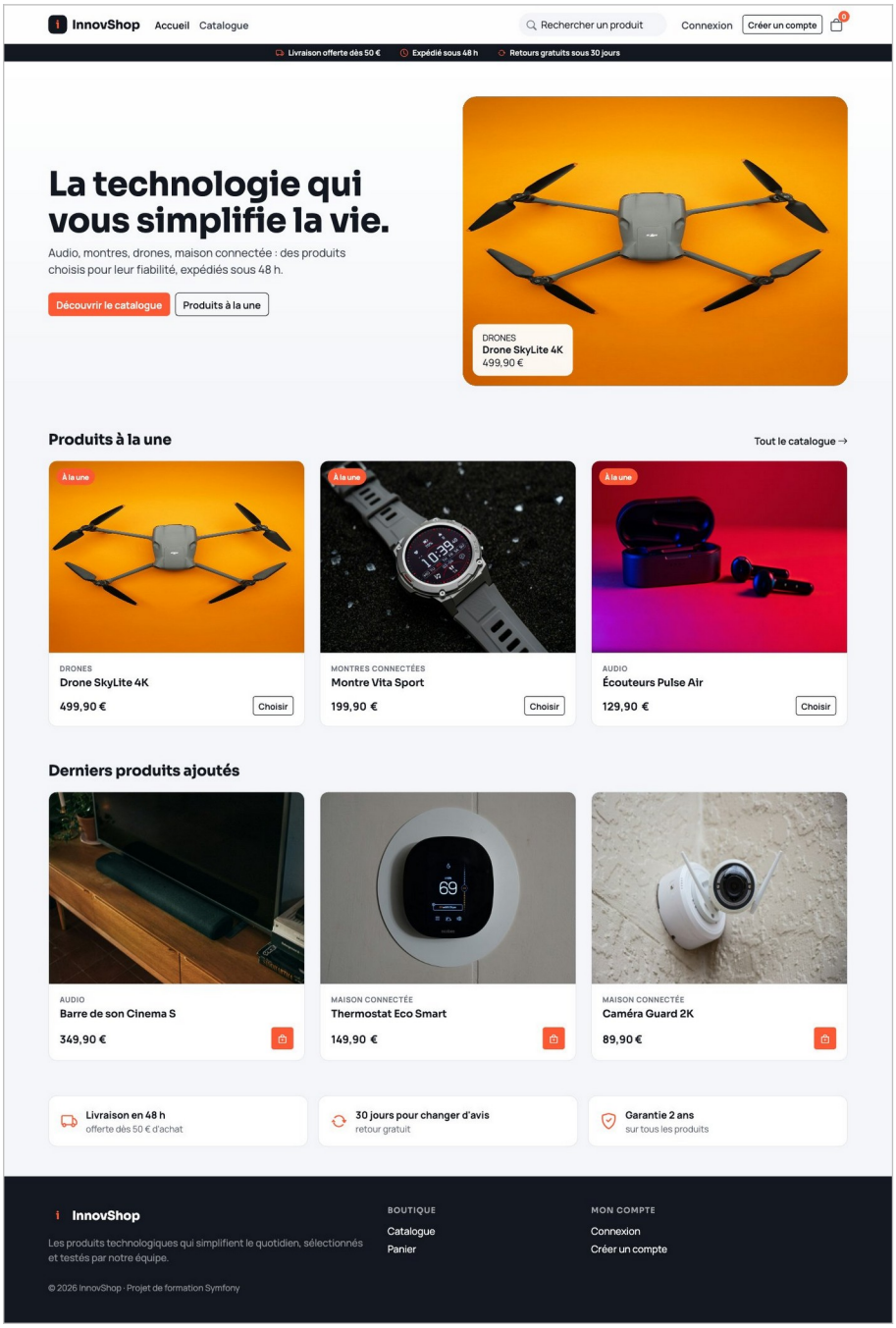
Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°2 — *Plateforme e-commerce InnovShop (Symfony / Doctrine)*

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Compétences mises en œuvre : CP 1 Installer et configurer son environnement de travail · CP 4 Développer la partie dynamique des interfaces · CP 5 Mettre en place une base de données relationnelle · CP 6 Développer des composants d'accès aux données SQL et NoSQL · CP 7 Développer des composants métier côté serveur · CP 8 Documenter le déploiement d'une application

InnovShop est une start-up qui vend des produits technologiques (audio, montres connectées, drones, lunettes connectées, maison connectée). Sa directrice, Madame Leroy (cas fictif fourni par la formation), souhaite digitaliser ses ventes. J'ai réalisé la première version de sa boutique en ligne avec **Symfony 8.1** et **Doctrine** : catalogue avec recherche et filtres, fiches produits avec options, panier AJAX, commande en trois étapes avec email, espace client, back-office, facture PDF et déploiement.



Page d'accueil d'InnovShop : produits à la une et derniers produits ajoutés

Analyse du besoin et organisation (CP 1)

J'ai traduit le cahier des charges en **30 user stories** au format « En tant que… je veux… afin de… », chacune avec ses critères d'acceptation. Je les ai priorisées avec la méthode **MoSCoW** (23 Must, 3 Should, 4 Could) et estimées en **story points** (83 au total, dont 68 en Must). Hors périmètre de cette première version : paiement en ligne, quantités dans le panier et gestion des stocks.

Backlog du produit — InnovShop (plateforme e-commerce Symfony)

User stories	Rôles/Personas	Fonctions/Fonctionnalités	Raisons/Objectifs	Acceptation	Évaluation	Story point
1	En tant que visiteur	Je veux voir sur la page d'accueil les 3 derniers produits ajoutés et 3 produits à la une	Afin de découvrir rapidement les nouveautés et les produits mis en avant	L'accueil affiche les 3 produits les plus récents (tri par date de création) et 3 produits marqués « à la une » ; chaque carte mène à la fiche produit	Must	3
2	En tant que visiteur	Je veux consulter le catalogue de tous les produits	Afin de parcourir l'offre d'InnovShop	Grille responsive (image, nom, prix, bouton « Voir ») ; affichage correct sur mobile, tablette et desktop	Must	3
3	En tant que visiteur	Je veux consulter la fiche détaillée d'un produit	Afin de connaître sa description, ses spécifications, son prix et voir ses images	La page /produit/{id} affiche description, spécifications, prix, images ; 404 personnalisée si le produit n'existe pas	Must	3
4	En tant que visiteur	Je veux choisir une option (couleur, taille…) sur la fiche produit	Afin de acheter la variante qui me convient	Les options disponibles du produit sont proposées dans une liste ; l'option choisie est conservée dans le panier	Must	5
5	En tant que visiteur	Je veux rechercher un produit par mot-clé ou filtrer par catégorie	Afin de trouver plus vite ce que je cherche	La recherche porte sur le nom et la description ; le filtre par catégorie réduit la liste ; aucun résultat = message explicite	Could	3
6	En tant que visiteur	Je veux ajouter un produit au panier sans recharger la page	Afin de constituer ma commande de façon fluide	Requête AJAX ; chaque ajout crée une nouvelle ligne (pas de quantité) ; le compteur du panier se met à jour immédiatement	Must	5
7	En tant que visiteur	Je veux supprimer une ligne de mon panier	Afin de corriger ma sélection	La ligne disparaît sans rechargement ; le total est recalculé ; panier vide = message	Must	2
8	En tant que visiteur	Je veux accéder à mon panier depuis n'importe quelle page	Afin de suivre mon panier en permanence	Icône + compteur dans le header sur toutes les pages ; lien vers la page panier	Must	2
9	En tant que visiteur	Je veux créer un compte avec mon email et un mot de passe sécurisé	Afin de pouvoir passer commande	Email unique et valide ; mot de passe d'au moins 8 caractères, hashé ; formulaire protégé par CSRF ; messages d'erreur clairs	Must	3
10	En tant que visiteur	Je veux me connecter et me déconnecter	Afin de accéder à mon espace personnel	Connexion par email/mot de passe ; message en cas d'identifiants incorrects ; lien de déconnexion dans le header	Must	2
11	En tant que visiteur	Je veux être redirigé vers la connexion si je tente de commander sans être connecté	Afin de comprendre qu'un compte est nécessaire	L'accès au tunnel de commande exige ROLE_USER ; après connexion, retour vers le tunnel ; le panier est conservé	Must	1
12	En tant que client	Je veux vérifier mon panier (étape 1 de la commande)	Afin de m'assurer du contenu avant de commander	Récapitulatif des lignes (produit, option, prix) et du total ; bouton « Continuer » ; impossible avec un panier vide	Must	2
13	En tant que client	Je veux saisir ou confirmer mon adresse de livraison (étape 2)	Afin de recevoir ma commande au bon endroit	Adresse pré-remplie depuis le profil ; champs obligatoires validés (Assert) : nom, adresse, code postal à 5 chiffres, ville, téléphone	Must	3
14	En tant que client	Je veux confirmer ma commande (étape 3)	Afin de finaliser mon achat	Commande enregistrée avec ses lignes, l'adresse, le total et le statut « en attente » ; panier vidé ; page de confirmation avec la référence	Must	5
15	En tant que client	Je veux recevoir un email récapitulatif après ma commande	Afin de garder une trace de mon achat	Email envoyé via Symfony Mailer avec template Twig : référence, lignes, total, adresse	Must	3
16	En tant que client	Je veux consulter l'historique de mes commandes	Afin de suivre mes achats	Liste triée par date (référence, date, total, statut) ; je ne vois que mes propres commandes (Voter)	Must	2
17	En tant que client	Je veux voir le détail et le statut d'une commande	Afin de savoir où en est ma livraison	Lignes, prix, adresse, badge de couleur selon le statut ; accès refusé (403) à la commande d'un autre client	Must	2
18	En tant que client	Je veux modifier mes informations de profil et de livraison	Afin de garder des coordonnées à jour	Modification du nom, de l'adresse et du téléphone avec validation ; message de succès	Must	3
19	En tant que client	Je veux changer mon mot de passe	Afin de sécuriser mon compte	Ancien mot de passe vérifié ; nouveau mot de passe confirmé et hashé	Should	2
20	En tant que client	Je veux définir un objectif de dépenses mensuel et voir où j'en suis	Afin de maîtriser mon budget	Montant saisi dans le profil ; le tableau de bord affiche le total du mois et une barre de progression	Should	3
21	En tant que client	Je veux télécharger la facture d'une commande en PDF	Afin de conserver un justificatif	Bouton sur le détail de la commande ; PDF généré avec les lignes et le total	Could	3
22	En tant que client	Je veux être notifié par email quand le statut de ma commande change	Afin de suivre l'expédition sans me connecter	Email envoyé quand l'admin modifie le statut	Could	2
23	En tant que administrateur	Je veux accéder à un back office réservé aux administrateurs	Afin de gérer la boutique en toute sécurité	/admin protégé par ROLE_ADMIN (security.yaml + IsGranted) ; un client reçoit une erreur 403	Must	2
24	En tant que administrateur	Je veux ajouter, modifier et supprimer des produits	Afin de tenir le catalogue à jour	CRUD EasyAdmin : nom, description, spécifications, prix, images, catégorie, « à la une » ; champs validés	Must	5
25	En tant que administrateur	Je veux gérer les options d'un produit (couleur, taille…)	Afin de proposer des variantes	Ajout/suppression d'options rattachées à un produit depuis EasyAdmin	Must	3
26	En tant que administrateur	Je veux visualiser toutes les commandes et modifier leur statut	Afin de traiter et expédier les commandes	Liste des commandes avec filtre par statut ; statut modifiable (en attente, validée, expédiée, annulée)	Must	3
27	En tant que administrateur	Je veux visualiser les clients et leur historique de commandes	Afin de assurer le suivi client	Liste des utilisateurs ; la fiche d'un client affiche ses commandes	Must	3
28	En tant que administrateur	Je veux modifier les informations et le rôle d'un utilisateur	Afin de nommer un autre administrateur	Rôle modifiable depuis EasyAdmin	Should	1
29	En tant que Mme Leroy (cliente)	Je veux que la plateforme soit en ligne et documentée	Afin de la présenter et la faire évoluer	Application déployée (APP_ENV=prod) avec une URL publique ; README avec les étapes d'installation	Must	3
30	En tant que visiteur	Je veux voir des pages d'erreur 404 et 403 claires	Afin de ne pas être perdu en cas d'erreur	Templates d'erreur Twig personnalisés avec un lien de retour à l'accueil	Could	1

Total story points		83
dont Must	Must	68
dont Should	Should	6
dont Could	Could	9

Légende
Évaluation : priorité MoSCoW — Must (indispensable, exigé par le sujet), Should (important), Could (bonus si le temps le permet), Won't (hors V1).
Story points : estimation relative de l'effort sur l'échelle de Fibonacci (1, 2, 3, 5, 8).
Hors V1 (d'après le sujet) : paiement en ligne, gestion des quantités dans le panier, gestion des stocks.

Backlog du produit : 30 user stories, priorité MoSCoW et story points

J'ai suivi l'avancement sur un tableau **Trello** public (Backlog, À faire, En cours, Fait). Chaque carte correspond à une user story, avec une étiquette de couleur selon sa priorité ; je la déplaçais au moment où je commençais puis terminais la tâche, et l'historique d'activité en garde la trace.

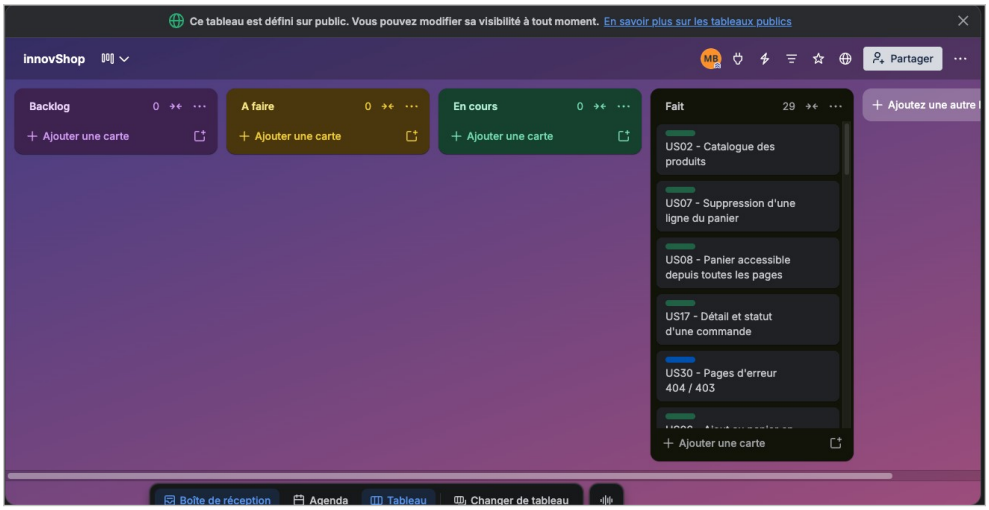


Tableau Trello en fin de projet : les 29 cartes sont dans la colonne « Fait »

Environnement de travail (CP 1)

Vérification des outils. J'ai d'abord contrôlé les versions installées avec php -v, composer -V, symfony -v, node -v et git --version. Symfony 8 demandant PHP 8.4 au minimum, j'ai vérifié que ma version (PHP 8.5, installée avec **Homebrew**) convenait. J'ai aussi vérifié la version exacte de MariaDB, car elle doit être indiquée à Doctrine.

```
innovshop — -zsh — 269×76
Last login: Wed Oct 28 15:06:52 on ttys016
berben@mac-1 ~ % cd ~/innovshop
php -v
composer -V
symfony -v
node -v
git --version
symfony check:requirements
PHP 8.5.10 (cli) (built: Aug 25 2026 21:09:32) (NTS)
Copyright (c) The PHP Group
Built by Homebrew
Zend Engine v4.5.10, Copyright (c) Zend Technologies
    with Zend OPcache v8.5.10, Copyright (c), by Zend Technologies
Deprecation Notice: Using null as an array offset is deprecated, use an empty string instead in /Users/berben/innovshop/vendor/symfony/flex/src/Flex.php:183
Composer version 2.10.3 2026-08-27 13:34:23
PHP version 8.5.10 (/opt/homebrew/Cellar/php/8.5.10/bin/php)
Run the "diagnose" command to get more detailed diagnostics output.
Symfony CLI version 5.22.2 (c) 2021-2026 Fabien Potencier (2026-10-16T16:25:57Z - stable)
Symfony CLI helps developers manage projects, from local code to remote infrastructure

These are common commands used in various situations:

Work on a project locally

  local:new                                                  Create a new Symfony project
  local:server:start                                         Run a local web server
  local:server:stop                                          Stop the local web server
  local:check:security                                       Check security issues in project dependencies
  composer                                                   Runs Composer without memory limit
  console                                                    Runs the Symfony Console (bin/console) for current project
  php, pecl, pear, php-fpm, php-cgi, php-config, phpdbg, phpize, pie  Runs the named binary using the configured PHP version

Manage a project on Cloud

  project:init              Initialize a new project using templates
  cloud:domain:list         Get a list of all domains
  cloud:environment:branch  Branch an environment
  cloud:environment:list    Get a list of environments
  cloud:environment:push    Push code to an environment
  cloud:environment:ssh     SSH to the current environment
  cloud:project:list        Get a list of all active projects
  cloud:tunnel:open         Open SSH tunnels to an app's relationships
  cloud:user:add            Add a user to the project
  cloud:variable:list       List variables

Show all commands with symfony help,
Get help for a specific command with symfony help COMMAND.
v24.15.0
git version 2.50.1 (Apple Git-155)

Symfony Requirements Checker
~~~~~~~~~~~~~~~~~~~~~~~~~~~~

> PHP is using the following php.ini file:
/opt/homebrew/etc/php/8.5/php.ini

> Checking Symfony requirements:

.....................................

[OK]
Your system is ready to run Symfony projects

Note  The command console can use a different php.ini file
~~~~  than the one used by your web server.
      Please check that both the console and the web server
      are using the same PHP version and configuration.
```

Vérification des outils : PHP 8.5.10, Composer 2.10.3, Symfony CLI 5.22.2, Node.js 24, Git 2.50, puis symfony check:requirements

Création du projet. J'ai créé le projet avec « symfony new innovshop --webapp », qui installe Symfony et les composants d'une application web (Twig, Doctrine, sécurité, formulaires, Mailer, MakerBundle). J'ai lancé le serveur local avec « symfony server:start -d » et vérifié la page d'accueil de Symfony.

Configuration de l'environnement. J'ai créé un fichier **.env.local**, non versionné, pour y placer l'adresse de la base de données sans modifier le fichier .env partagé :

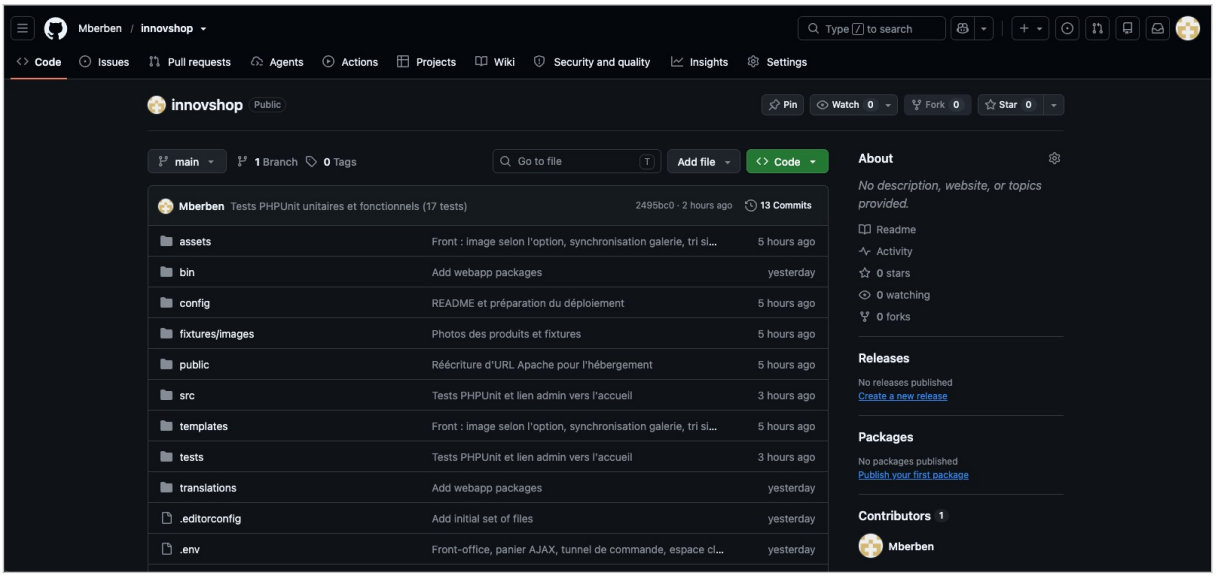
```
.env.local
DATABASE_URL="mysql://root:@127.0.0.1:3306/innovshop?serverVersion=10.4.28-MariaDB&charset=utf8mb4"
```

Fichier .env.local : connexion à la base MariaDB locale

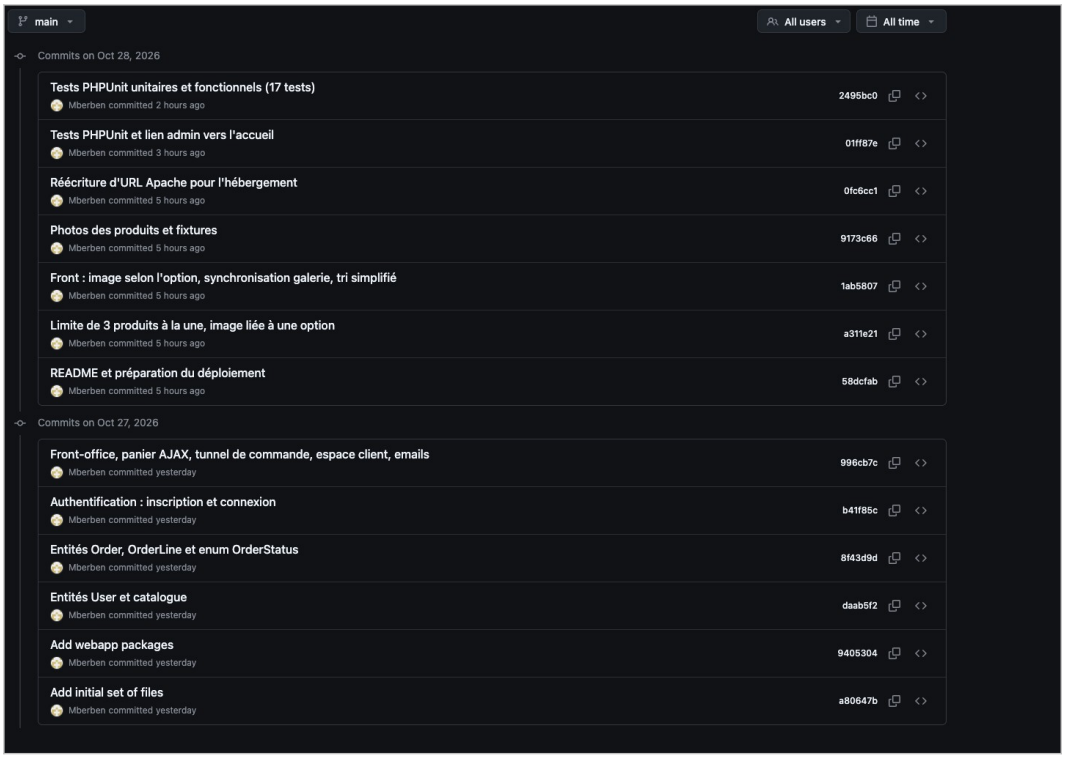
Lors de la création de la base, Symfony tentait de se connecter à PostgreSQL : mon fichier .env.local était vide, donc la valeur par défaut de .env s'appliquait. J'ai corrigé le fichier, puis créé la base avec « php bin/console doctrine:database:create ». La commande « debug:dotenv » m'a permis de vérifier quelle valeur Symfony utilisait réellement.

Outils de débogage. J'utilise la barre de débogage et le **profiler** de Symfony pour contrôler les requêtes SQL, les codes de réponse HTTP, l'utilisateur connecté et les emails envoyés.

Le code est versionné avec **Git** et publié sur **GitHub** (dépôt public Mberben/innovshop). J'ai fait un commit à la fin de chaque étape fonctionnelle et testée, avec un message descriptif (« Entités Order, OrderLine et enum OrderStatus », « Authentification : inscription et connexion »…). Le .gitignore exclut .env.local, vendor/, var/ et les images envoyées.



Dépôt GitHub public Mberben/innovshop



Historique des commits : un commit par étape, les 27 et 28 octobre

Architecture et base de données avec Doctrine (CP 5)

Symfony organise l'application selon le modèle **MVC**. Par exemple, pour /produit/montre-vita-sport, le routeur appelle ProductController::show(), Doctrine charge l'entité Product correspondant au slug (ou renvoie une erreur 404), puis le contrôleur transmet le produit à la vue Twig. J'ai ajouté une couche de **services** (CartService, OrderMailer) pour que les contrôleurs restent courts.

Modélisation. J'ai réalisé un diagramme de classes avant d'écrire le code : 7 entités (User, Category, Product, ProductImage, ProductOption, Order, OrderLine) et l'énumération OrderStatus.

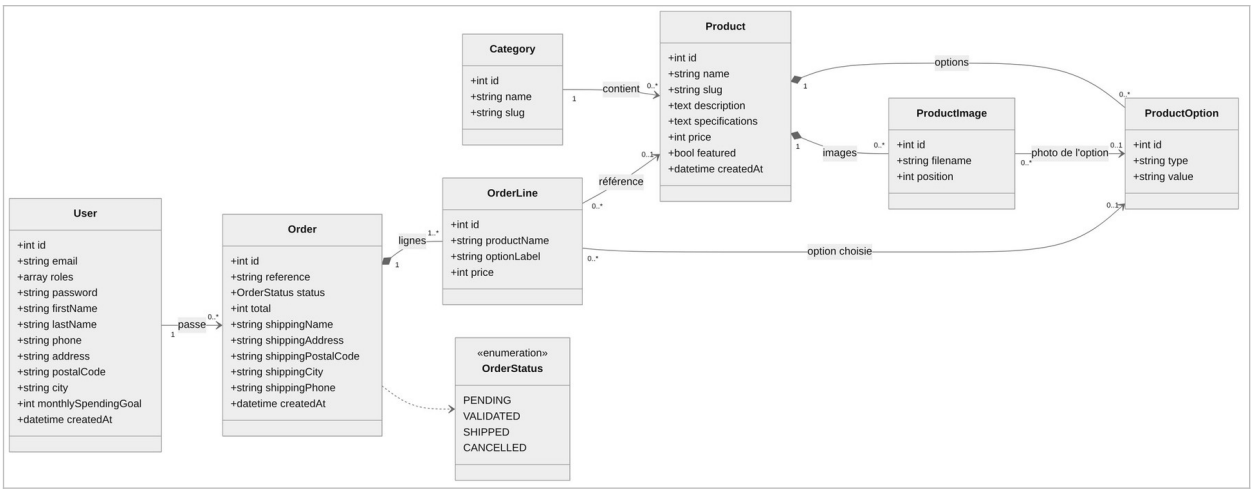


Diagramme de classes d'InnovShop (prix et montants stockés en centimes, type int)

Les relations sont traduites en clés étrangères :

- une catégorie contient des produits (1 → 0..n, product.category_id facultative) ;
- un produit a des images et des options (1 → 0..n, product_image.product_id et product_option.product_id obligatoires) ;
- un client passe des commandes (1 → 0..n, order.user_id obligatoire) ;
- une commande contient des lignes (1 → 1..n, order_line.customer_order_id obligatoire) ;
- une ligne référence un produit et une option, et une image peut illustrer une option (n → 0..1, ON DELETE SET NULL).

Choix de conception pour garantir l'intégrité des données :

- **Prix en centimes** dans des colonnes entières (INT) : 19,99 € est stocké 1999. Les nombres à virgule flottante créent des erreurs d'arrondi incompatibles avec des montants ;
- **Données copiées dans la commande** : chaque ligne enregistre le nom, l'option et le prix au moment de l'achat, et la commande enregistre l'adresse de livraison. Une modification du catalogue ou du profil ne fausse jamais l'historique ;
- **ON DELETE SET NULL** sur les liens d'une ligne de commande : on peut supprimer un produit déjà vendu sans casser les commandes existantes ;
- **Statut sous forme d'énumération** : la colonne status ne peut contenir que pending, validated, shipped ou cancelled ;
- **Contrainte d'unicité** sur l'email des utilisateurs ;
- **Table order** : ORDER étant un mot réservé en SQL, le nom de la table est protégé par des guillemets obliques, et la relation côté ligne s'appelle customerOrder.

Création avec Doctrine. J'ai créé les entités avec make:user puis make:entity, en choisissant pour chaque champ son type, sa longueur et s'il accepte la valeur NULL. Par exemple, le prénom et le nom sont obligatoires dès l'inscription, alors que l'adresse et le téléphone peuvent rester vides jusqu'à la première commande.

Migrations. Chaque évolution du schéma passe par une migration : make:migration compare les entités à la base et génère le SQL, que je relis avant de l'appliquer avec doctrine:migrations:migrate. Exemple généré quand j'ai corrigé une faute de frappe dans un nom de champ :

migrations/ – SQL généré par make:migration

```sql
ALTER TABLE `order` CHANGE shipping_adress shipping_address VARCHAR(255) NOT NULL;
```

SQL généré par une migration Doctrine

Jeu de données de test. J'ai écrit des fixtures (AppFixtures) qui remplissent la base avec 5 catégories, 15 produits avec leurs options et leurs photos, 3 comptes et 5 commandes. Les mots de passe y sont hachés comme lors d'une vraie inscription. Une base séparée, innovshop_test, sert aux tests automatisés.

Accès aux données (CP 6)

Dans InnovShop, tous les accès à la base passent par **Doctrine**. J'ai regroupé les requêtes dans les **repositories**, jamais dans les contrôleurs ni dans les templates.

Requêtes simples. Pour l'accueil, deux méthodes renvoient les 3 derniers produits et les 3 produits à la une :

src/Repository/ProductRepository.php

```php
public function findFeatured(int $limit = 3): array
{
    return $this->findBy(['featured' => true], ['createdAt' => 'DESC'], $limit);
}
```

ProductRepository : produits à la une

Recherche multicritère avec le QueryBuilder. Le catalogue combine un mot-clé, une catégorie, une fourchette de prix et un tri. J'ai écrit une méthode search() qui n'ajoute une condition que si le critère est renseigné :

src/Repository/ProductRepository.php – search()

```php
$qb = $this->createQueryBuilder('p')
    ->leftJoin('p.category', 'c')->addSelect('c')
    ->leftJoin('p.images', 'i')->addSelect('i');

if ($q) {
    $qb->andWhere('p.name LIKE :q OR p.description LIKE :q')
       ->setParameter('q', '%'.$q.'%');
}
if (null !== $maxEuros) {
    $qb->andWhere('p.price <= :max')->setParameter('max', $maxEuros * 100);
}
```

ProductRepository : recherche multicritère

Toutes les valeurs passent par setParameter() : ce sont des requêtes préparées, ce qui protège contre l'injection SQL. Les jointures avec addSelect() chargent la catégorie et les images dans la même requête, au lieu d'une requête supplémentaire par produit. J'ai vérifié le nombre de requêtes exécutées dans l'onglet Doctrine du profiler.

Requêtes d'agrégation. Pour l'objectif de dépenses de l'espace client, une requête additionne les commandes du mois du client, en excluant les commandes annulées. Une autre méthode calcule le chiffre d'affaires par jour sur 14 jours pour le graphique du tableau de bord administrateur.

src/Repository/OrderRepository.php

```php
return (int) $this->createQueryBuilder('o')
    ->select('COALESCE(SUM(o.total), 0)')
    ->andWhere('o.user = :user')
    ->andWhere('o.createdAt >= :start')
    ->andWhere('o.status != :cancelled')
    ->setParameter('user', $user)
    ->setParameter('start', new \DateTimeImmutable('first day of this month midnight'))
    ->setParameter('cancelled', OrderStatus::CANCELLED)
    ->getQuery()
    ->getSingleScalarResult();
```

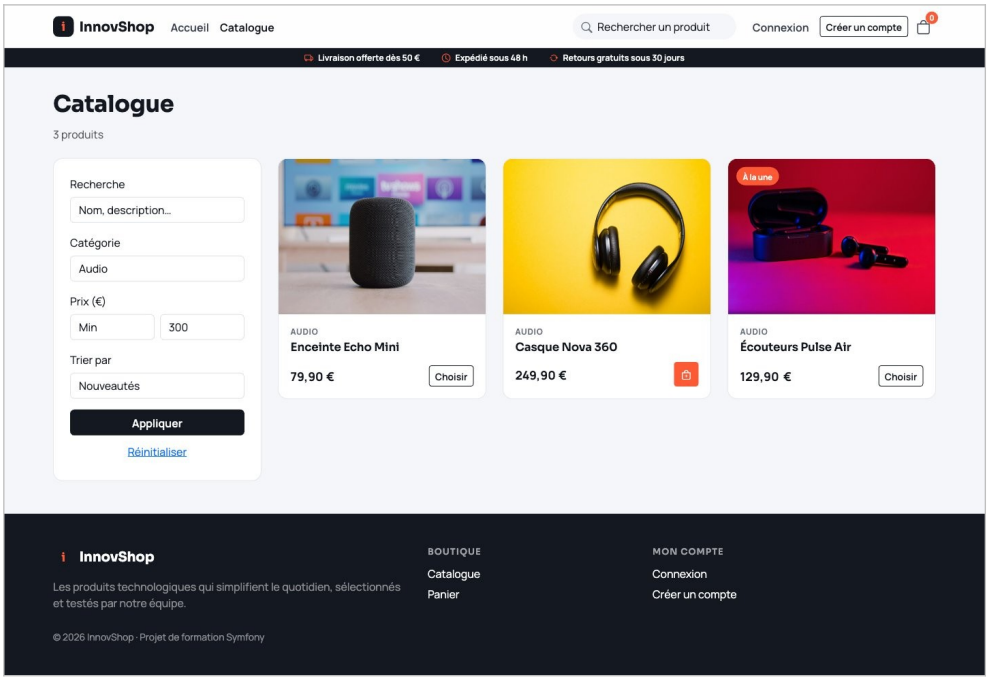
OrderRepository : total des commandes du mois

Écriture des données. À la validation d'une commande, je crée l'objet Order et ses OrderLine, puis j'enregistre le tout en une fois avec persist() et flush(). Grâce à l'option cascade: ['persist'], les lignes sont enregistrées avec la commande dans la même transaction.

Contrôle d'accès aux données. Un client ne peut lire que ses propres commandes : les requêtes filtrent sur l'utilisateur connecté et un voter refuse l'accès direct par URL à la commande d'un autre client.

Données hors base relationnelle. Le panier est stocké dans la session, sous forme de tableau d'identifiants ; il est rechargé depuis la base à chaque affichage pour toujours présenter les prix à jour.

Interface dynamique du front-office (CP 4)



Catalogue avec un filtre appliqué (catégorie, fourchette de prix, tri)

Dans InnovShop, le cahier des charges demandait un panier « accessible à tout moment et qui reflète les changements en temps réel ». J'ai développé plusieurs comportements dynamiques en JavaScript, sans bibliothèque externe, dans le fichier assets/cart.js.

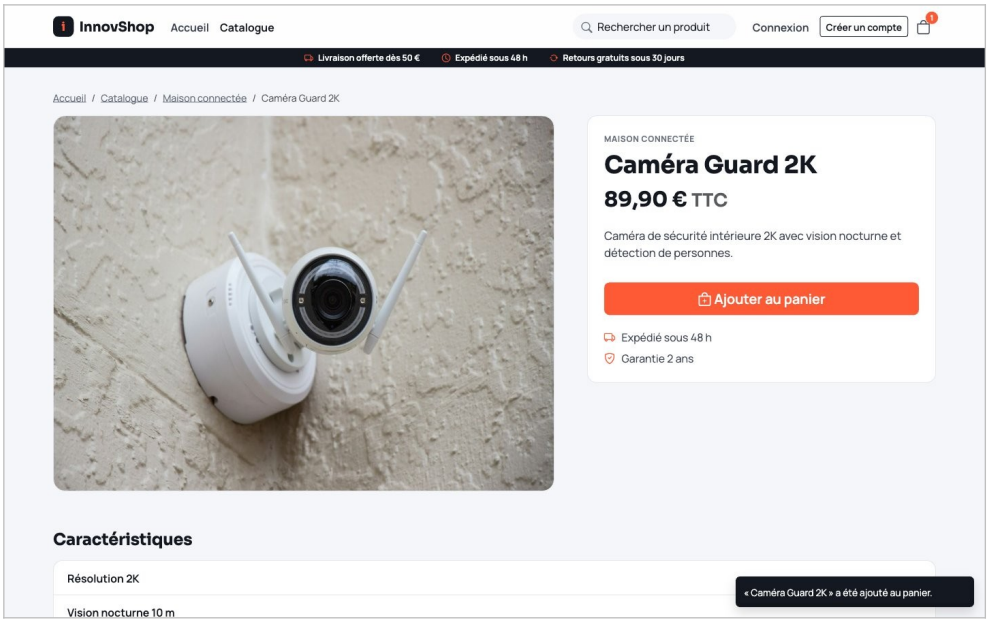
Ajout au panier sans rechargement (AJAX). Les boutons « Ajouter au panier » sont de vrais formulaires HTML. Mon script intercepte leur envoi, l'envoie au serveur avec fetch() en demandant une réponse JSON, puis met à jour le compteur du menu et affiche une notification.

assets/cart.js

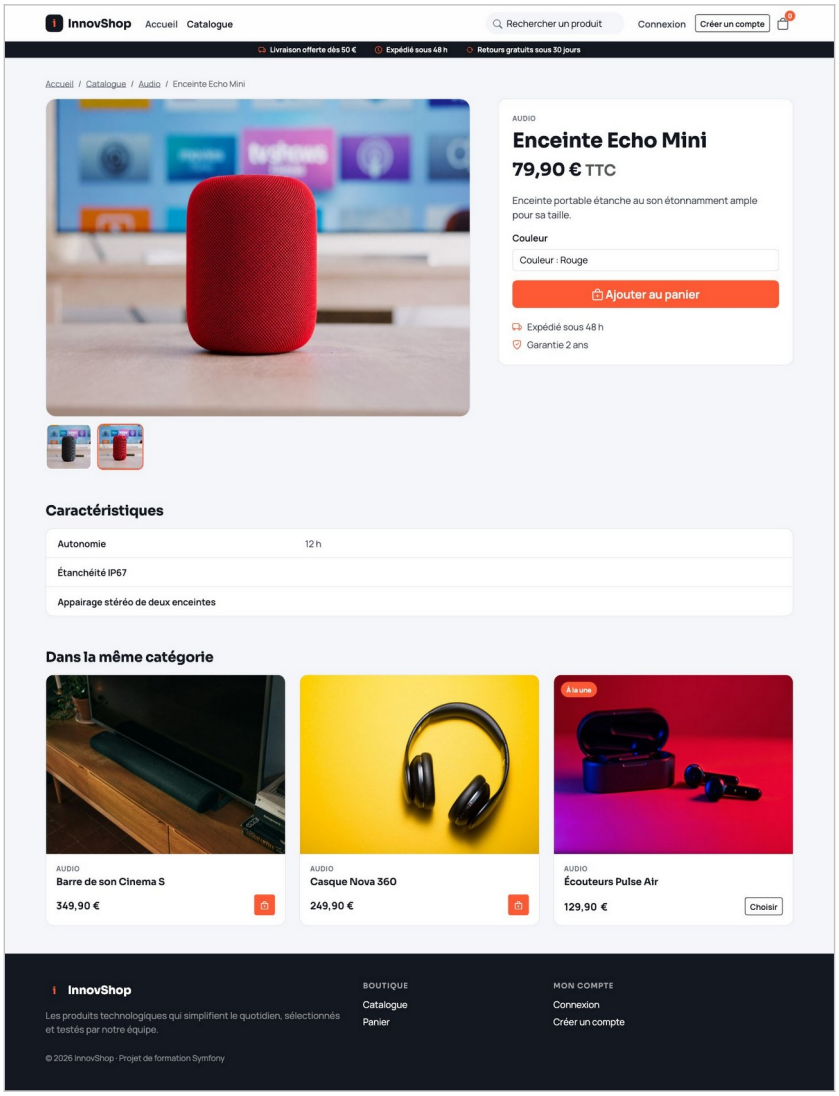
```
document.addEventListener('submit', async (event) => {
    const form = event.target;
    if (form.matches('[data-cart-add]')) {
        event.preventDefault();
        const response = await fetch(form.action, {
            method: 'POST',
            body: new FormData(form),
            headers: { Accept: 'application/json' },
        });
        const data = await response.json();
        notify(data.message, data.success);
        if (data.success) {
            updateCount(data.count);
        }
    }
});
```

assets/cart.js : ajout au panier en AJAX

J'utilise la **délégation d'événements** (un seul écouteur sur le document) pour que le script fonctionne sur toutes les pages, y compris pour les cartes produits du catalogue. Côté serveur, le contrôleur répond en JSON pour un appel AJAX et par une redirection sinon : si JavaScript est désactivé ou si le réseau échoue, le formulaire est envoyé normalement (**amélioration progressive**).



Notification « ajouté au panier » et compteur du menu mis à jour sans rechargement

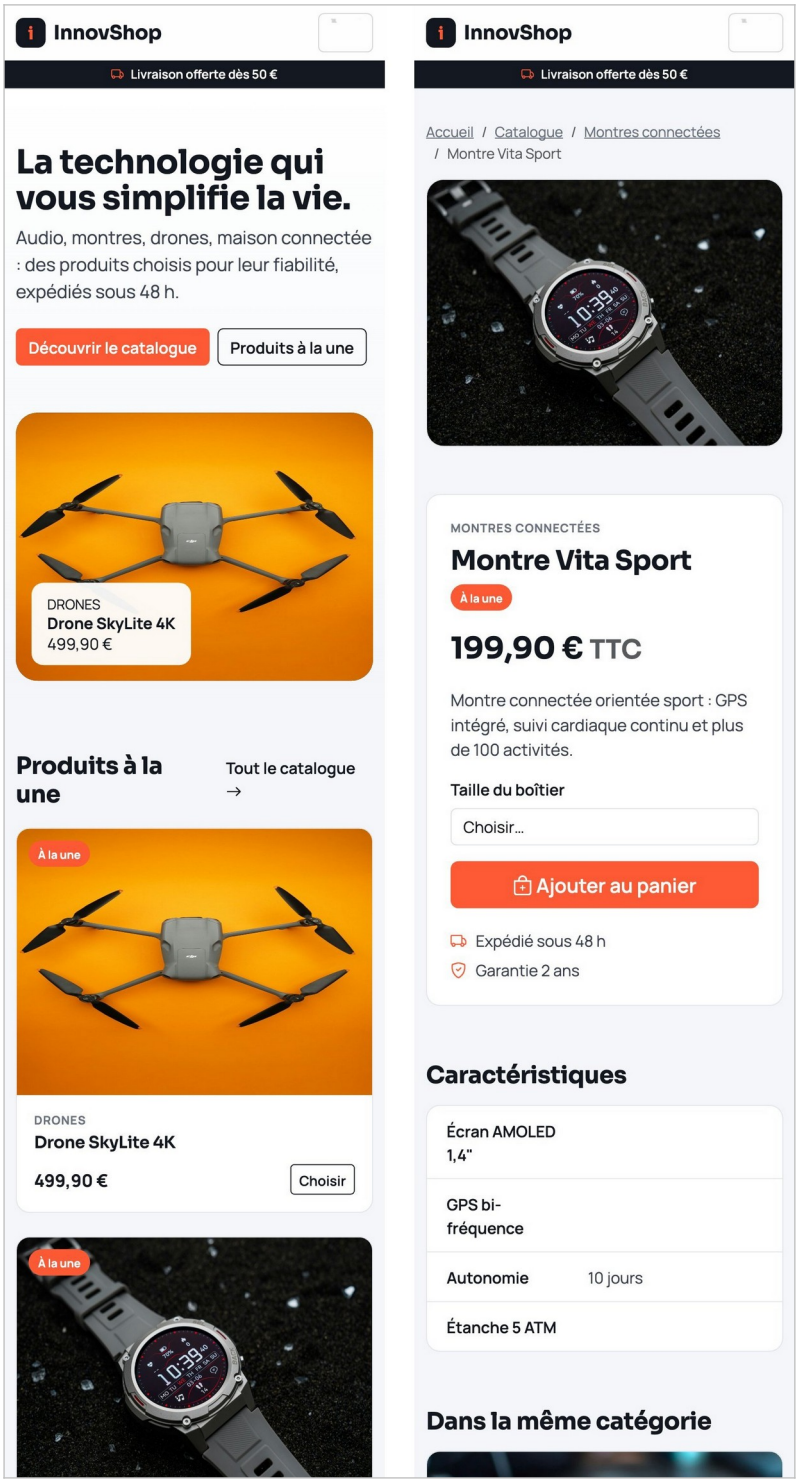


Fiche produit : l'option Rouge est sélectionnée et la photo rouge s'affiche

Suppression d'une ligne du panier. La ligne s'efface avec une transition, le total et le compteur se mettent à jour, et la page se recharge uniquement quand le panier devient vide, pour afficher le message « Votre panier est vide ».

Galerie et options synchronisées. Sur la fiche produit, cliquer sur une miniature change l'image principale. Chaque option de la liste déroulante (par exemple « Couleur : Blanc ») porte l'adresse de sa photo dans un attribut data-image : choisir une option affiche la photo correspondante, et inversement. Au chargement, l'image s'aligne sur l'option déjà sélectionnée (cas du retour arrière du navigateur).

Interface pilotée par les données avec Twig. Les pages sont générées à partir de la base : boucles sur les produits, conditions (badge « À la une », bouton « Choisir » pour un produit avec options, menu différent selon que l'utilisateur est connecté ou administrateur). J'ai créé des éléments réutilisables (carte produit, bouton panier, étapes de commande) et un filtre Twig price pour afficher les prix.



Version mobile (375 px) : accueil et fiche produit

Composants métier côté serveur (CP 7)

J'ai développé la logique métier d'InnovShop en séparant les responsabilités : les contrôleurs reçoivent la requête et renvoient la réponse, les **services** portent les règles métier.

Service de panier (CartService). Il ajoute, retire et liste les articles, calcule le total et le nombre d'articles.

Conformément au cahier des charges, il n'y a pas de quantité : chaque ajout crée une ligne avec un identifiant unique.

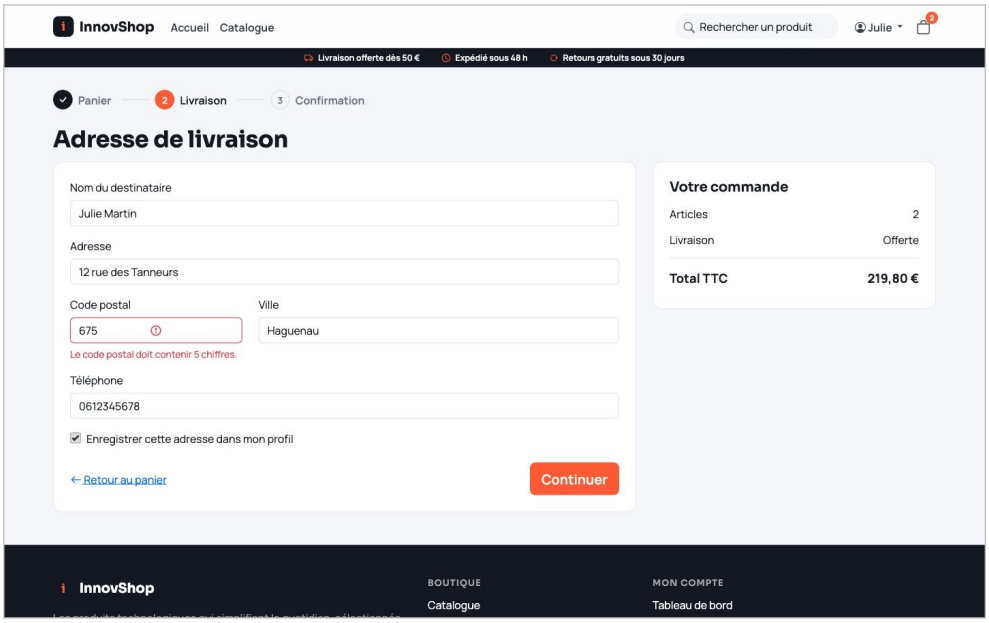
src/Service/CartService.php

```php
public function add(Product $product, ?ProductOption $option = null): void
{
    $lines = $this->getRawLines();
    $lines[] = [
        'id' => bin2hex(random_bytes(6)),
        'product' => $product->getId(),
        'option' => $option?->getId(),
    ];
    $this->save($lines);
}
```

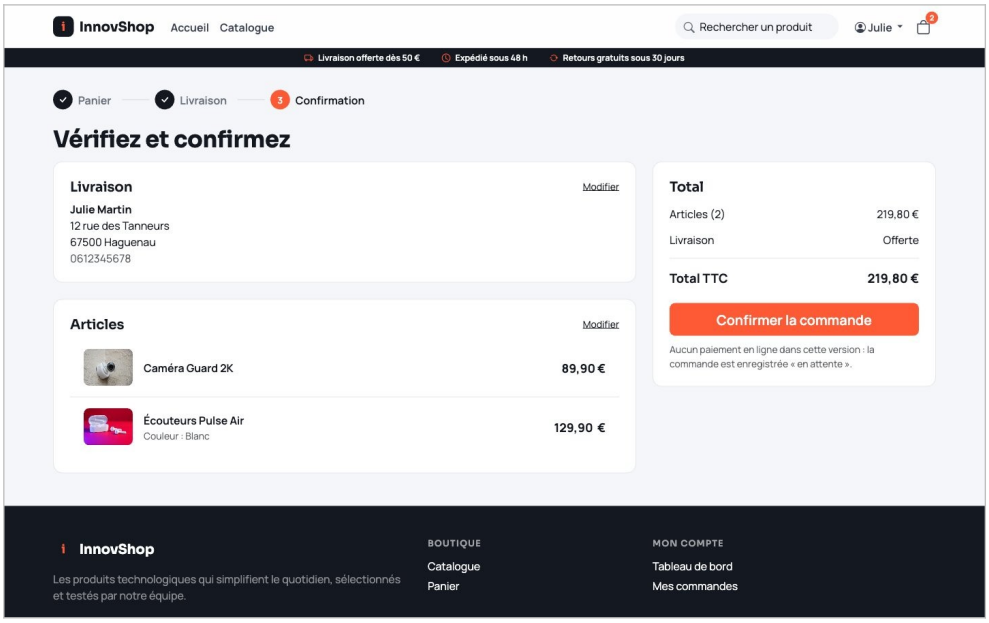
CartService : ajout d'un article au panier

Contrôleur du panier. Avant d'ajouter un produit, il vérifie le **jeton CSRF** et que l'option envoyée appartient bien au produit, pour qu'un utilisateur ne puisse pas falsifier la requête.

Tunnel de commande (CheckoutController). Il est réservé aux utilisateurs connectés et suit trois étapes. À l'étape de livraison, le formulaire est validé par des contraintes déclarées sur l'entité Order (champs obligatoires, code postal à 5 chiffres, numéro de téléphone français par expression régulière). À la validation, il crée la commande avec une référence unique, copie chaque ligne du panier, **recalcule le total côté serveur**, enregistre le tout, vide le panier et envoie l'email de confirmation.

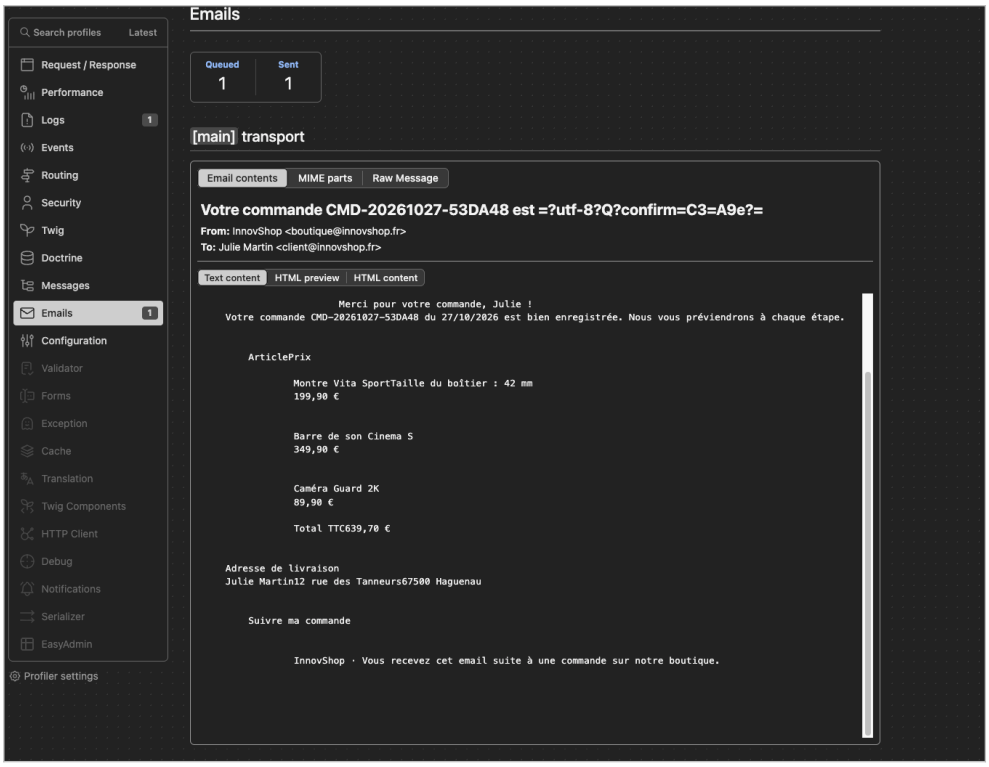


Étape « Livraison » : erreurs de validation affichées sous les champs

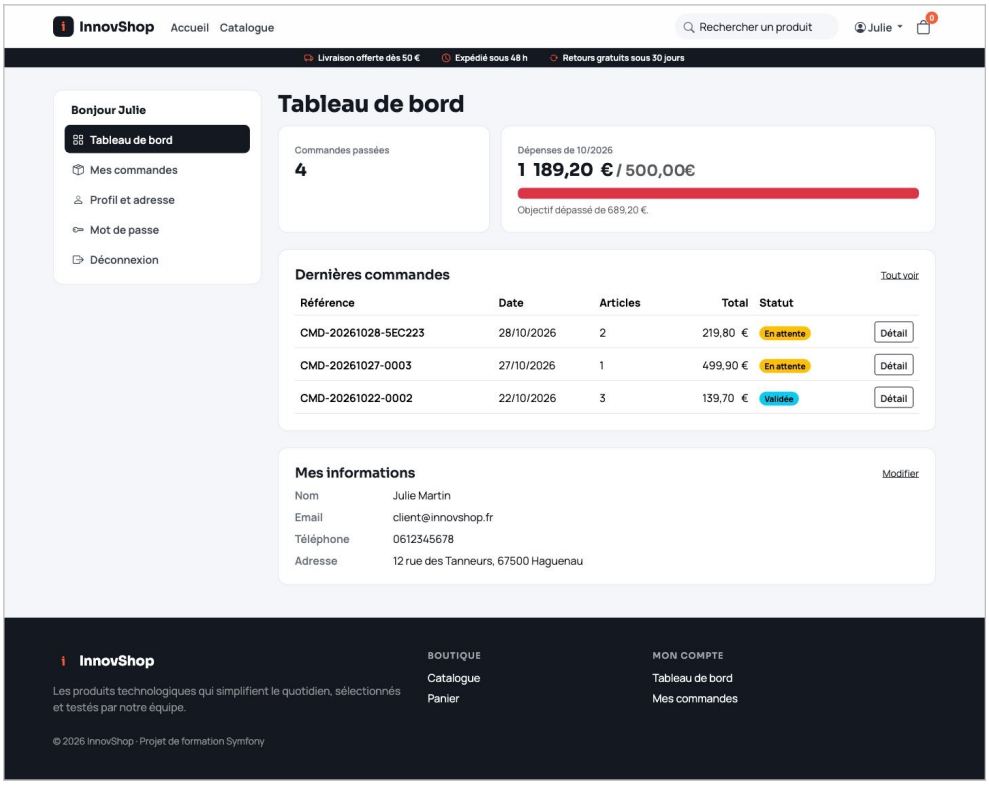


Étape « Confirmation » : récapitulatif avant validation

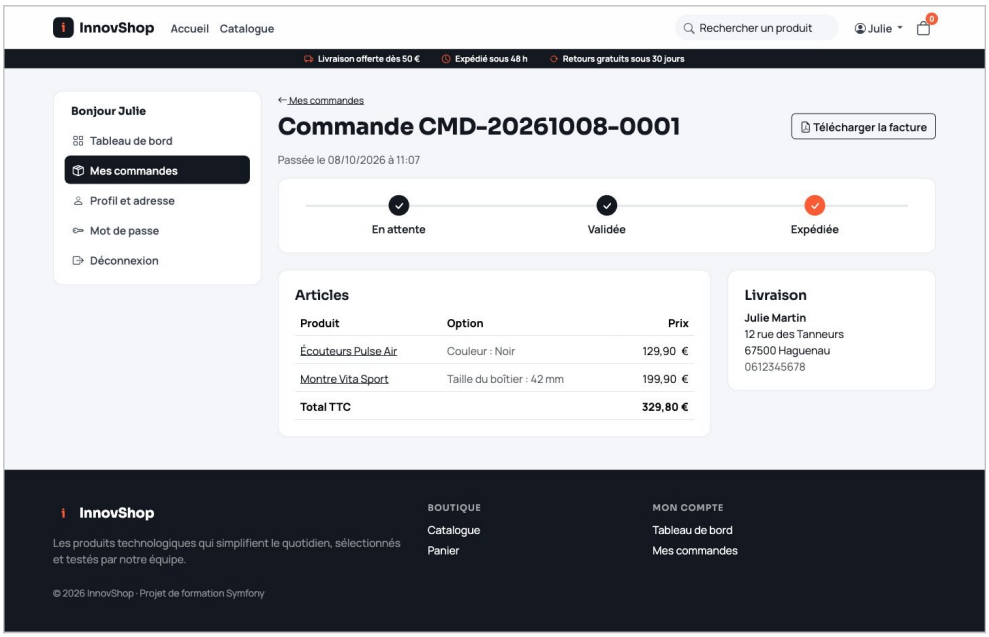
Emails (OrderMailer). Ce service construit les emails avec des templates Twig. Un listener Doctrine (OrderStatusListener) détecte le changement du champ status d'une commande (preUpdate) et envoie une notification au client une fois la modification enregistrée (postUpdate). Les envois sont protégés par un try/catch : une panne du serveur mail ne bloque jamais une commande.



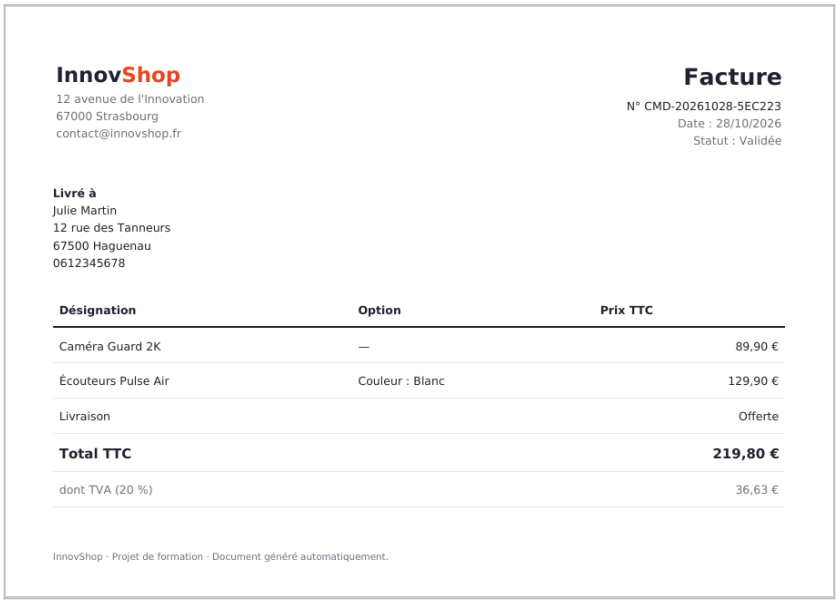
Email récapitulatif de commande, vu dans l'onglet Emails du profiler Symfony



Espace client : tableau de bord et objectif de dépenses du mois



Détail d'une commande avec le suivi visuel des étapes



InnovShop
12 avenue de l'Innovation
67000 Strasbourg
contact@innovshop.fr

Facture
N° CMD-20261028-5EC223
Date : 28/10/2026
Statut : Validée

Livré à
Julie Martin
12 rue des Tanneurs
67500 Haguenau
0612345678

Désignation	Option	Prix TTC
Caméra Guard 2K	—	89,90 €
Écouteurs Pulse Air	Couleur : Blanc	129,90 €
Livraison		Offerte
Total TTC		**219,80 €**
dont TVA (20 %)		36,63 €

InnovShop · Projet de formation · Document généré automatiquement.

Facture PDF générée par Dompdf depuis l'espace client

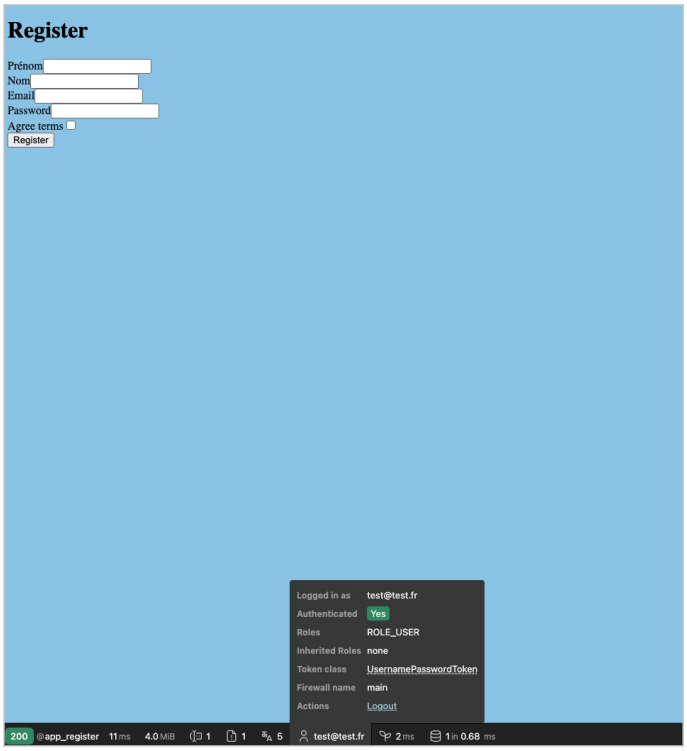
Sécurité et droits d'accès. Authentification par formulaire, mots de passe hachés, option « Se souvenir de moi ». Les rôles ROLE_USER et ROLE_ADMIN sont contrôlés à deux niveaux : access_control dans security.yaml et attribut #[IsGranted] sur les contrôleurs. Le voter **OrderVoter** n'autorise un client qu'à consulter ses propres commandes, un administrateur toutes.

src/Security/Voter/OrderVoter.php

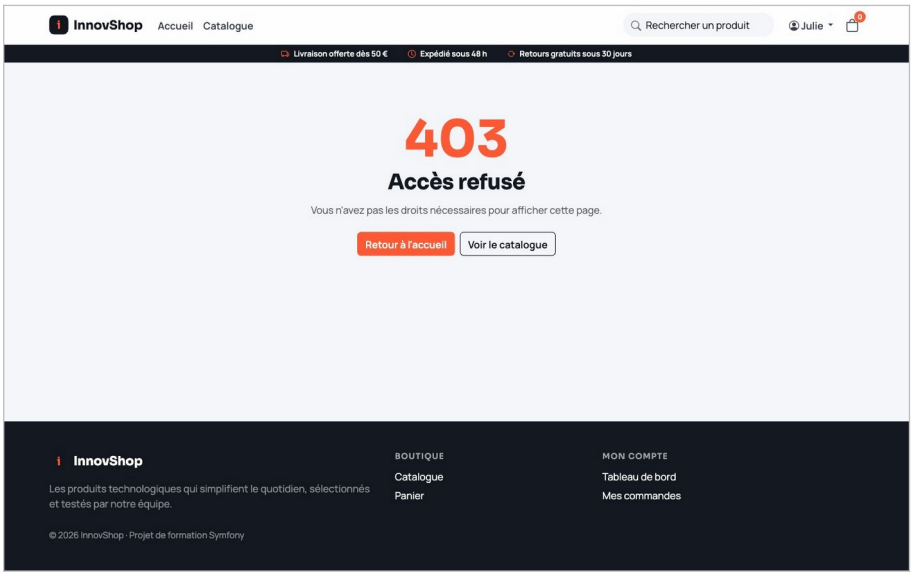
```php
if (\in_array('ROLE_ADMIN', $user->getRoles(), true)) {
    return true;
}

return $subject->getUser() === $user;
```

OrderVoter : un client ne voit que ses commandes

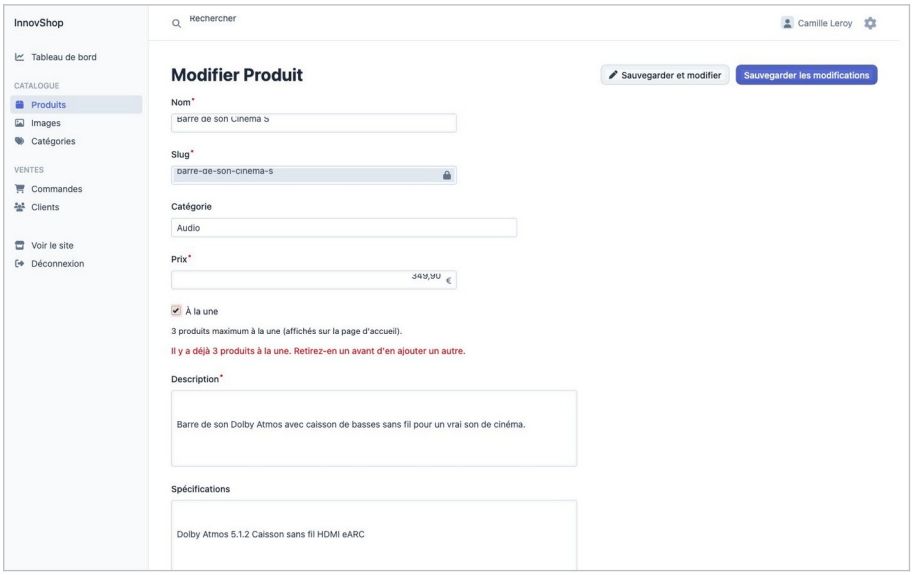


Barre du profiler : utilisateur connecté avec le rôle ROLE_USER



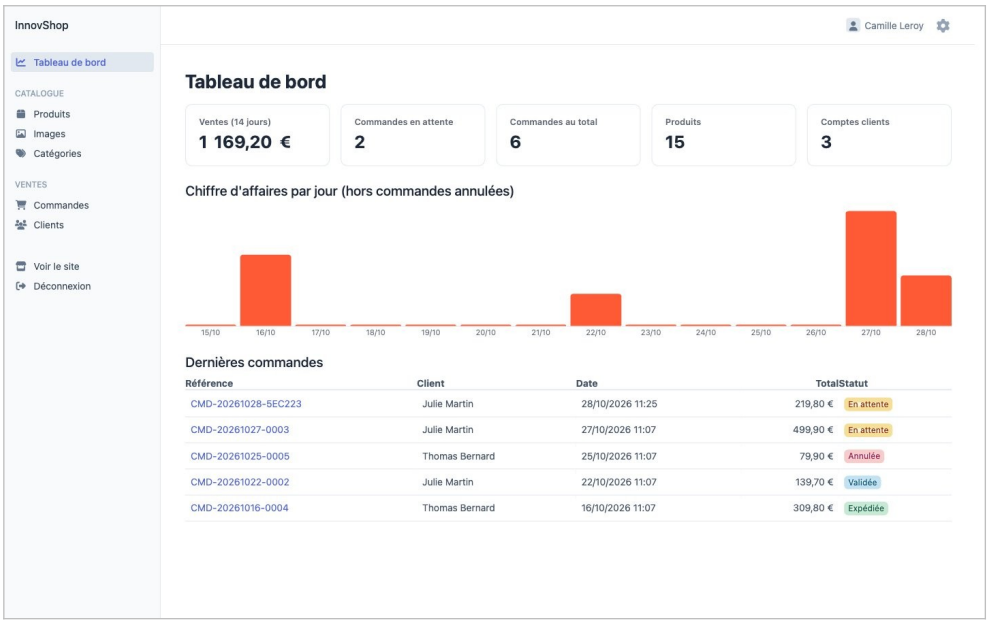
Page 403 personnalisée (client qui tente d'accéder au back-office)

Règle métier personnalisée. L'accueil affiche 3 produits à la une. J'ai créé une contrainte de validation #[MaxFeatured(max: 3)] et son validateur, qui interroge la base et empêche un administrateur d'en sélectionner un quatrième.

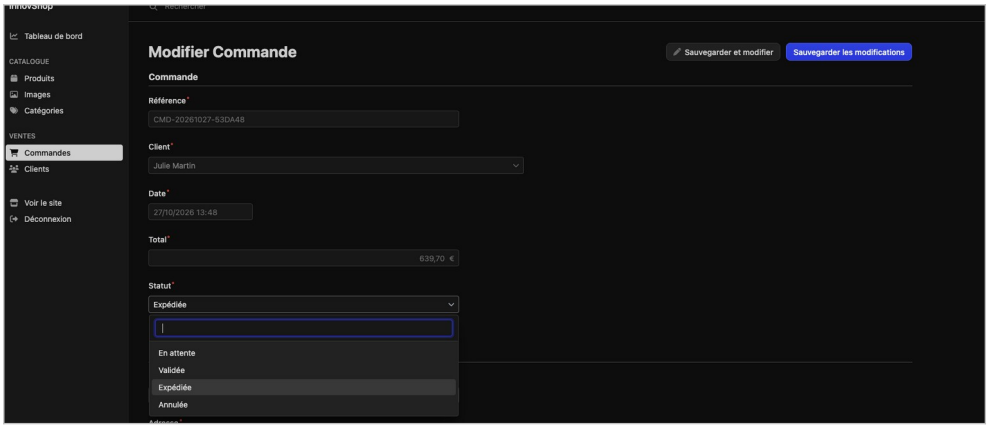


Back-office : la contrainte MaxFeatured bloque un quatrième produit à la une

Back-office EasyAdmin. Tableau de bord (chiffre d'affaires sur 14 jours, commandes en attente, dernières commandes), gestion des produits avec leurs options et images, des catégories, des commandes (seul le statut est modifiable ; création et suppression désactivées) et des clients avec l'historique de leurs commandes.

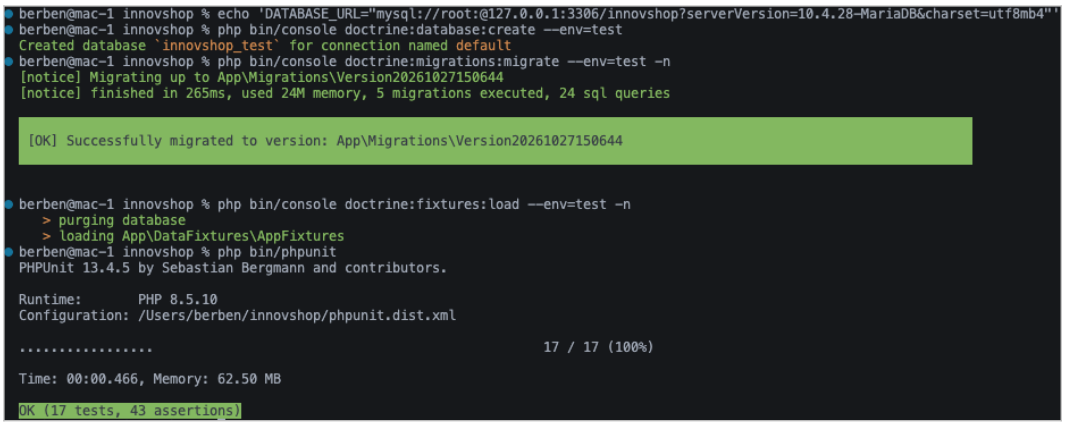


Back-office : tableau de bord avec le chiffre d'affaires sur 14 jours



Modification d'une commande : choix du nouveau statut, qui déclenche l'email au client

Facture et tests. La facture PDF est générée avec **Dompdf** à partir d'un template Twig. J'ai écrit **17 tests PHPUnit** (43 assertions) : tests unitaires (PriceFilterTest, OrderStatusTest, ProductImageTest) et tests fonctionnels (PublicPagesTest, AccessControlTest : redirection des visiteurs, 403 pour un client sur /admin, voter, ajout au panier en JSON). Ils tournent sur une base dédiée innovshop_test, remplie avec les fixtures. Avant chaque commit, j'ai aussi testé le parcours à la main en contrôlant dans le profiler les codes HTTP (422 pour un formulaire invalide, 302 après une validation réussie).



Préparation de la base de test puis php bin/phpunit : 17 tests, 43 assertions, tous au vert

Documentation et déploiement (CP 8)

Projet 2 : documentation et déploiement d'InnovShop (octobre 2026)

Préparation au déploiement :

- ajout d'un fichier public/.htaccess pour qu'Apache transmette toutes les URL à Symfony ;
- envoi des emails en direct, sans file d'attente, pour éviter d'avoir un processus permanent à faire tourner sur un hébergement mutualisé ;
- données de démonstration disponibles en production, avec les photos des produits compressées (de 38 Mo à 1,2 Mo) pour un chargement rapide ;
- secrets (base de données, clé de l'application, serveur mail) gardés hors du dépôt, dans un fichier .env.local propre au serveur.

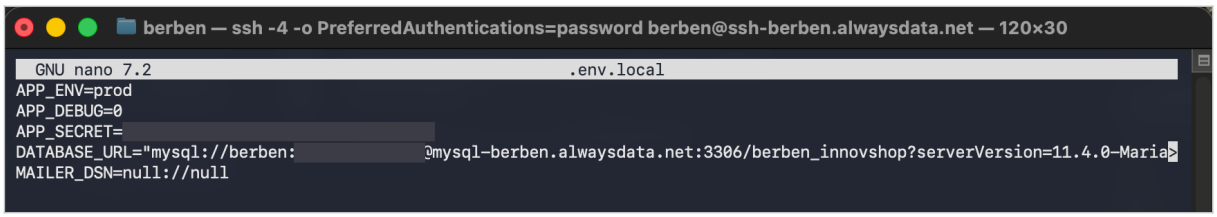
Mise en production sur alwaysdata :

1. Création de la base de données MariaDB berben_innovshop et activation de l'accès SSH dans l'interface d'alwaysdata.
2. Connexion au serveur en SSH et récupération du code avec git clone.
3. Création du fichier .env.local de production (APP_ENV=prod, APP_SECRET, DATABASE_URL, MAILER_DSN).
4. Installation des dépendances sans les outils de développement : composer install --no-dev --optimize-autoloader.
5. Création des tables avec doctrine:migrations:migrate, puis chargement des données de démonstration.
6. Préparation des fichiers CSS et JavaScript : importmap:install et asset-map:compile.
7. Vidage du cache de production.
8. Configuration du site dans alwaysdata, avec le dossier public/ comme racine.
9. Tests en ligne : inscription, commande, email, espace client, back-office, pages d'erreur.

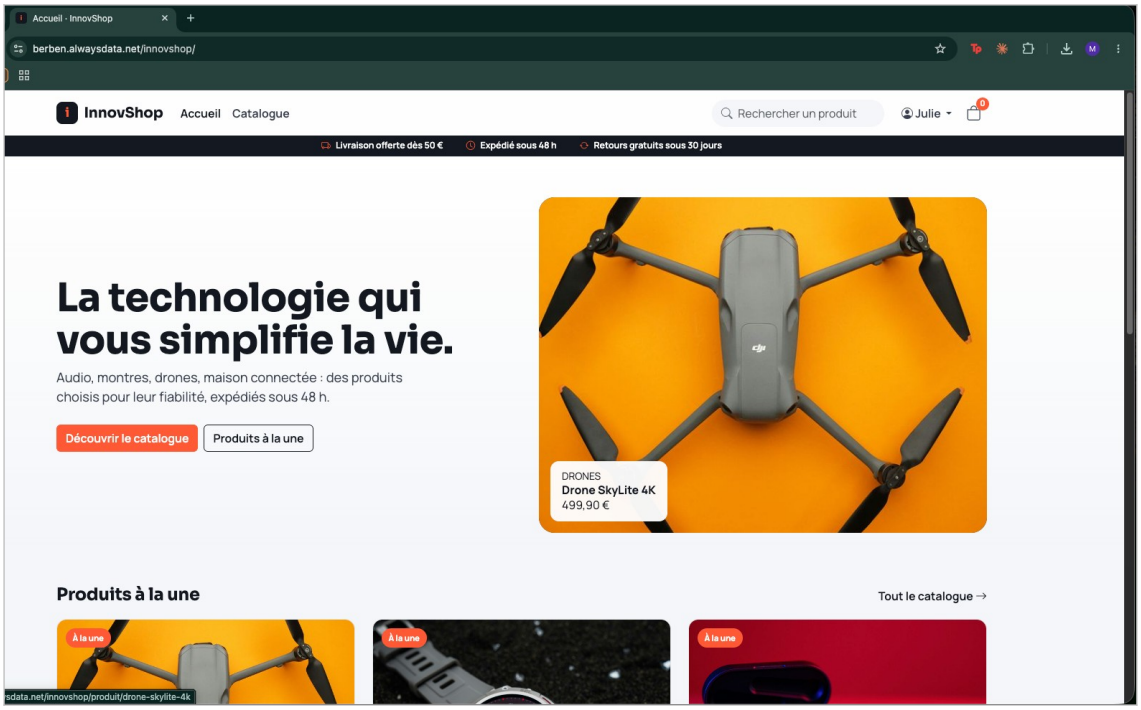
Le site est en ligne à l'adresse https://berben.alwaysdata.net/innovshop/, à côté du portfolio et du site du cabinet sur le même hébergement.

Difficultés rencontrées. J'ai d'abord tenté la même méthode que pour le cabinet : préparer le projet sur mon ordinateur et l'envoyer avec FileZilla. Mais le dossier vendor/ contient environ 9 000 fichiers : l'envoi était très lent, et une partie des transferts a échoué quand j'ai augmenté le nombre de connexions simultanées. J'ai aussi constaté qu'il ne fallait pas envoyer le dossier var/cache, qui contient les chemins de mon ordinateur. J'ai donc changé de méthode : après avoir activé la connexion SSH, j'ai récupéré le code avec git clone et installé les dépendances directement sur le serveur avec Composer. C'est plus rapide, reproductible, et c'est la procédure décrite dans le README.

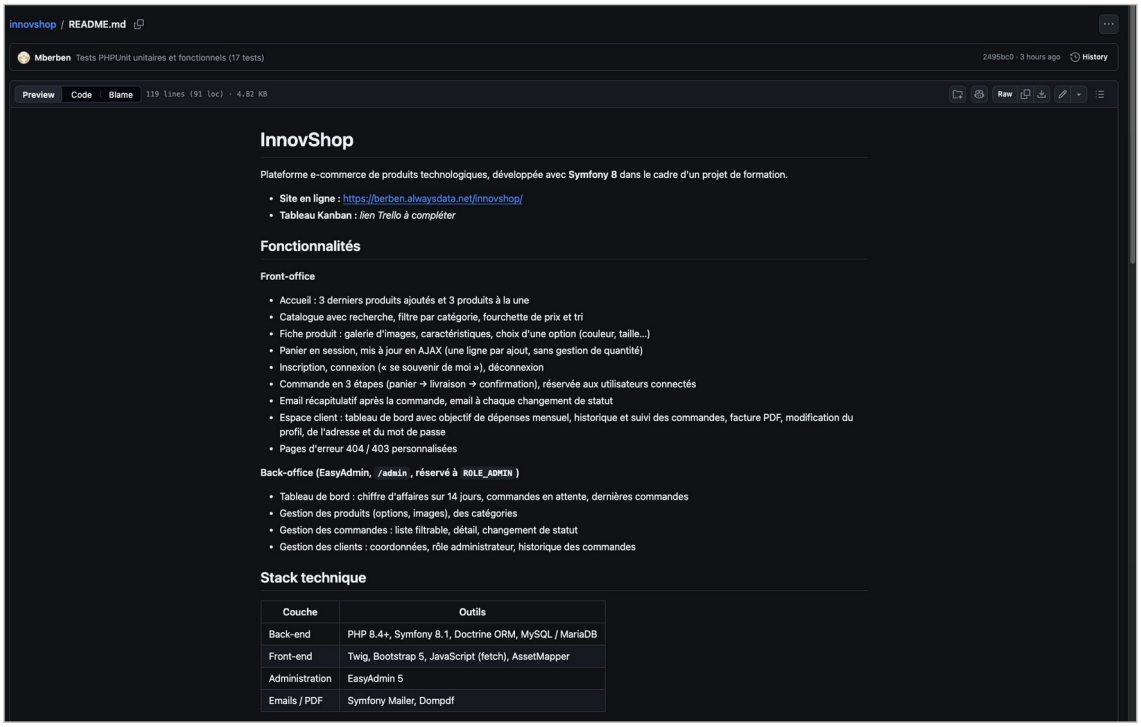
J'ai aussi dû distinguer les différents identifiants d'alwaysdata (compte, FTP, SSH, MySQL), car une erreur « Access denied » venait d'un mauvais mot de passe MySQL, que j'ai vérifié avec le client mysql en ligne de commande. Enfin, le site étant servi sous /innovshop/, j'ai remplacé un lien écrit en dur (/) par une route Symfony pour que les adresses restent correctes.



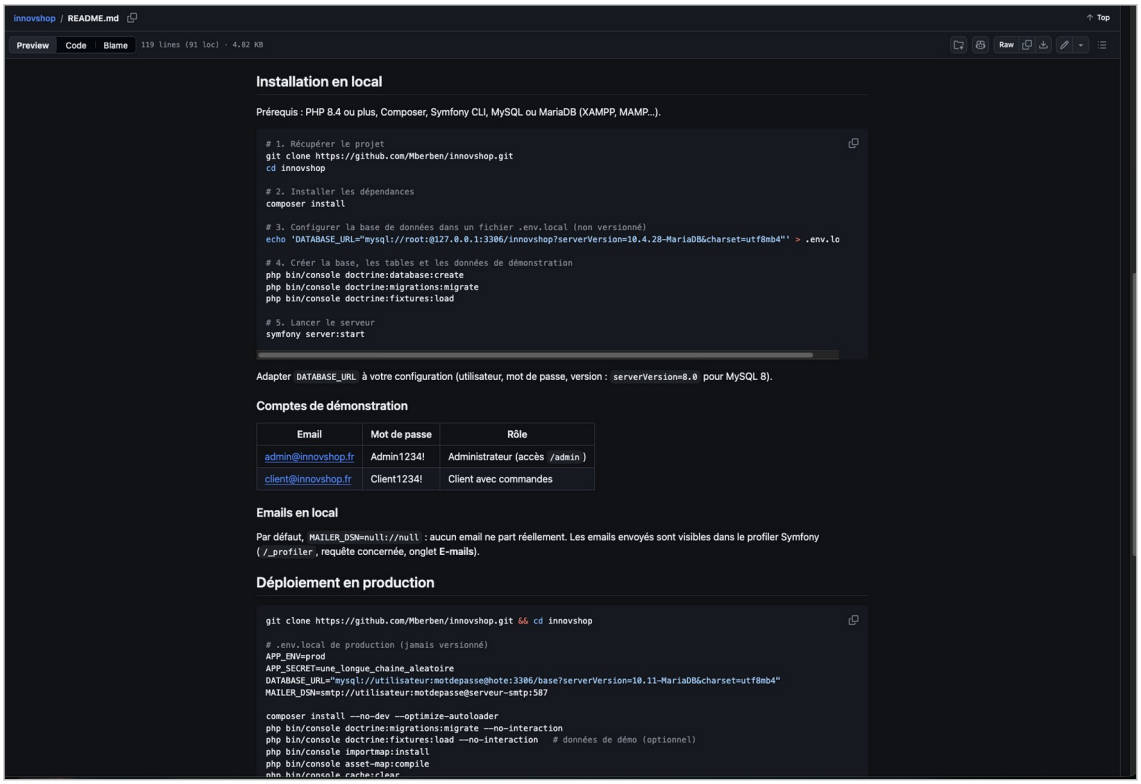
Session SSH sur alwaysdata : création du .env.local de production avec nano (APP_ENV=prod, APP_DEBUG=0 ; secrets masqués)



InnovShop en ligne à l'adresse berben.alwaysdata.net/innovshop, client connecté



README.md sur GitHub : présentation, fonctionnalités et stack technique



innovshop / README.md

Preview Code Blame 119 lines (91 loc) · 4.82 KB

Installation en local

Prérequis : PHP 8.4 ou plus, Composer, Symfony CLI, MySQL ou MariaDB (XAMPP, MAMP...).

```
# 1. Récupérer le projet
git clone https://github.com/Mberben/innovshop.git
cd innovshop

# 2. Installer les dépendances
composer install

# 3. Configurer la base de données dans un fichier .env.local (non versionné)
echo 'DATABASE_URL="mysql://root:@127.0.0.1:3306/innovshop?serverVersion=10.4.28-MariaDB&charset=utf8mb4"' > .env.lo

# 4. Créer la base, les tables et les données de démonstration
php bin/console doctrine:database:create
php bin/console doctrine:migrations:migrate
php bin/console doctrine:fixtures:load

# 5. Lancer le serveur
symfony server:start
```

Adapter `DATABASE_URL` à votre configuration (utilisateur, mot de passe, version : `serverVersion=8.0` pour MySQL 8).

Comptes de démonstration

Email	Mot de passe	Rôle
admin@innovshop.fr	Admin1234!	Administrateur (accès `/admin`)
client@innovshop.fr	Client1234!	Client avec commandes

Emails en local

Par défaut, `MAILER_DSN=null://null` : aucun email ne part réellement. Les emails envoyés sont visibles dans le profiler Symfony (`/_profiler`, requête concernée, onglet **E-mails**).

Déploiement en production

```
git clone https://github.com/Mberben/innovshop.git && cd innovshop

# .env.local de production (jamais versionné)
APP_ENV=prod
APP_SECRET=une_longue_chaine_aleatoire
DATABASE_URL="mysql://utilisateur:motdepasse@hote:3306/base?serverVersion=10.11-MariaDB&charset=utf8mb4"
MAILER_DSN=smtp://utilisateur:motdepasse@serveur-smtp:587

composer install --no-dev --optimize-autoloader
php bin/console doctrine:migrations:migrate --no-interaction
php bin/console doctrine:fixtures:load --no-interaction   # données de démo (optionnel)
php bin/console importmap:install
php bin/console asset-map:compile
php bin/console cache:clear
```

README.md : installation en local, comptes de démonstration et procédure de mise en production

2. Précisez les moyens utilisés :

- PHP 8.5 (Homebrew), Composer, Symfony CLI ; Symfony 8.1 (Security, Form, Validator, Mailer, EventDispatcher)
- Doctrine ORM 3, MakerBundle, migrations, DoctrineFixturesBundle ; MariaDB 10.4 (XAMPP) et phpMyAdmin
- Twig, Bootstrap 5.3 et Bootstrap Icons, AssetMapper, JavaScript natif (fetch)
- EasyAdmin 5, Symfony Mailer (TemplatedEmail), Dompdf, PHPUnit 13, profiler Symfony
- Git, GitHub, GitHub CLI, Trello (Kanban), méthode MoSCoW et story points
- Hébergement alwaysdata (PHP, MariaDB, SSH, HTTPS)
- Assistant d'intelligence artificielle (Claude) : explication des concepts, aide à la rédaction du code et au débogage ; j'ai relu, testé et documenté chaque partie

3. Avec qui avez-vous travaillé ?

J'ai réalisé ce projet seul, en répondant au cahier des charges de Madame Leroy, directrice fictive d'InnovShop (cas fourni par la formation). J'ai suivi l'avancement des user stories dans Trello et sollicité mon formateur sur les choix techniques.

4. Contexte

Nom de l'entreprise, organisme ou association **Learning Campus**

Chantier, atelier, service	Projet InnovShop : boutique en ligne Symfony
Période d'exercice	**Du** 27/10/2026 **au** 28/10/2026

5. Informations complémentaires *(facultatif)*

J'ai rencontré une erreur Doctrine (« A new entity was found through the relationship … ») en enregistrant un produit avec de nouvelles options. Je l'ai résolue en ajoutant cascade: ['persist'] sur la relation, ce qui m'a fait comprendre comment Doctrine suit les objets qu'il doit enregistrer.

Autres difficultés résolues : erreur « Case mismatch » due à un dossier nommé enum alors que le namespace est App\Enum (autoloading PSR-4) ; upload d'images peu fiable dans un sous-formulaire EasyAdmin, résolu par un écran « Images » dédié ; photos de 38 Mo redimensionnées à 1200 px et compressées (1,2 Mo au total).

Évolutions possibles : paiement en ligne (Stripe) et gestion des stocks, quantités dans le panier, vérification du code postal avec l'API geo.api.gouv.fr, intégration continue des tests avec GitHub Actions.

Site en ligne : https://berben.alwaysdata.net/innovshop/ – dépôt https://github.com/Mberben/innovshop

Activité-type 2 — Développer la partie back-end d'une application web ou web mobile sécurisée

Exemple n°3 — *Base de données de la salle de concerts SoundWave (SQL)*

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Compétences mises en œuvre : CP 1 Installer et configurer son environnement de travail · CP 5 Mettre en place une base de données relationnelle · CP 6 Développer des composants d'accès aux données SQL et NoSQL

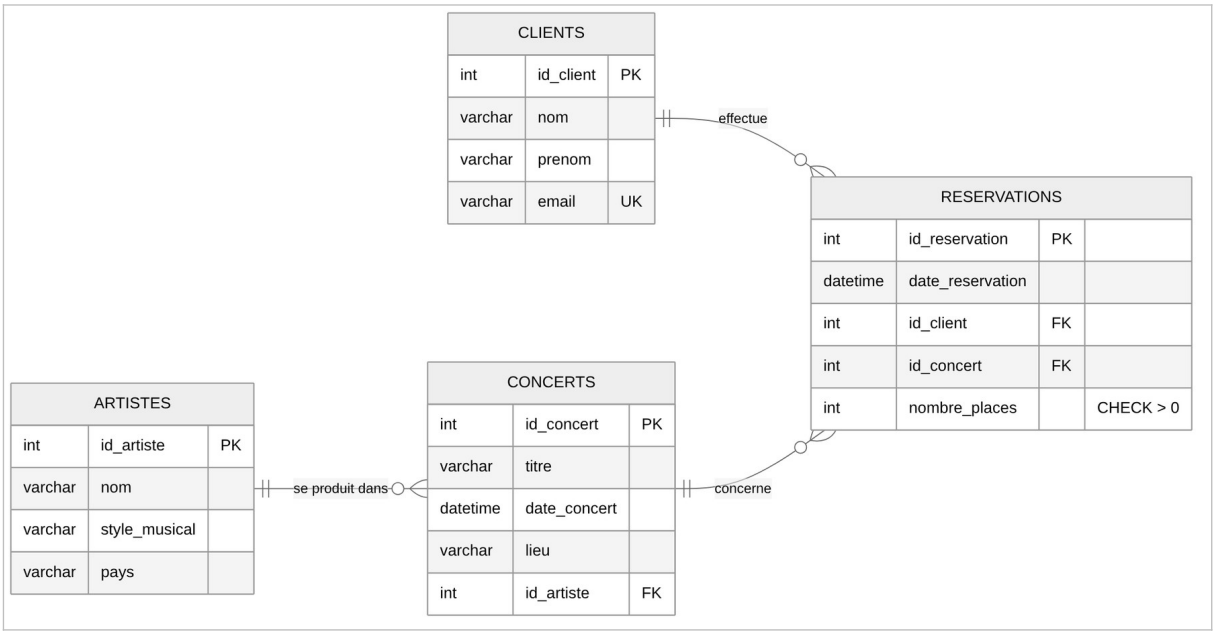
SoundWave est une salle de concerts fictive qui souhaite gérer ses artistes, ses concerts, ses clients et leurs réservations. L'objectif du projet était de concevoir la base de données **sans ORM**, en écrivant moi-même tout le SQL : création, insertion d'un jeu de données et requêtes d'interrogation. J'ai exécuté les scripts sur le serveur MariaDB local de XAMPP, depuis le client **TablePlus** (CP 1).

Modélisation (CP 5)

J'ai identifié quatre entités et leurs relations :

- un **artiste** se produit dans plusieurs **concerts** ; un concert est donné par un seul artiste (1 → n) ;
- un **client** peut réserver pour plusieurs concerts, et un concert reçoit les réservations de plusieurs clients (n → n).

La relation plusieurs-à-plusieurs entre clients et concerts est résolue par la table de liaison **reservations**, qui porte aussi ses propres données : la date de réservation et le nombre de places.



Modèle physique de données de SoundWave

Script de création (CP 5)

Le script 01_creation.sql crée la base en **utf8mb4** (accents et emojis), supprime les tables existantes dans l'ordre inverse des dépendances, puis crée les quatre tables :

- clés primaires en **INT UNSIGNED AUTO_INCREMENT** ;
- **clés étrangères** du côté « plusieurs » de chaque relation, avec le même type que la clé référencée ;
- contrainte **UNIQUE** sur l'email des clients ;
- contrainte **CHECK** (nombre_places > 0) pour refuser une réservation de 0 ou de -3 places ;
- valeur par défaut **CURRENT_TIMESTAMP** pour la date de réservation ;
- moteur **InnoDB**, nécessaire aux clés étrangères. J'ai nommé la colonne date_concert plutôt que date, qui est un mot réservé en SQL.

soundwave/01_creation.sql

```sql
-- 1) Création de la base
--    utf8mb4 = jeu de caractères qui gère les accents et les emojis.
CREATE DATABASE IF NOT EXISTS soundwave
    CHARACTER SET utf8mb4
    COLLATE utf8mb4_unicode_ci;

USE soundwave;

-- 2) On supprime les tables si elles existent déjà.

DROP TABLE IF EXISTS reservations;
DROP TABLE IF EXISTS concerts;
DROP TABLE IF EXISTS clients;
DROP TABLE IF EXISTS artistes;

-- 3) Tables

-- ARTISTES
CREATE TABLE artistes (
    id_artiste     INT UNSIGNED NOT NULL AUTO_INCREMENT,
    nom            VARCHAR(100) NOT NULL,
    style_musical  VARCHAR(50)  NOT NULL,
    pays           VARCHAR(50)  NOT NULL,
    PRIMARY KEY (id_artiste)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

--  CLIENTS
CREATE TABLE clients (
    id_client  INT UNSIGNED NOT NULL AUTO_INCREMENT,
    nom        VARCHAR(100) NOT NULL,
    prenom     VARCHAR(100) NOT NULL,
    email      VARCHAR(150) NOT NULL,
    PRIMARY KEY (id_client),
    -- Un email ne peut appartenir qu'à un seul client :
    UNIQUE KEY uq_clients_email (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

01_creation.sql : création de la base et des tables artistes et clients

soundwave/01_creation.sql

```sql
-- CONCERTS (dépend de artistes)
CREATE TABLE concerts (
    id_concert    INT UNSIGNED NOT NULL AUTO_INCREMENT,
    titre         VARCHAR(150) NOT NULL,
    date_concert  DATETIME     NOT NULL,   -- renommé (Date = mot réservé SQL)
    lieu          VARCHAR(150) NOT NULL,
    id_artiste    INT UNSIGNED NOT NULL,   -- même type que artistes.id_artiste
    PRIMARY KEY (id_concert),
    -- La FK part de concerts (côté "plusieurs") vers artistes (côté "un") :
    CONSTRAINT fk_concerts_artiste
        FOREIGN KEY (id_artiste) REFERENCES artistes (id_artiste)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

--  RESERVATIONS (dépend de clients ET concerts)
CREATE TABLE reservations (
    id_reservation    INT UNSIGNED NOT NULL AUTO_INCREMENT,
    date_reservation  DATETIME     NOT NULL DEFAULT CURRENT_TIMESTAMP,
    id_client         INT UNSIGNED NOT NULL,
    id_concert        INT UNSIGNED NOT NULL,
    nombre_places     INT          NOT NULL,
    PRIMARY KEY (id_reservation),
    -- Les deux FK qui font de "reservations" la table de liaison Client <-> Concert :
    CONSTRAINT fk_reservations_client
        FOREIGN KEY (id_client)  REFERENCES clients  (id_client),
    CONSTRAINT fk_reservations_concert
        FOREIGN KEY (id_concert) REFERENCES concerts (id_concert),
    -- On ne réserve pas 0 ou -3 places.
    CONSTRAINT chk_nombre_places CHECK (nombre_places > 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

01_creation.sql : tables concerts et reservations, clés étrangères et contrainte CHECK

Jeu de données (CP 5)

Le script 02_insertion.sql insère 5 artistes, 5 clients, 6 concerts et 8 réservations, **dans l'ordre imposé par les clés étrangères** : artistes et clients, puis concerts, puis réservations. J'ai volontairement prévu des concerts passés et à venir, et des clients ayant réservé plusieurs concerts, pour pouvoir tester mes requêtes.

soundwave/02_insertion.sql

```sql
USE soundwave;

-- Ordre d'insertion imposé par les clés étrangères :
-- 1) artistes et clients  2) concerts  3) reservations

--  ARTISTES
-- On ne précise pas id_artiste : AUTO_INCREMENT attribue 1, 2, 3...
INSERT INTO artistes (nom, style_musical, pays) VALUES
    ('Daft Punk',   'Électro', 'France'),    -- id 1
    ('Stromae',     'Pop',     'Belgique'),  -- id 2
    ('Angèle',      'Pop',     'Belgique'),  -- id 3
    ('The Weeknd',  'R&B',     'Canada'),    -- id 4
    ('Jul',         'Rap',     'France');     -- id 5

--  CLIENTS
INSERT INTO clients (nom, prenom, email) VALUES
    ('Dupont',  'Jean',   'jean.dupont@email.fr'),    -- id 1
    ('Martin',  'Sophie', 'sophie.martin@email.fr'),  -- id 2
    ('Bernard', 'Lucas',  'lucas.bernard@email.fr'),  -- id 3
    ('Petit',   'Emma',   'emma.petit@email.fr'),     -- id 4
    ('Moreau',  'Hugo',   'hugo.moreau@email.fr');    -- id 5

-- CONCERTS (référencent artistes)
-- (date du jour de référence : 2026-08-24).
INSERT INTO concerts (titre, date_concert, lieu, id_artiste) VALUES
    ('Alive Tour',          '2026-11-15 20:00:00', 'Zénith de Strasbourg',     1), -- futur
    ('Multitude Tour',      '2026-12-05 20:30:00', 'Accor Arena, Paris',       2), -- futur
    ('Nonante-Cinq Live',   '2027-02-20 21:00:00', 'Halle Tony Garnier, Lyon', 3), -- futur
    ('After Hours',         '2026-03-10 20:00:00', 'Paris La Défense Arena',   4), -- passé
    ('Extraterrestre Tour', '2026-06-18 20:00:00', 'Orange Vélodrome, Marseille', 5), -- passé
    ('D.A.N.C.E Night',     '2027-05-30 19:30:00', 'Zénith de Lille',          1); -- futur

--  RESERVATIONS (référencent clients ET concerts)
-- id_client / id_concert doivent exister (sinon la FK refuse).
INSERT INTO reservations (date_reservation, id_client, id_concert, nombre_places) VALUES
    ('2026-08-01 14:30:00', 1, 1, 2),  -- Jean  -> Alive Tour
    ('2026-08-02 09:15:00', 1, 2, 3),  -- Jean  -> Multitude Tour   (Jean a 2 concerts)
    ('2026-08-03 18:00:00', 2, 1, 4),  -- Sophie -> Alive Tour
    ('2026-08-05 11:45:00', 2, 3, 2),  -- Sophie -> Nonante-Cinq     (Sophie a 2 concerts)
    ('2026-08-06 20:10:00', 3, 3, 2),  -- Lucas -> Nonante-Cinq
    ('2026-08-07 16:20:00', 3, 6, 2),  -- Lucas -> D.A.N.C.E Night   (Lucas a 2 concerts)
    ('2026-08-08 13:05:00', 4, 2, 1),  -- Emma  -> Multitude Tour
    ('2026-02-15 10:00:00', 5, 4, 2);  -- Hugo  -> After Hours (concert passé)
```

02_insertion.sql : insertion des données dans l'ordre des dépendances

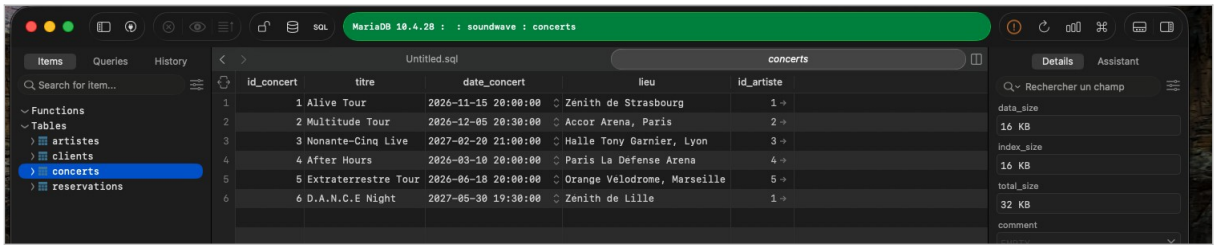


Table concerts dans TablePlus (clé étrangère id_artiste)

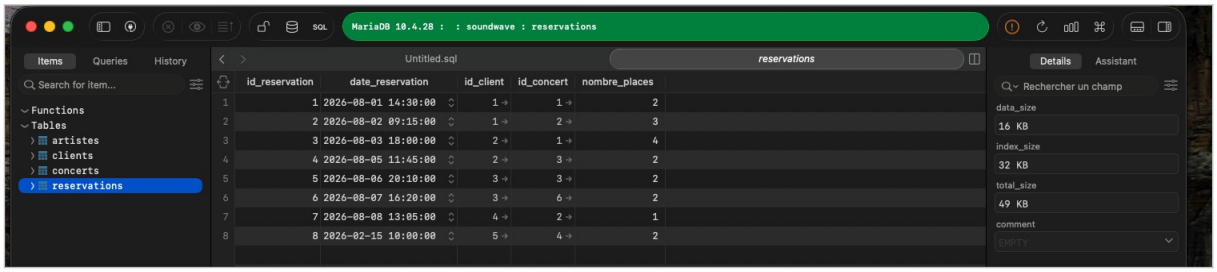
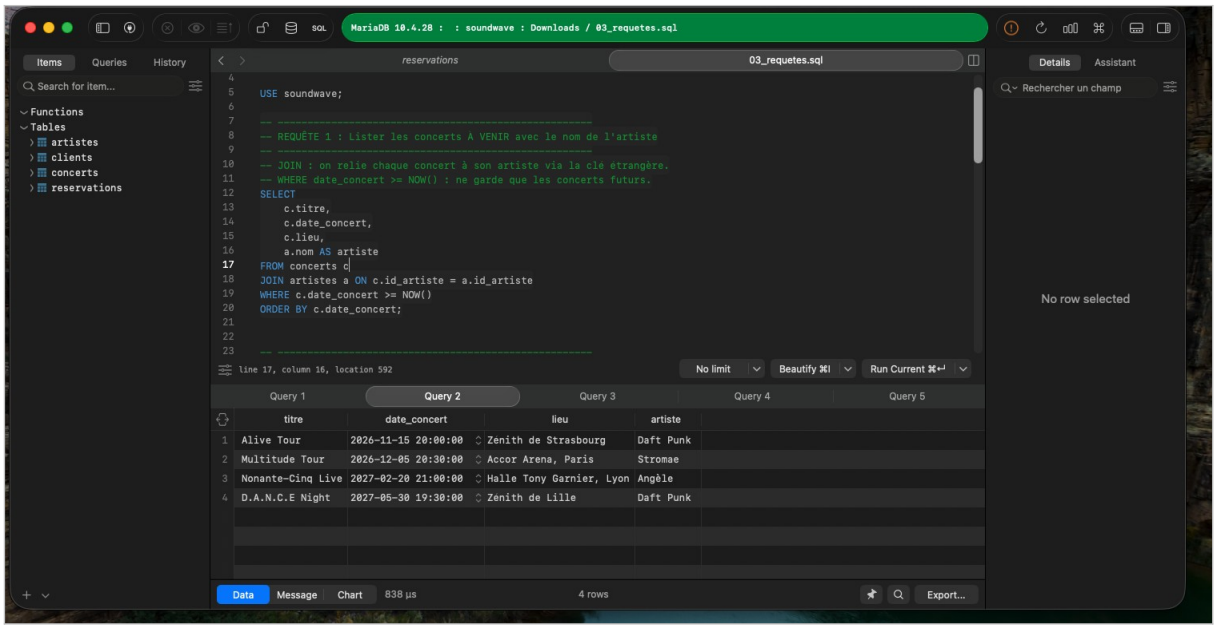


Table de liaison reservations dans TablePlus

Requêtes d'interrogation (CP 6)

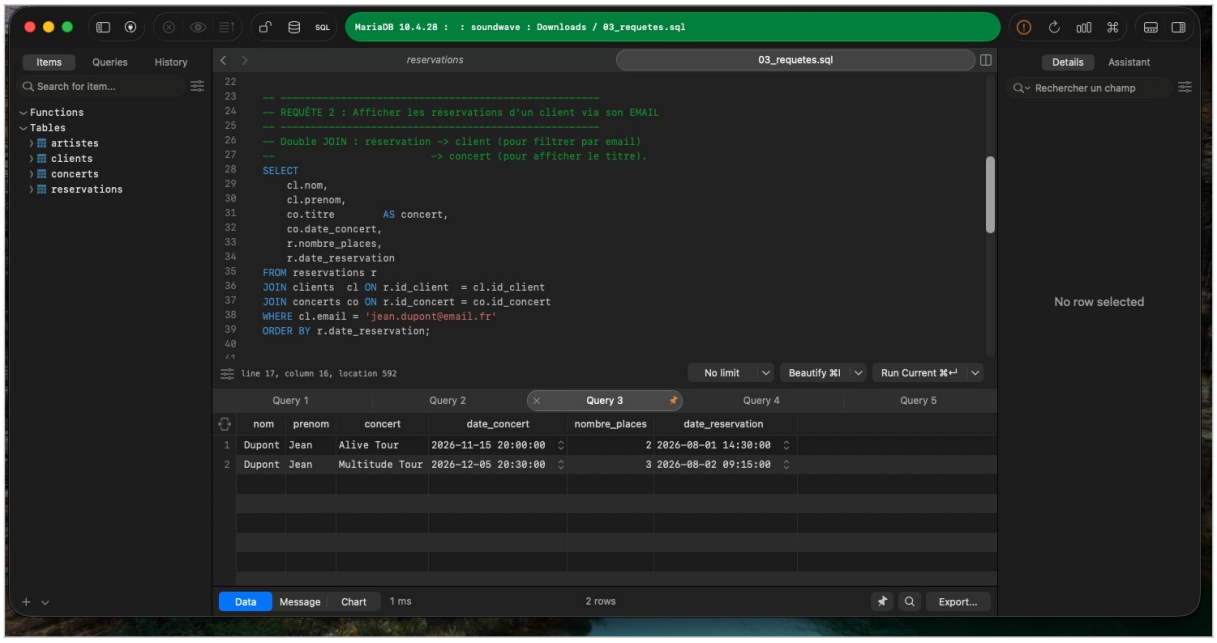
J'ai écrit et testé dans TablePlus quatre requêtes de complexité croissante (fichier 03_requetes.sql) :

1. Concerts à venir avec le nom de l'artiste : une jointure relie chaque concert à son artiste, et WHERE date_concert >= NOW() ne garde que les concerts futurs, triés par date.



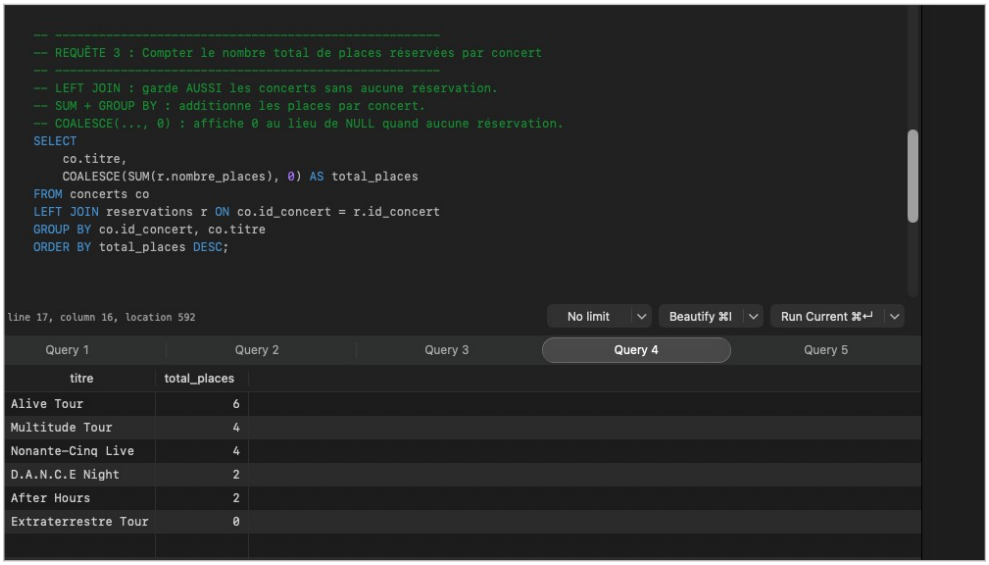
Requête 1 : concerts à venir et leur artiste

2. Réservations d'un client à partir de son email : une double jointure relie la réservation au client (pour filtrer par email) et au concert (pour afficher son titre).



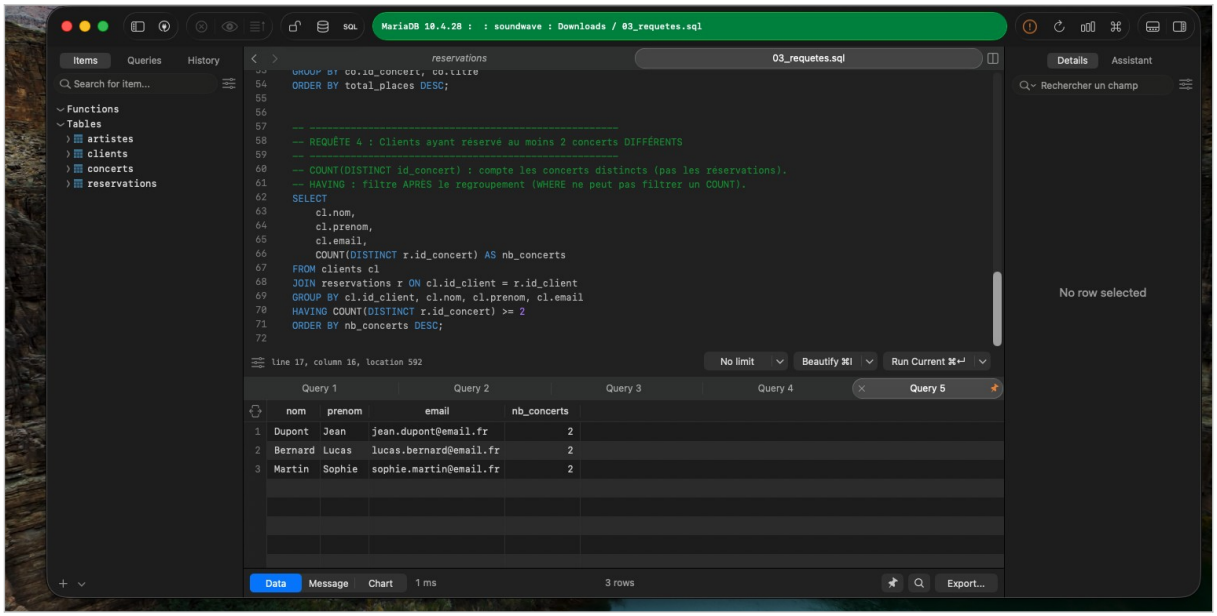
Requête 2 : réservations de Jean Dupont

3. Nombre de places réservées par concert : le LEFT JOIN garde aussi les concerts sans réservation, SUM et GROUP BY additionnent les places par concert, et COALESCE affiche 0 au lieu de NULL.



Requête 3 : total des places par concert (Extraterrestre Tour à 0)

4. Clients ayant réservé au moins deux concerts différents : COUNT(DISTINCT id_concert) compte les concerts distincts et HAVING filtre après le regroupement, car WHERE ne peut pas filtrer sur un COUNT.



Requête 4 : clients ayant réservé au moins deux concerts

Ce travail en SQL direct m'a aidé à comprendre ce que génère un ORM comme Doctrine : le QueryBuilder d'InnovShop produit le même type de jointures et d'agrégations.

2. Précisez les moyens utilisés :

- MariaDB 10.4 (XAMPP)
- TablePlus pour exécuter les scripts et tester les requêtes
- SQL : DDL (CREATE DATABASE, CREATE TABLE, contraintes) et DML (INSERT, SELECT, JOIN, LEFT JOIN, GROUP BY, HAVING, COALESCE)
- Modèle physique de données

3. Avec qui avez-vous travaillé ?

J'ai réalisé ce projet seul, à partir du sujet fourni par la formation ; mon formateur a validé le modèle de données.

4. Contexte

Nom de l'entreprise, organisme ou association	**Learning Campus**
Chantier, atelier, service	Projet SoundWave : conception et interrogation d'une base de données
Période d'exercice	**Du** 03/08/2026 **au** 24/08/2026

5. Informations complémentaires *(facultatif)*

Les trois scripts (01_creation.sql, 02_insertion.sql, 03_requetes.sql) permettent de recréer et de tester la base de

zéro, dans l'ordre.

Titres, diplômes, CQP, attestations de formation

(facultatif)

Intitulé	Autorité ou organisme	Date
BTS Maintenance des systèmes	Ministère de l'Éducation nationale	2023

Déclaration sur l'honneur

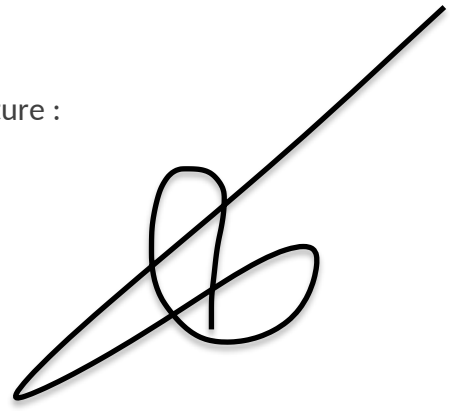
Je soussigné(e) [prénom et nom] *Mathias Berben*,

déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je suis l'auteur(e) des réalisations jointes.

Fait à *Haguenau* le *28/10/2026*

pour faire valoir ce que de droit.

Signature :



Documents illustrant la pratique professionnelle

(facultatif)

Intitulé
Portfolio (HTML/CSS) – https://berben.alwaysdata.net/ et https://mberben.github.io/portfolio/
Site de l'association ACT NOW (WordPress) – https://berben.alwaysdata.net/wordpress/
Site du cabinet du Dr Dupont (PHP/MySQL) – https://berben.alwaysdata.net/cabinet/
Boutique InnovShop (Symfony) – https://berben.alwaysdata.net/innovshop/
Maquettes Figma de la To-Do List – https://www.figma.com/design/O9Z004LEmotAiDlTU61ayK/Projet-To-Do-Mathias
Dépôts GitHub – https://github.com/Mberben/portfolio et https://github.com/Mberben/Todolist

ANNEXES

(Si le RC le prévoit)